TECHNICAL RECONSTRUCTION REPORT · PERSONAL STUDY EDITION

Hey Vayro: a low-latency wake-word activator on an ESP32-S3

The complete engineering history and final state of the PS 26172 prototype — research basis, hardware, firmware, model, server, every validation test, what failed, what was fixed, and what is still open.

Problem statement	SIH 2026 · PS 26172 (ISRO, Department of Space) — "Low Latency and Efficient Voice Activator for Edge Devices" · Category: Hardware · Theme: Smart Automation
Team	DATA_Edgepulse (AUC-SIH26-172041)
System	ESP32-S3 + INMP441 microphone · keyword "Hey Vayro" · int8 DS-CNN on TensorFlow Lite Micro · Wi-Fi/TCP stream · Vosk (Kaldi) ASR server
Reconstructed from	repository `D:\ps26172` at commit `9a6e700` (30 Sep 2026), evidence ledger E-01…E-63, raw run folders under `evidence/`, and the two documents supplied at the start: the PS text and the Track C architecture study
Date	30 September 2026

Every number in this report comes from a file in the repository, and the file is named next to it. Headline measurements were re-computed from the raw logs for this report, not copied from summaries. Where evidence is missing or weaker than a claim would need, the report says so with an explicit label. This is an engineering study document; it deliberately contains no presentation material.

Contents

Executive Technical Summary 5

How to Read This Report 7

Evidence labels 7
Identifiers you will meet 7
Sources that were available, and sources that were not 7
Units and conventions 8
Deliberately left out 8

1 Project Overview and Engineering Objective 9

1.1 The project in one paragraph 9
1.2 The problem being addressed 9
1.3 Real-world use and intended users 10
1.4 Main technical challenges 10
1.5 Expected system behaviour 10
1.6 Input, processing, output 10
1.7 Operating sequence 11
1.8 What was actually achieved 11

2 The Research Foundation and How It Was Implemented 12

2.1 What was supplied, precisely 12
2.2 The research problem the study addresses 12
2.3 The study's method 12
2.4 Key technical principles in the study 13
2.5 Assumptions and constraints in the study 14
2.6 From research to implementation: the traceability table 15
2.7 How the measurements compare with the study's estimates 16
2.8 What the implementation does *not* establish about the research 17
2.9 Which parts are the study's, and which are this project's 17

3 System Requirements and Design Constraints 18

3.1 Where the requirements come from 18
3.2 Functional requirements 18
3.3 Measurable constraints 19
3.4 Software constraints 20
3.5 Hardware constraints 20
3.6 Communication requirements 20
3.7 Environmental and operational constraints 20
3.8 Integration requirements 21
3.9 Requirement compliance summary 21

4 Hardware Inventory and Technical Specifications 22

4.1 Inventory 22
4.2 The microcontroller: ESP32-S3 22
4.3 The microphone: INMP441 23
4.4 Wiring 25
4.5 The interfaces, explained 26
4.6 Clocking and data-rate arithmetic 26
4.7 Power 27
4.8 What remains unverified about the hardware 27

5 Software Stack and Development Environment 28

5.1 Four environments 28
5.2 Firmware toolchain and libraries 28
5.3 Training environment 29
5.4 Host tools and server 29
5.5 How the pieces talk to each other 30
5.6 Compliance tooling 30
5.7 Versions that could not be confirmed 31

6 Complete System Architecture 32

6.1 The big picture 32

6.2 Subsystems and their responsibilities 32
6.3 Tasks and cores 33
6.4 Data flow 33
6.5 Control flow, initialisation and runtime behaviour 35
6.6 Communication between device and server 36
6.7 External dependencies at runtime 37
6.8 Failure handling that is actually implemented 37
6.9 Operational sequence (final system) 37

7 Implementation and Source-Code Walkthrough 39
7.1 Repository structure 39
7.2 Build entry points 39
7.3 Firmware, in execution order 40
7.4 Model pipeline code (summary; details in Chapter 8) 44
7.5 Server code 45
7.6 Host tools 45
7.7 Where the tests live 46

8 Data, Algorithms, Models and the Processing Pipeline 47
8.1 The pipeline at a glance 47
8.2 Feature extraction: turning sound into 40 numbers 47
8.3 The model: a small depthwise-separable CNN 48
8.4 Data 50
8.5 Training 52
8.6 Quantisation and export 52
8.7 Runtime inference and decision 53
8.8 Evaluation methodology 53
8.9 Model versions 54
8.10 The recogniser side 54

9 Hardware–Software Integration and Deployment 56
9.1 Preparing the hardware 56
9.2 Installing the software 56
9.3 Building 56
9.4 Flashing 57
9.5 What exactly is deployed 57
9.6 Configuration and credentials 58
9.7 Memory allocation: where the 245 KiB goes 58
9.8 Debugging and observability interfaces 58
9.9 Deployment restrictions (as built) 59
9.10 A working-deployment checklist 59

10 Complete Experimental and Validation History 60
10.1 Test categories used in this project 60
10.2 Summary of all tests 60
10.3 Phase 0 — Toolchain (26 Sep) 62
10.4 Phase 1 — Physical audio (26–27 Sep) 62
10.5 Phase 2 — Front-end parity (27 Sep) 63
10.6 Phase 3 — Model on the device (27 Sep) 64
10.7 Phase 4 — Streaming and the first real test (27 Sep evening) 65
10.8 Phase 5 — The overnight run (27–28 Sep) 66
10.9 Phase 6 — Product configuration (28 Sep morning) 66
10.10 Phase 7 — Speech recognition (28 Sep afternoon and evening) 67
10.11 Phase 8 — Memory under streaming, and v4 (28 Sep night) 68
10.12 Phase 9 — The final decisions (29 Sep) 69
10.13 Phase 10 — Latency on the final configuration (30 Sep) 70
10.14 What cannot be reconstructed reliably 71

11 Debugging, Failures and Engineering Iterations 72
11.1 Problem 1 — Audio captured one bit late 72
11.2 Problem 2 — The evidence link lost data 72
11.3 Problem 3 — Device features did not exactly match training features 73
11.4 Problem 4 — The model's first int8 export contained unsupported-looking operators 73
11.5 Problem 5 — The synthetic voices said the wrong word 73
11.6 Problem 6 — Validation on synthetic data picked the worse model 73

11.7 Problem 7 — Labelling real keywords automatically was unreliable 74
11.8 Problem 8 — Results misread before checking what was said 74
11.9 Problem 9 — The first product build did not fit in 256 KB 74
11.10 Problem 10 — The speech recogniser heard nothing useful 75
11.11 Problem 11 — The ASR server caused stream overruns 75
11.12 Problem 12 — False triggers in an active household 76
11.13 Smaller problems 76

12 Performance, Resource Usage and Final Results 77
12.1 Memory (RAM) 77
12.2 Flash and model footprint 78
12.3 CPU utilisation 78
12.4 Latency 79
12.5 Detection accuracy 79
12.6 False activations 80
12.7 Streaming reliability and integrity 80
12.8 Speech-recognition quality 80
12.9 Stability 81
12.10 Data overhead 81
12.11 Power 81
12.12 Final results at a glance 81

13 The Most Distinctive Engineering Contributions 82
13.1 How this chapter judges "innovation" 82
13.2 Level A — Conceptual contributions 82
13.3 Level B — Architectural contributions 83
13.4 Level C — Implementation contributions 84
13.5 How the contributions relate to the research 86

14 Final System State and Limitations 87
14.1 Snapshot as of 30 September 2026 87
14.2 What is fully implemented 88
14.3 What is partially implemented 88
14.4 Physically tested versus tested in software only 88
14.5 What has not been verified 88
14.6 Known limitations 88
14.7 Dependencies required to run the system 89
14.8 What would have to change for larger-scale deployment 89

15 Future Technical Development 90
15.1 Reduce household false triggers without losing new speakers 90
15.2 Use the operating curve that already exists in the recordings 90
15.3 Convert the model to stateful streaming 90
15.4 Measure what is still estimated 90
15.5 Measure and reduce power 91
15.6 Make the device deployable 91
15.7 Bound the heap and automate the checks 91
15.8 Compress the stream for poor networks 91
15.9 Improve command recognition on the server 91
15.10 Prepare the model for commercial use 92

16 Technical Glossary and Study Notes 93
16.1 Glossary 93
16.2 How to understand this project in 15 minutes 96
16.3 What to study deeply 96

A Source and Evidence Index 98
A.1 Documents 98
A.2 Evidence ledger index 98
A.3 Decisions index 100

B Discrepancies Found During This Reconstruction 101

C Completeness Audit 102

D Command Reference 104

Executive Technical Summary

What was built. A two-part voice activation system. An ESP32-S3 microcontroller with an INMP441 digital microphone listens all the time. Every 10 ms it turns the last 30 ms of sound into 40 log-mel numbers, and every 60 ms a 10 665-parameter int8 depthwise-separable CNN (DS-CNN) scores the last 1.02 s of those numbers for the custom keyword **"Hey Vayro"**. When the mean of three consecutive scores reaches 0.80, the device starts streaming audio — beginning 571 ms *before* the detection, so the keyword's tail and anything said straight after it are included — over a TCP connection that was opened at boot. A Python server on a PC decides when the speaker has finished (energy-based voice-activity detection), and transcribes the command with the open-source Vosk (Kaldi) recogniser. Nothing leaves the device until a detection happens. VERIFIED

Final configuration. Product firmware (`firmware/sdkconfig.product`: decisions DEC-22 and DEC-24), model **v2b** at threshold 0.80 (SHA-256 `f7c913efdbb28f64…`, decision DEC-26), ESP-IDF 5.5.5, esp-tflite-micro 1.4.1 with esp-nn 1.4.1, and the threaded ASR server with Vosk `en-in-0.5`. VERIFIED

Table 0.1. Headline results on the final configuration (all re-checked against the raw files)

Quantity	Result	Conditions	Evidence
Peak internal RAM (S4, strictest scope)	**250 899 B = 245.0 KiB**	while streaming; PSRAM disabled; static 139 695 B + peak heap 111 204 B	E-60 VERIFIED
Idle CPU, busiest core (core 1)	**9.32 % mean, 9.51 % max**	18 windows of 10 s, Wi-Fi + TCP up, FreeRTOS run-time stats	E-60 VERIFIED
Idle CPU, both cores averaged	**5.53 % mean**	same run	E-60 VERIFIED
Model	28 024 B int8, 10 665 parameters, 212 584 MACs, arena 13 404 B	weights stay in flash	E-24, E-60 VERIFIED
Keyword end → audio at the ASR server (L2)	**138 ms median, 196 ms p95**	T1: n = 12, one speaker, LAN, capture term 10 ± 7 ms estimated	E-63 VERIFIED, part INFERRED
Detection → first audio at the server	18 ms median (p95 30 ms)	same	E-63 VERIFIED
Keyword detection, speaker 1, held-out session	**22 / 25** prompts (0.5–2 m)	live on device; speaker 1 is also in the training data	E-35 VERIFIED
Keyword detection, new speaker	**20 / 25** prompts	speaker never used in training	E-59 VERIFIED
False triggers, quiet night	**1 in 5 h** (0.2 /h)	declared held-out hours	E-39 VERIFIED
False triggers, household with TV and talk	**39 in 2.47 h (15.8 /h)**	offline replay equal to the device's live triggers	E-51 VERIFIED
Streamed audio integrity	23/23 streams bit-exact; 30/30 and 12/12 streams without loss	E-44, E-60, E-63	VERIFIED
Train/deploy feature parity	12 000 / 12 000 frames bit-exact	device vs host twin	E-21 VERIFIED

The engineering that mattered most. Six results shaped the final system and are explained in Chapters 10–13:

1. The microphone data was captured one bit late with the textbook I2S framing; a measured alignment sweep switched the firmware to MSB-justified framing (E-18).
2. Device features differed from training features in 3 of 5 998 frames because the chip's maths library rounds differently from the PC's; the host's tables are now copied into the device, making them bit-identical (E-21, DEC-16).
3. The first "product" build used 260.2 KiB — over the limit — once instruction code in internal RAM was counted; configuration trims brought the streaming peak to 245.0 KiB (E-42, E-52, E-53, E-60).

4. The server's speech recogniser blocked its own network reads, which backed audio up into the device and caused 3 of 31 streams to overrun; moving recognition to its own thread fixed it (E-58, E-60).
5. A fourth model (v4) passed its pre-declared selection rule but failed badly on the first new speaker (14/25 vs 20/25), so the team shipped v2b in a documented override (E-59, DEC-26).
6. The final latency test on the exact shipped configuration gave 138 ms, replacing an earlier 95 ms figure measured on a different build without the recogniser running (E-37, E-63).

Main limitations. False triggers with TV or family speech (15.8 per hour) are the largest open problem. Only two real speakers were tested. The busiest core's idle margin is about half a percentage point, and while streaming it reaches 10.5 %. Part of the latency (the "capture term") is estimated, not measured, and no logic analyser was available. Power was never measured. The server address is compiled into the firmware, and the firmware waits up to 30 s for a USB host at boot. The ASR's word accuracy is that of an off-the-shelf model (publisher WER 36 %). **This is a working, physically tested prototype, not a production-ready product** (Chapter 14).

How to Read This Report

Evidence labels

Every statement that a reader might need to trust is marked with one of six labels. They are stricter than normal writing, on purpose.

Table 0.2. Evidence labels used throughout

Label	Meaning in this report
VERIFIED	Checked directly during this reconstruction: read in the source code, or re-computed from the raw run files.
DOCUMENTED	Stated in the project's own records (evidence ledger, state record, commit messages) but not re-derived for this report.
INFERRED	A reasonable interpretation of the evidence that no file states directly. The reasoning is given.
UNVERIFIED	The evidence is not enough to confirm the claim.
PLANNED	Proposed in a design document or plan but not built.
FUTURE	Suggested in this report as later work (Chapter 15). Not built.

The project's own ledger (`docs/EVIDENCE_LEDGER.md`) uses a second vocabulary — PROVEN, PRELIMINARY, ESTIMATED, PROPOSED, BLOCKED, NOT RUN — and the supplied architecture study uses tags such as [HW], [EST] and [MEAS]. Those words appear in quotations from those documents; the six labels above are this report's own verdicts.

Identifiers you will meet

- **R1–R10** — the ten requirements taken from the problem statement (Chapter 3).
- **M0–M6** — the milestones of the build plan (toolchain, audio, front end, local KWS, trigger, stream, ASR).
- **EXP-n** — experiments defined in the architecture study (EXP-1 memory, EXP-2 CPU, …).
- **E-nn** — rows of the evidence ledger; each points to raw files.
- **DEC-nn** — implementation decisions recorded in `docs/PROJECT_STATE.md`; **C-nn** — conflicts and open issues recorded there.
- **X-nn**, **P-nn**, **H-nn**, **M-nn**, **AMB-nn** — failure modes, microphone facts, SoC facts, module facts and ambiguities as numbered in the architecture study.
- **S01**, **S02** — the two real speakers who recorded test sessions. **v0…v4** — the six trained models (v2a and v2b are two variants of v2).

Sources that were available, and sources that were not

Available and used: the problem-statement PDF (`D:\sih final\sih 2026 (1).pdf`); the architecture study `PS26172_TRACK_C_HARDWARE_ARCHITECTURE.md` (1 810 lines, dated 12 Sep 2026); the complete git repository (50 commits, 26–30 Sep 2026); the evidence ledger, state record, SBOM and consistency sheet; every raw run folder under `evidence/`.

Not available: the "Master Spec", "Track A" and "Track B" documents that the architecture study cites throughout; the vendor datasheets it quotes (their facts are used here only as the study quotes them); any official SIH guidelines. No peer-reviewed research paper was supplied — the architecture study is the research document this project was built from (Chapter 2).

Units and conventions

- **KiB** = 1 024 bytes and **kB** = 1 000 bytes. The PS says "256KB" without defining it, so memory is always checked against both 262 144 B and 256 000 B.
- **"Core 1 busy"** is 100 % minus the share of time core 1 spent in its idle task, measured by FreeRTOS over 10 s windows. **C3** is the mean of the two cores.
- Times of day are local (IST). "p95" is the 95th percentile.
- File paths are relative to the repository root `D:\ps26172` unless they start with a drive letter.

Deliberately left out

This is not a presentation. The submission slides, the demonstration video and any advice on presenting are excluded. Where the demonstration run produced technical evidence (E-61), its measurements are reported like any other test.

CHAPTER 1

Project Overview and Engineering Objective

1.1 The project in one paragraph

Think of a smart speaker, but split in two. A small, cheap chip sits in the room and does only one job: it waits to hear one particular phrase, **"Hey Vayro"**. It never sends the room's sound anywhere while it waits. When it hears the phrase, it immediately starts sending the audio — including a little of what came just before, so nothing is cut off — to a more powerful computer that works out what the person asked for ("turn on the light"). The chip is an **ESP32-S3** microcontroller with a digital **INMP441** microphone; the computer is a PC running an open-source speech recogniser. The project's job was to make the chip's part small, fast, accurate and provably within the limits the problem statement sets. VERIFIED

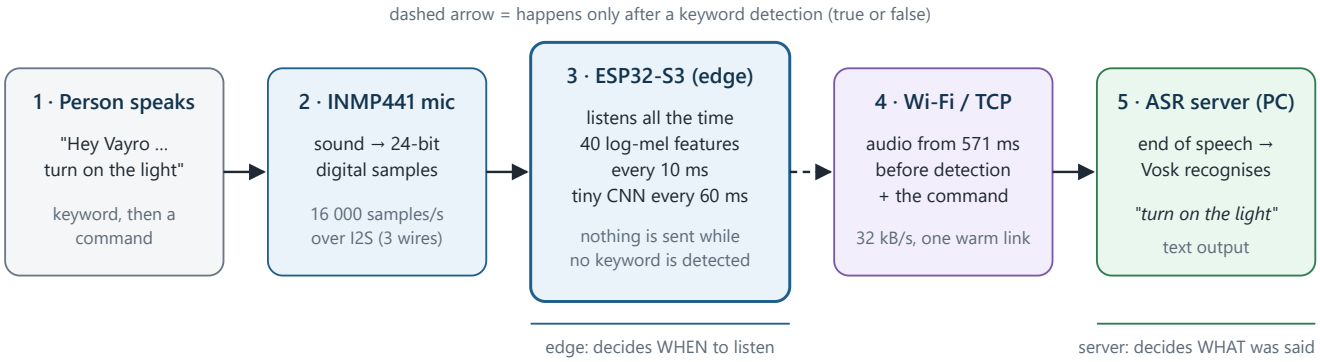


Figure 1.1. The whole system in five steps. Everything left of the dashed arrow runs continuously on the ESP32-S3; everything right of it runs only after a detection.

1.2 The problem being addressed

The problem statement (PS 26172, from ISRO) describes a common situation in voice-controlled devices. If a device sends *all* sound to a cloud recogniser, three things go wrong: it costs a lot (the server processes silence and background noise all day), it invades privacy (the room is always being recorded somewhere else), and it is slow (every request waits for the network). The PS asks for a **hybrid**: the edge device handles the "wake-up" moment locally, and the cloud does the heavy recognition only after that. VERIFIED PS text

Technically, the PS asks for three things (full list in Chapter 3):

1. A **keyword spotting (KWS)** model — a small neural network that recognises one phrase — running *locally* on a low-power device.
2. On detection, **streaming the audio that follows** to a remote **automatic speech recognition (ASR)** server, with minimal data overhead and latency.
3. Doing this under hard limits: **less than 256 KB of RAM and less than 10 % CPU while idle-listening**, open-source tools only, no vendor wake-word SDK, and a keyword that the team trained itself.

> **IN SIMPLE WORDS**
>
> "Keyword spotting" and "speech recognition" are different jobs. Keyword spotting answers one yes/no question over and over ("was that 'Hey Vayro'?"). Speech recognition turns any speech into words. The first is small enough for a microcontroller; the second is not. This project runs the first on the chip and the second on a PC.

1.3 Real-world use and intended users

The PS gives the setting only as "voice-controlled IoT" under the theme **Smart Automation**; it names no user group or deployment environment. The project did not define a specific deployment either. Everything tested ran in one home: a desk, a household with a television and family members, and a home Wi-Fi network. Application ideas that appear in the team's documents (laboratory control rooms, assistive use, home automation) are PLANNED ideas, not tested use cases. VERIFIED no deployment study exists in the repository

1.4 Main technical challenges

Table 1.1. The challenges that shaped the design

Challenge	Why it is hard	Where it is handled
Always-on listening inside 256 KB RAM and 10 % CPU	The microphone, feature extraction and model run 100 times a second forever; Wi-Fi and the TCP/IP stack also need RAM	Chapters 7, 12
"Near-zero false activations"	A household is full of speech-like sound (TV, conversation, singing) and the model is tiny	Chapters 8, 10, 12
A brand-new keyword with almost no real recordings	No public dataset says "Hey Vayro"; the team had one speaker at first	Chapter 8
Train/deploy mismatch	A model trained on features computed on a PC fails silently if the chip computes them even slightly differently	Chapters 7, 13
Not losing the start of the command	The chip only knows the keyword was said *after* it ends; by then the user may already be speaking	Chapters 6, 7
Measuring latency honestly	"Keyword end → ASR receives audio" spans two computers with different clocks	Chapters 10, 12
Open-source compliance	The ESP32's Wi-Fi driver is closed source; vendor wake-word SDKs are the obvious shortcut and are forbidden	Chapters 2, 5

1.5 Expected system behaviour

In words, the system is meant to behave like this: while nobody says the keyword, the device listens, computes, and sends nothing but occasional small clock-synchronisation packets. When someone says "Hey Vayro" and then a command, the device detects the keyword within a fraction of a second, and the server receives audio that starts shortly before the detection and continues until the speaker stops, then prints what was said. A second keyword said within two seconds of the first is ignored (the "refractory" period), and only one stream is open at a time.

1.6 Input, processing, output

Table 1.2. What goes in, what happens, what comes out

Stage	What it is in this system
Input	Sound pressure at the INMP441 microphone, sampled 16 000 times a second as 24-bit numbers
Processing on the chip, every 10 ms	160 new samples → 16-bit conversion → stored in a 0.75 s ring buffer → 40 log-mel "features"
Processing on the chip, every 60 ms	the last 100 feature frames (1.02 s) → int8 neural network → probability of "Hey Vayro" → average of the last 3 probabilities compared with 0.80
Output 1 (chip)	a detection event: telemetry line `@KW`, a pulse on GPIO15, and a stream request
Output 2 (network)	audio packets: from 571 ms before the detection until the server says the speaker has stopped
Processing on the server	end-of-speech detection, 150 Hz high-pass filter + level correction, Vosk speech recognition
Output 3 (server)	the recognised command text, plus WAV and JSON files recording exactly what arrived and when

1.7 Operating sequence

1. The board is powered from the PC's USB port. The firmware starts, waits for the PC's capture tool to open the port (up to 30 s), prints its identity and memory state, and checks the microphone.
2. It builds the feature extractor and the neural network, joins Wi-Fi, and opens one TCP connection to the server at a fixed address. It keeps that connection alive with a clock-synchronisation ping every 10 s.
3. From then on, core 1 processes audio in 10 ms steps without stopping; core 0 handles the network.
4. When the averaged score reaches 0.80, the device queues a stream request and carries on listening.
5. The streaming task sends a START message, then the 571 ms of stored audio as fast as the link allows, then new audio every 40 ms.
6. The server watches the level of the incoming audio. After 0.7 s of quiet (or 1.5 s with no speech at all) it tells the device to stop. The device sends END with its own statistics.
7. The server finishes recognition in a separate thread and prints and stores the transcript. The device is already listening for the next keyword.

1.8 What was actually achieved

By 30 September 2026 the complete chain existed on real hardware and had been measured (Table 0.1 in the Executive Summary). The final firmware fits the strict RAM reading with about 5 kB to spare, idles below 10 % CPU on each core, detects the keyword with 88 % recall on a held-out session of the training speaker and 80 % on a new speaker, triggers falsely about once in five quiet hours but about 16 times an hour when a television is on, and delivers audio to the server a median 138 ms after the keyword ends. Chapters 10 to 14 give the evidence and the limits of each of these statements.

CHAPTER 2

The Research Foundation and How It Was Implemented

2.1 What was supplied, precisely

The project was started from two documents, both in `D:\sih final\`:

- **The problem statement** (`sih 2026 (1).pdf`, one page). It is the only *binding* source.
- **"PS26172_TRACK_C_HARDWARE_ARCHITECTURE"** (a 1 810-line Markdown study, dated 12 Sep 2026, written in the role of a "Principal Edge-Systems Architect"). This is the research document the implementation follows. It is not a peer-reviewed paper: it is an engineering research study that combines datasheet facts, numbers from open-source projects, published keyword-spotting literature (through a companion document called "Track B"), and its own calculations. VERIFIED

The study says it was written from three further documents — a "Master Spec", "Track A (evidence base)" and "Track B (technical intelligence)" — and from the ESP32-S3, ESP32-S3-WROOM-1 and INMP441 datasheets. **None of these was provided.** Their identifiers (AMB-01, ASM-07, F2, I9 …) appear in this report only as the study quotes them, and any fact the study attributes to them is at most DOCUMENTED. VERIFIED PROJECT_STATE.md §1 records the same

The report also cites one external paper that nobody supplied: Zhang et al., "Hello Edge: Keyword Spotting on Microcontrollers" (arXiv:1711.07128, 2017), which introduced the depthwise-separable CNN (DS-CNN) family the final model belongs to. It is background, not a source of any number used here.

2.2 The research problem the study addresses

The study's question is narrow and physical: *can an ESP32-S3 with an INMP441 microphone host a PS 26172-compliant system, where are the real limits, and how should it be proven?* Its central argument, inherited from Track B, is that PS 26172 is **"a false-activation problem wearing an accuracy problem's clothes"**: a wake word that fires on television speech is useless however well it scores on a benchmark. It therefore insists on measuring false activations per hour of real audio, not "accuracy". VERIFIED Track C, closing statement and §5.1

2.3 The study's method

The study is disciplined about the difference between *known* and *estimated*. Every statement carries a tag — [SIH] requirement, [HW] datasheet fact, [LIB] verified library fact, [ASM] assumption, [EST] estimate, [MEAS] measurement, [PROP] proposal — and it states plainly that it contains **zero measurements**. It then:

1. extracts the hardware facts with page citations (40 SoC facts H-01…H-40, 26 microphone facts P-01…P-26);
2. builds budgets before any code exists: memory in five nested scopes **S0–S4**, CPU in three scopes **C1–C3**, latency in three definitions **L1–L3**;
3. maps 12 candidate model families against the PS's real criteria (§5), and 10 runtimes against the open-source rule (§6);
4. proposes three complete architectures (A minimal-robust, B best trade-off, C lowest latency) and a trade-off matrix;
5. lists 25 failure modes (X1–X25), a 7-milestone plan (M0–M6) with binary exit criteria, and 12 experiments (EXP-1…EXP-12), several of them "kill conditions" that could invalidate the platform;

6. ends with a build blueprint: repository layout, subsystem split, configuration policy, pin map, data formats and boot sequence. VERIFIED

2.4 Key technical principles in the study

This section explains the study's main ideas in plain language, because the whole implementation follows from them.

2.4.1 The microphone sets hard rules

- **The INMP441 only speaks one I2S dialect.** It needs exactly 64 bit-clock pulses per stereo frame, so every sample travels in a 32-bit slot holding a 24-bit value. At 16 kHz the bit clock is 64 × 16 000 = **1.024 MHz**. There is no 16-bit mode, so the firmware must convert. (P-02, P-03)
- **Its low-pass filter scales with the sample rate.** The flat pass band ends at 0.423 × fs, which is **6 768 Hz at 16 kHz**. Standard keyword front ends use frequencies up to ~7.5 kHz, so the top bands would see a microphone roll-off that training audio does not have. The study's fix (option O1) is to stop the mel filterbank at **6 500 Hz** in both training and firmware. (P-16, §2.3b)
- **It has a fixed delay** of 17.2 samples (1.078 ms at 16 kHz), and outputs zeros for 2^18 clock cycles after power-up (≈ 256 ms), so the design includes a 300 ms `MIC_SETTLE` state. (P-15, P-19)

2.4.2 Memory: the model is not the problem

The study's memory budget is its most important finding. Estimated peak use was about **227 KiB** without encryption. The model and its working memory were only **17 %** of that; the Wi-Fi driver, TCP/IP stack and operating system were **58 %**. Its advice: "optimisation effort should follow the bytes, not the intuition". It also argued that **PSRAM must be disabled** — using the board's extra 8 MB of external RAM would make the 256 KB limit meaningless — and that a TLS handshake at trigger time would break both the memory and the latency budget. DOCUMENTED Track C §3.5–§3.7; every number there is an estimate

2.4.3 CPU: use the vector unit and never ship reference kernels

The study estimated the whole always-on pipeline at **≈ 2.8 % of one core** at 240 MHz, with the FFT (the step that turns sound into frequencies) as the biggest single cost, and warned that forgetting the optimised kernels (esp-dsp for FFT, esp-nn for the network) would multiply the cost by 5–20×. It proposed measuring CPU three ways, with the MLPerf Tiny GPIO duty-cycle method as the primary instrument. DOCUMENTED §4

2.4.4 The model: shortlist, and train from scratch

After mapping 12 model families, the study concluded that CPU is not the discriminator on this chip — every candidate fits. What matters is whether TensorFlow Lite Micro has kernels for every operation, whether there is any false-activation evidence, and how much streaming state the model needs. It shortlisted a microWakeWord-style reference, a TC-ResNet8-style primary, and a dilated-convolution "false-activation hardened" model, to be compared on detection-error curves. It insisted on **training from random initialisation** so the PS's ban on pre-trained assistant keywords could never be argued. DOCUMENTED §5, §6.4

2.4.5 Compliance: two traps

- **ESP-SR / WakeNet**: Espressif's own speech SDK is one line of configuration away on this chip, is a vendor wake-word SDK with a "Custom" licence, ships binary models, and includes "Alexa" as a wake word — so it breaks three PS rules at once. The study required an **automated build gate** against it (failure mode X25). DOCUMENTED §6.3
- **The Wi-Fi blob**: ESP-IDF is open source, but its Wi-Fi/PHY libraries are closed binaries. The study's position: this is a radio driver, not a voice-activation SDK, it is outside the KWS pipeline, and it must be **disclosed** in a software bill of materials rather than hidden. DOCUMENTED §6.5

2.4.6 Buffering: derive the pre-roll, share one ring

The device only fires *after* the keyword ends, so it must keep recent audio. The study derived the minimum stored audio as detection lag (≤ 374 ms) + 250 ms lead-in + 100 ms scheduling jitter = **724 ms**, rounded to a **1.0 s ring** (32 000 bytes), with pre-computed smaller rows for shorter smoothing windows (for a 5-score window: 571 ms in a 0.75 s ring). It also proposed a **single ring buffer with several read positions** instead of three copies, saving about 34 KB. DOCUMENTED §8.3, §8.4

2.4.7 Transport: open the connection at boot

Opening a TCP connection (and certainly a TLS session) when the keyword fires puts a network round trip on the measured path. The study chose **plain TCP opened at boot and kept warm**, 20 ms PCM frames, an 8-byte header, and an OVERRUN marker so that a stream the device could not keep up with ends short rather than corrupted. DOCUMENTED §8.5, §9

2.4.8 Latency: three definitions, because the PS is ambiguous

"The time delta between the keyword ending and the cloud ASR receiving the audio stream" can mean detection → server (**L1**), acoustic end of the keyword → first audio at the server (**L2**, the literal reading), or keyword end → the server holding audio up to real time (**L3**). The study instrumented all three and estimated **L1 ≈ 13 ms, L2 ≈ 205 ms**. DOCUMENTED §10

2.4.9 Decision logic and the refractory period

The study proposed averaging the last 5 scores with a 0.95 threshold, and — critically — a **2 s refractory period**: without it, one spoken keyword whose score stays high for 200 ms would produce six detections, and any false-activation count would be meaningless (X14). DOCUMENTED §8.6

2.4.10 The parity rule

A single JSON file defines the front end; the firmware compiles it, the training code reads it, and a "golden vector" test must show the device and the PC produce **bit-identical** features from the same audio. The file's hash is embedded so that a model trained with a different front end refuses to load. DOCUMENTED §7.3

2.4.11 The recommended architecture

The study's decision was **"Architecture A as the built spine, with a measurement-gated promotion path to Architecture B"**: build the shared skeleton (capture, front end, ring, transport, server, instrumentation) once, start with a small 1-D DS-CNN, and swap only the model and decision parameters if measurements justified it. Architecture C (speculative streaming, UDP) was not to be built. DOCUMENTED "Architecture decision"

2.5 Assumptions and constraints in the study

Table 2.1. The study's working interpretations of ambiguous PS wording

Ambiguity	Study's interpretation	What the implementation did
AMB-01: what "256 KB RAM" covers	strictest: peak total device SRAM — data, heap, stacks, network, DMA — with PSRAM off	measured "S4" = static internal RAM (incl. code in IRAM) + peak heap; checked against 256 000 B and 262 144 B VERIFIED
AMB-02: what "10 % CPU" means	duty cycle, per core and system-wide, mean and p99 over ≥ 10 min	FreeRTOS run-time stats per core and averaged, 12 min run; GPIO method not possible (no analyser) VERIFIED
AMB-06: latency end points	measure L1, L2, L3	L1 and L2 measured; L3 not computed VERIFIED
AMB-09: scope of "open source only"	every firmware component OSI-licensed; disclose the Wi-Fi blob	SBOM discloses 9 closed Wi-Fi/PHY libraries + boot ROM VERIFIED
AMB-13/14: stream start and end	start before the keyword end; server ends the stream, device has a hard timeout	571 ms pre-roll; server VAD sends END; 10 s device timeout VERIFIED

Ambiguity	Study's interpretation	What the implementation did
Hardware platform	ESP32-S3 is a choice, not a requirement; "the platform loses, not the requirement"	ESP32-S3 kept; EXP-1 did not trigger the kill condition VERIFIED

2.6 From research to implementation: the traceability table

This is the core of the chapter. For each important idea in the study: what it proposed, whether it was adopted, how, whether it changed, why, and how the result was checked.

Table 2.2. Research idea → implementation → validation

#	Study proposal	Adopted?	How it was implemented	Change and reason	Validation
1	ESP32-S3 at 240 MHz, PSRAM disabled	Yes	`sdkconfig.defaults`: 240 MHz, `CONFIG_SPIRAM` not set; firmware prints `psram=disabled_in_sdkconfig`	board has 8 MB PSRAM in the package; left off as the study said	`@MEM`, `@BOOT emb_psram=0` in every run VERIFIED
2	INMP441 on I2S0, 32-bit × 2 slots, BCK 1.024 MHz, Philips framing	Partly	`audio_capture.c`, 16 kHz, 32-bit slots, 4 × 160-frame DMA	**Philips → MSB-justified**: measured one-bit misalignment on this board (DEC-06)	alignment sweep E-18 VERIFIED
3	Pin map BCK 4 / WS 5 / SD 6	No	`sdkconfig.board`: BCK 6 / WS 5 / SD 4	follows the existing wiring (C-05)	audio live E-02 VERIFIED
4	int16 = bits 23:8 (word >> 16)	Yes	`reader_task`: `(int16_t)(raw[i] >> 16)`	level concern (C-07) measured and closed (DEC-09)	E-06 DOCUMENTED
5	MONO slot mode may halve DMA traffic (EXP-10)	Tested, used	mono RX with the measured mic slot	—	63 976 vs 127 963 B/s, E-05 VERIFIED
6	MIC_SETTLE ≥ 300 ms for power-up zeros	Kept	stereo probe discards 300 ms	no zero-blanking seen; a DC transient instead, settles in-band within 100 ms (DEC-10)	E-04, E-19 DOCUMENTED
7	microfrontend: 30 ms / 10 ms, 40 mel, **125–6 500 Hz**, noise reduction + PCAN, int8	Yes	vendored TFLM microfrontend, all parameters from `model/frontend.json`	—	parity E-21 VERIFIED
8	512-pt real FFT via esp-dsp, **float**	No	microfrontend's own fixed-point KissFFT	bit-exact parity with training needs the same FFT code on both sides (C-02)	CPU measured instead (E-23, E-56) VERIFIED
9	Single `frontend.json` + golden-vector parity test + hash check	Yes (manual, not CI)	`gen_frontend_config.py`, `psfe.py`, `parity_test.py`, SHA in firmware and model	a libm rounding difference required host tables on the device (DEC-16)	12 000/12 000 bit-exact VERIFIED
10	1-D DS-CNN, **stateful streaming**, inference every 30 ms	Partly	DS-CNN with "valid" convolutions, **windowed** (whole 1.02 s window) every **60 ms**	deadline risk of int8 streaming state in TFLM (DEC-11, user-approved)	CPU still < 10 % (E-56, E-60) VERIFIED
11	Shortlist of 3 model families compared on DET curves (EXP-8)	No	one family (DS-CNN) with six data/weighting variants	descoped for time (PROJECT_STATE §7)	— VERIFIED
12	Train from random init; no pre-trained keyword model	Yes	`train.py` builds layers from scratch; seed 26172; no weight loading	—	code review E-14 VERIFIED
13	TFLite Micro + esp-tflite-micro + esp-nn, pinned	Yes	`idf_component.yml` `==1.4.1`; `dependencies.lock` hashes	—	device output = host reference, 1 984/1 984 (E-22) VERIFIED
14	No ESP-SR anywhere; automated gate	Yes	`tools/forbidden_deps.py` runs on every build	—	"forbidden_deps: PASS" in build logs VERIFIED
15	End-of-keyword labels ± 160 ms with masking	Yes (variant)	positives = windows ending 0–160 ms after the keyword end; prefix windows negative; the rest masked	window is 0…+160 ms, not ±	lag and prefix behaviour, E-25 DOCUMENTED
16	Smoothing window 5, threshold 0.95	Changed	mean of **3** scores, threshold **0.80**	chosen by a declared rule on validation data (selection_v2_outcome.json)	test sessions E-35, E-59 VERIFIED

#	Study proposal	Adopted?	How it was implemented	Change and reason	Validation
17	2 s refractory before any false-activation count	Yes	`kws_on_frame`: 2 000 ms	—	live = offline replay, E-26 DOCUMENTED
18	Unified ring, 1.0 s; pre-roll 624 ms	Yes, then trimmed	`audio_ring.c`; product ring **0.75 s**, pre-roll **571 ms** (the study's own "window 5" row)	memory (DEC-22), justified by measured lag ≤ 144 ms	min slack 169–179 ms, 0 overruns (E-53, E-63) VERIFIED
19	Three cursors on one ring	Simplified	one writer + one reader; pre-roll is "w − N"; the front end reads the fresh hop directly	a front-end cursor was unnecessary	— VERIFIED
20	Warm TCP from boot, 8-byte header, 20 ms frames, 2 frames/packet, OVERRUN marker	Yes	`netstream.c`; fixed payload sizes per packet type (the header has no length field)	keepalive 10 s instead of 15 s (the clock pings double as keepalive)	24/24 and 30/30 clean streams (E-36, E-60) VERIFIED
21	ADPCM mode, throughput probe, degradation ladder	No	PCM16 only	descoped	— PLANNED
22	NTP-style clock-offset estimator over the same socket	Yes	PING/PONG/INFO; lowest RTT of the last 8	—	± 1.2–1.8 ms half-RTT in T1 VERIFIED
23	Self-hosted Vosk/Kaldi ASR, WebRTC-VAD endpointing	Yes / changed	Vosk `en-in-0.5`; **energy VAD** in the speech band	no extra package; ASR endpoint unusable on this audio (E-48)	E-49, E-61 DOCUMENTED
24	Latency L1/L2/L3 with GPIO + playback rig as ground truth	Partly	L1 and L2 from software timestamps; keyword end located in the streamed audio	no logic analyser or playback rig (C-06); a 10 ± 7 ms term is estimated	E-37, E-63 VERIFIED
25	CPU by GPIO duty cycle (primary) + cycle counters + run-time stats	Partly	GPIO16 toggled per hop; `@PERF` cycle counts; `@CPU` run-time stats	GPIO never captured (no analyser); run-time stats became primary	E-56, E-60 VERIFIED
26	"256 KiB Arena Discipline": report, bound, enforce	Report only	S4 reported from linker map + heap minimum	bounded heap and allocation-failure hook not built	— PLANNED
27	Disclose the Wi-Fi blob in an SBOM	Yes	`docs/SBOM.md`; 9 closed libraries listed from the link map	—	E-15 VERIFIED
28	≥ 6–10 h held-out ambient audio for false-activation claims	Partly	10 h overnight (5 h declared held-out) + 2.47 h active household	quiet and active figures reported separately	E-39, E-51 VERIFIED
29	≥ 100 activations per latency distribution	No	n = 22 (E-37), n = 12 (T1)	time	— VERIFIED

2.7 How the measurements compare with the study's estimates

The implementation is, in effect, the experiment the study asked for. Comparing its estimates with what was measured is the most useful lesson in this chapter.

Table 2.3. Study estimates versus measurements on the final build

Quantity	Study estimate [EST]	Measured	Main reason for the difference
Peak RAM, S4	≈ 227 KiB (232 400 B)	245.0 KiB (250 899 B), E-60	the budget left out code placed in internal RAM (IRAM, 54 591 B) — conflict C-10 found at M0
Model flash	40–60 kB	28 024 B	a smaller model than the study's reference class
Tensor arena	32 KiB budget	13 404 B used	same
Pipeline CPU (C1, one core)	≈ 2.8 %	≈ 9.25 % (i2s_reader task)	windowed model recomputes the whole 1 s window (≈ 3.9 %), and the fixed-point microfrontend with noise reduction costs ≈ 4.5 %
System CPU, both cores (C3)	≈ 2.1 %	5.53 %	as above, plus Wi-Fi with power save disabled

Quantity	Study estimate [EST]	Measured	Main reason for the difference
L1 detection → server	≈ 13 ms	2.4 ms (START) / 18 ms (first audio, with ASR load)	LAN and warm TCP as designed; the extra ≈ 16 ms is the server's receive scheduling while Vosk runs
L2 keyword end → server	≈ 205 ms	138 ms median	smoothing over 3 scores (≈ 120 ms span) instead of 5
Mic pass-band edge	6 768 Hz	visible in the recorded spectrum (E-03)	agrees

> **KEY IDEA**
>
> The study was right about *where* the problems are (network stack and RTOS memory, false activations, parity) and wrong by about 3× about *how much CPU* the audio path takes — mostly because two of its assumptions (a float esp-dsp FFT and a streaming model) were not implemented. The measured numbers still pass, but with less margin than the study predicted.

2.8 What the implementation does *not* establish about the research

- It does **not** validate the study's model comparison: only one model family was trained, so nothing here shows that DS-CNN is better or worse than TC-ResNet or dilated convolutions.
- It does **not** test Architectures B or C, ADPCM, UDP or speculative streaming.
- It does **not** reproduce the microWakeWord numbers the study quotes (tensor-arena sizes, esp-nn issues); those remain DOCUMENTED claims about other projects.
- It **does** give real measurements for the study's own budget categories on the same chip and microphone, which is exactly what the study said it lacked ("there are currently zero [MEAS] values in this document").

2.9 Which parts are the study's, and which are this project's

The **study** contributed the problem framing (false activations first), the datasheet analysis, the budgeting method (S0–S4, C1–C3, L1–L3), the compliance positions (no ESP-SR, disclose the Wi-Fi blob, train from scratch, non-transformer ASR), the pre-roll derivation, the warm-TCP transport and the parity rule.

The **implementation** contributed everything that required hardware: the measured I2S framing, the libm-table parity fix, the IRAM discovery and memory trims, the decision to run a windowed model, the device-identical replay evaluation, the hard-negative mining from the device's own mistakes, the server's ASR conditioning and threading, the measured latency decomposition, and all the evidence. Chapter 13 discusses which of these are genuinely distinctive.

CHAPTER 3

System Requirements and Design Constraints

3.1 Where the requirements come from

Only the problem statement is binding. The project turned it into ten numbered requirements, R1–R10, in `docs/PROJECT_STATE.md` §2; they are reproduced here with their wording checked against the PS text. Everything else — the choice of ESP32-S3, the 6 500 Hz filterbank edge, the 571 ms pre-roll — is a *design decision*, not a requirement, and is labelled as such. VERIFIED

Table 3.1. Requirements taken from the problem statement

ID	Requirement (PS wording condensed)	Kind
R1	Lightweight, highly accurate KWS model running **locally** on a low-power device	functional
R2	On detection, stream the **subsequent audio** to a **remote ASR server** with minimal data overhead and latency	functional
R3	Evaluated on efficiency (RAM/flash footprint, idle CPU), accuracy (high true-positive rate, near-zero false activations) and latency (keyword end → cloud ASR receiving audio)	evaluation metrics
R4	Proprietary, closed-source or commercial **voice-activation SDKs** are prohibited	software constraint
R5	KWS pipeline built with open-source ML / TinyML frameworks (e.g. TFLite Micro)	software constraint
R6	No models pre-trained on generic assistant keywords; train on a **custom keyword**	software / data constraint
R7	Edge software within **< 256 KB RAM** and **< 10 % CPU** while idling in continuous listening	numeric constraint
R8	Heavy or uncompressed pre-trained transformers are disqualified	software constraint
R9	Evaluated on a physical low-power microcontroller (e.g. Raspberry Pi, ESP32)	hardware constraint
R10	The model must work for the given custom keyword	functional

> **CAUTION**
>
> The PS gives **only two numeric limits**: 256 KB RAM and 10 % idle CPU. It gives **no numeric target** for latency, true-positive rate, false activations per hour, model size, flash size, data overhead or power. This report never invents one. Where a figure is compared with something, it is compared with the PS limit, the architecture study's estimate, or an earlier measurement — and says which.

3.2 Functional requirements

Table 3.2. Functional requirements — design response, test and status

Req	Why it exists	How the design answers it	How it was tested	Status
R1 local KWS	edge "wake-up" avoids cloud cost, privacy loss and delay	int8 DS-CNN (28 024 B) in TFLite Micro on the ESP32-S3, inference every 60 ms	live sessions with scripted prompts; 12/12 in T1	**Met** VERIFIED (E-22, E-35, E-63)
R2 stream subsequent audio	the heavy recognition is done remotely	warm TCP, 571 ms pre-roll + live audio until the server's END	integrity (bit-exact), overruns, latency	**Met** VERIFIED (E-44, E-60, E-63)
R2 minimal overhead	the PS asks for it without a number	PCM16 in 1 280-byte payloads with an 8-byte header; nothing sent while idle except pings	calculated, not measured on the wire	**Met by design** INFERRED (Section 3.6)
R2 remote ASR	the PS chain ends at "ASR receiving the audio"	self-hosted Vosk en-in-0.5 on a PC	demo run, command sessions	**Met** (receiving); transcript quality limited VERIFIED (E-49, E-61)

Req	Why it exists	How the design answers it	How it was tested	Status
R10 works for the custom keyword	the keyword is the whole point	"Hey Vayro", trained from synthetic + real speech	two speakers, distances 0.5–2 m	**Met with limits** — 2 speakers only VERIFIED

3.3 Measurable constraints

3.3.1 RAM: < 256 KB

Why it exists. To force a genuinely small edge footprint. *The ambiguity.* The PS does not say what counts (the whole device? the application? the model?) or whether KB means 1 000 or 1 024 bytes. *Design response.* The project uses the strictest reading, **S4**: everything in the chip's internal RAM that the application occupies at its peak — global variables, instruction code copied into RAM (IRAM), every task stack, the Wi-Fi and TCP/IP buffers, the tensor arena, the audio ring — with PSRAM disabled so nothing can hide in external memory, and compares it with both 256 000 B and 262 144 B.

How it is measured. S4 = static internal RAM from the linker (`idf.py size`: `.bss + .text in IRAM + .data + vectors`) **plus** the peak heap use, computed as the heap's total size minus its lowest free value reported during a run that includes streaming. The 96 KB of on-chip SRAM configured as instruction and data cache is excluded as hardware-reserved (a stated choice, see Chapter 12). *Result:* **250 899 B** — passes 256 000 B by 5 101 B and 262 144 B by 11 245 B. **Met** VERIFIED (E-60).

3.3.2 CPU: < 10 % while idling in continuous listening

Why it exists. An always-on device must leave most of the processor free. *The ambiguity.* "10 % CPU" on a dual-core chip can mean the busiest core or both cores averaged; "idle" here means "listening but no keyword", not "doing nothing". *Design response.* All always-on work is on core 1; networking is on core 0; both scopes are reported. *How measured.* FreeRTOS run-time statistics every 10 s (`@CPU`), cross-checked with cycle counters around the front end and inference (`@PERF`). *Result:* busiest core **9.32 % mean, 9.51 % max**; both cores **5.53 %** (E-60); a 12-minute run gave core-1 max 9.04 % (E-56). **Met, with thin margin on the per-core reading** VERIFIED. While *streaming* (not idle) core 1 reached 10.5 % — outside the requirement but reported (Chapter 12).

3.3.3 Latency

Definition used. L2 = acoustic end of the keyword → first audio packet received by the server's application. *Target.* None in the PS. *Result.* 138 ms median, 196 ms p95, n = 12 (E-63). **Measured; no pass/fail possible** VERIFIED one component estimated.

3.3.4 Accuracy

Target. "High true-positive rate" and "near-zero false activations", no numbers. *Result.* 22/25 and 20/25 keyword prompts; 0.2 false triggers/h in a quiet house, 15.8/h with television and conversation. The quiet figure is close to zero; the household figure is not. **Partially met** VERIFIED.

3.3.5 Model and flash size

Target. None ("model size (RAM/Flash footprint)" is a scored metric). *Result.* model 28 024 B; tensor arena 13 404 B used of 16 384 B reserved; full firmware image 901 059 B in a 1.5 MB app partition. **Reported** VERIFIED.

3.3.6 Power

Target. None; the PS says "low-power device" qualitatively. *Result.* **Never measured.** Wi-Fi modem sleep is deliberately disabled for lower latency, which raises consumption INFERRED. UNVERIFIED

3.4 Software constraints

Table 3.3. Software constraints and how compliance is shown

Req	Constraint	Design response	Evidence	Status
R4	no proprietary voice-activation SDK	everything in the audio → features → model → decision path is open source; `tools/forbidden_deps.py` fails the build if ESP-SR, WakeNet, Porcupine, etc. appear anywhere	"forbidden_deps: PASS" in every build log	**Met** VERIFIED
R4 (strict reading)	nothing closed at all	9 closed Espressif Wi-Fi/PHY libraries and the boot ROM remain; disclosed in `docs/SBOM.md`	link map	**Disclosed exception** VERIFIED
R5	open-source TinyML framework	TensorFlow Lite Micro via esp-tflite-micro 1.4.1, esp-nn 1.4.1 (Apache-2.0)	`dependencies.lock`	**Met** VERIFIED
R6	custom keyword, no pre-trained assistant model	"Hey Vayro"; model built layer by layer from random weights (seed 26172); no weight loading anywhere	`model/train/train.py`	**Met** VERIFIED
R8	no heavy transformers	edge: 10 665-parameter CNN; server: Kaldi-based Vosk (not a transformer)	SBOM	**Met** VERIFIED

3.5 Hardware constraints

R9 asks for a physical low-power microcontroller; every result in this report comes from one physical ESP32-S3 board with one INMP441 microphone. The ESP32-S3 was the project's choice (the PS says "e.g. Raspberry Pi, ESP32"). The architecture study stated explicitly that if the chip could not meet the limits, "the platform loses, not the requirement"; measurement did not trigger that condition. VERIFIED

3.6 Communication requirements

The PS asks for "minimal data overhead" without defining it. The design choices and their consequences:

- **Nothing is transmitted while idle** except one clock-synchronisation exchange every 10 s: PING (8-byte header + 8 B), PONG (8 + 24 B) and INFO (8 + 16 B) — 72 bytes of application data, plus TCP/IP headers. VERIFIED `netstream.c`
- **Audio is raw PCM16**: 16 000 samples/s × 2 B = 32 000 B/s of payload. Compression (ADPCM, 4:1) was designed in the study but not built. VERIFIED
- **Protocol overhead**: each audio packet carries 1 280 B of audio and an 8-byte header — **0.6 %** at the application layer. With a 40-byte TCP/IP header per packet (one segment per packet, since the configured TCP MSS is 1 440 B and `TCP_NODELAY` is set), the overhead on the network is about **3.7 %**. This figure is calculated, not measured with a packet capture. INFERRED
- **Per activation**: a stream of 2.08 s (a keyword with no command, as in T1) carries 66 560 B of audio; a keyword + command of ~3.5 s about 112 kB. INFERRED from measured stream lengths

3.7 Environmental and operational constraints

- **One microphone.** No beamforming or echo cancellation is possible; all noise robustness must come from the features, the training data and the decision rule. VERIFIED
- **One home, one room layout.** Every recording was made in the same household: a desk with a fan, evening insect/appliance noise (E-34), a television and family conversation (E-51), devotional singing from another room (E-47). This is both the strength (realistic) and the limit (one environment) of the evidence.
- **Home Wi-Fi, fixed server address.** The server's IPv4 address is compiled into the firmware (`CONFIG_PS_SERVER_HOST`). When the router restarted overnight and handed out a different subnet, the device could not reconnect until rebuilt (E-41). VERIFIED

- **PC must be awake.** The server runs on a desktop PC; when the PC slept, the server died and streams failed (E-58). DOCUMENTED

3.8 Integration requirements

The latency metric can only be measured if the team controls both ends: the device's detection time and the server's receive time. That is why the ASR server is self-hosted and instrumented (microsecond receive timestamps, ping-based clock offset), rather than a commercial cloud API — which would also have raised the open-source question. VERIFIED `server/ingest.py`

3.9 Requirement compliance summary

Table 3.4. Compliance at a glance (details in Chapter 12)

Req	Status	One-line reason
R1	Met	28 KB int8 model runs on the chip, outputs bit-exact with the PC reference
R2	Met	every detection streamed with pre-roll; 0 lost packets in the final tests
R3 efficiency	Met	245.0 KiB peak RAM; idle CPU 9.32 % busiest core / 5.53 % both
R3 accuracy	Partially	good recall; false triggers near zero only in a quiet house
R3 latency	Measured	138 ms median keyword end → server (no PS target)
R4	Met, with disclosure	no voice SDK; closed Wi-Fi driver disclosed
R5	Met	TFLite Micro, esp-nn
R6	Met	custom keyword, random initialisation
R7 RAM	Met	5.1 kB margin on the strictest reading
R7 CPU	Met	≈ 0.5 point margin on the busiest core; 4.5 points on the average
R8	Met	no transformer on either side
R9	Met	physical ESP32-S3
R10	Met with limits	2 real speakers tested

CHAPTER 4

Hardware Inventory and Technical Specifications

4.1 Inventory

The hardware is deliberately minimal: one microcontroller board, one microphone breakout, a USB cable to a PC, and the household's Wi-Fi router. There is no codec, no amplifier, no level shifter, no external RAM in use, no logic analyser and no loudspeaker test rig.

Table 4.1. Hardware actually used (and what was planned but not available)

Item	Role	Physically used and tested?	Evidence
ESP32-S3 development board ("N16R8": 16 MB flash, 8 MB PSRAM)	runs capture, features, model, decision, Wi-Fi streaming	yes — every run	esptool identification `evidence/m1/esptool_flash_id.txt` VERIFIED
INMP441 MEMS microphone breakout (TDK InvenSense part)	the only sensor: digital audio over I2S	yes — every run	M1 runs E-02…E-05, E-18 VERIFIED
USB cable to the PC (native USB port of the S3)	power, flashing, console, telemetry and data capture	yes	`@BOOT reset=USB`, COM7 VERIFIED
Home Wi-Fi access point (2.4 GHz)	LAN between the board and the server PC	yes	`@NET wifi=up` VERIFIED; model not recorded UNVERIFIED
Desktop PC (Windows 11, Intel i3-8100, 4 cores, 15.9 GB RAM, no GPU)	ASR server, build host, training host (via WSL2)	yes	PROJECT_STATE §6 DOCUMENTED
Logic analyser (for GPIO duty-cycle and detection-pulse timing)	planned primary CPU and latency instrument	**no — not available**	C-06 VERIFIED
Loudspeaker playback rig with a GPIO marker (IN-1)	planned ground truth for "keyword end"	**no — not built**	PLANNED
100 kΩ pull-down resistor on SD; 0.1 µF decoupling at the mic	recommended by the study (P-07, P-24)	**not recorded**	UNVERIFIED

4.2 The microcontroller: ESP32-S3

4.2.1 What the chip is

The ESP32-S3 is a system-on-chip from Espressif: two processor cores, on-chip RAM, Wi-Fi and Bluetooth radios, and many peripherals on one piece of silicon. For this project the relevant parts are the two cores (one for audio, one for networking), the internal RAM (the budget), the I2S peripheral with DMA (the microphone), the vector instructions used by the neural-network kernels, the USB port (debugging and evidence), and Wi-Fi (streaming).

4.2.2 Measured identity of this board

The first thing done with the board (26 Sep) was to read its identity with esptool and back up its entire flash — it held an unrelated Arduino sketch from 2024 (`D:\ps26172_backup\preexisting_flash_16MB.bin`). DOCUMENTED

Table 4.2. Chip identity reported by esptool v4.12.0

Property	Value
Chip	ESP32-S3 (QFN56), revision v0.2
Features	Wi-Fi, BLE, **embedded PSRAM 8 MB (AP_3v3)**
Crystal	40 MHz

Property	Value
USB mode	USB-Serial/JTAG (native port, appears as COM7)
Flash	16 MB, quad SPI, 3.3 V (flash ID manufacturer 0x68, device 0x4018)

8 MB of in-package PSRAM corresponds to the "R8" variant, which the study notes is octal PSRAM and occupies GPIO35–37 INFERRED from the study's variant table. It does not matter here because **PSRAM is disabled in the firmware** and none of those pins is used. The firmware is also built for **4 MB of flash** (`CONFIG_ESPTOOLPY_FLASHSIZE_4MB`) although the chip has 16 MB; the firmware's own `@BOOT` line therefore reports `flash_bytes=4194304`, and the esptool reading is the authoritative one. VERIFIED

The physical board's maker and model, and whether it sits on a breadboard, are **not recorded** anywhere in the repository. UNVERIFIED

4.2.3 Specifications that matter to this design

The following facts are quoted from the architecture study, which cites the ESP32-S3 datasheet v2.2 by page. The datasheet itself was not supplied, so they are DOCUMENTED; where the project measured something, the measurement is given alongside.

Table 4.3. ESP32-S3 facts used by the design

Property	Datasheet value (via the study)	How it is used / what was measured
CPU	2 × Xtensa LX7, up to 240 MHz, single-precision FPU	run at 240 MHz (`CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ=240`) VERIFIED
Vector extension (PIE)	128-bit SIMD instructions	used by esp-nn's int8 kernels (0.47 multiply-accumulates per cycle measured, E-23) DOCUMENTED
Internal SRAM	512 KB (plus 384 KB ROM, 16 KB RTC RAM)	96 KB configured as cache (32 KB instruction + 64 KB data); the application's heap is 289 708 B in the product build VERIFIED `@MEM`
Cache	16/32 KB instruction, 32/64 KB data	32 KB I + 64 KB D chosen, "biased toward data" because model weights stream from flash VERIFIED sdkconfig
I2S	2 controllers, master/slave, 8–32-bit slots, BCK 10 kHz–40 MHz, DMA	I2S0 as master receiver, 32-bit slots, 1.024 MHz VERIFIED
GDMA	shared DMA channels for peripherals	I2S receive uses 4 descriptors × 160 frames VERIFIED
System timer	52-bit, 16 MHz	microsecond timestamps via `esp_timer_get_time()` VERIFIED
Wi-Fi	802.11 b/g/n, 2.4 GHz	station mode, modem sleep disabled VERIFIED
USB	USB-Serial-JTAG + USB OTG	USB-Serial-JTAG used as console and capture link VERIFIED
Supply (module)	3.0–3.6 V	powered from USB via the board's regulator INFERRED
Modem-sleep current @ 240 MHz	32.9–107.9 mA depending on load (Table 5-9)	**not measured** on this board UNVERIFIED

4.3 The microphone: INMP441

4.3.1 What it is

The INMP441 is a MEMS (micro-electro-mechanical) microphone with a built-in analog-to-digital converter and an I2S digital output. Because the conversion to numbers happens inside the microphone package, there is no analog wiring to pick up noise and no codec chip — three digital wires carry the audio.

4.3.2 Specifications (from the datasheet via the study)

Table 4.4. INMP441 facts that constrain the design DOCUMENTED

Property	Value	Consequence here
Interface	I2S slave, 24-bit two's complement, MSB first	32-bit slots, 24-bit samples; firmware keeps the top 16 bits
Frame	exactly 64 bit clocks per stereo frame	BCK = 64 × fs = 1.024 MHz at 16 kHz
Bit-clock range	0.5–3.2 MHz	1.024 MHz is well inside
Sample-rate range	7.8–50 kHz	16 kHz used
Channel select	L/R pin low → left slot; data line tri-stated in the other slot	L/R tied to ground; the right slot is empty
Supply	1.62–3.63 V; 2.2 mA typical at 3.3 V	powered from 3.3 V
Sensitivity / SNR	−26 dBFS at 94 dB SPL; 61 dBA SNR	quiet speech at 1–2 m is only ~10–25 dB above the room floor
Pass band	±0.04 dB up to 0.423 × fs (6 768 Hz at 16 kHz); −6 dB at fs/2	mel filterbank stops at 6 500 Hz
Group delay	17.2 / fs (1.078 ms at 16 kHz)	a fixed term inside the latency
Start-up	zero output for 2^18 bit clocks after power-up (≈ 256 ms)	300 ms settle before measuring

4.3.3 What was measured on this unit

The M1 milestone (26–27 Sep) tested the microphone before anything was built on top of it. VERIFIED unless marked

- **It is in the left slot, and the right slot is truly empty**: the right slot read 100 % zeros, as expected with L/R tied low and the data line pulled low when tri-stated (E-18).
- **Framing**: with the standard "Philips" I2S format the 24-bit value arrived one bit late — its lowest bit was never 1 and sign bits leaked into the empty slot. With "MSB-justified" framing the lowest bit was 1 in 50.1–50.9 % of samples (as random audio should be) and the empty slot stayed zero. The firmware uses MSB-justified (E-18, DEC-06). The electrical root cause was not isolated. VERIFIED behaviour; cause UNVERIFIED
- **Mono mode halves DMA traffic**: stereo capture moved 127 963 B/s, mono 63 976 B/s (E-05).
- **No zero-output start-up period was seen** — the first non-zero sample came 0.56 ms after the clock started, on warm boots and one cold power-up. Instead the output carried a large low-frequency transient (DC up to 26 % of full scale) that decays in about 1.5 s; within the 125–6 500 Hz band the level reaches the room floor within 100 ms (E-04, E-19). DOCUMENTED small n
- **The pass-band edge is visible**: in the recorded room noise, the 6 768–8 000 Hz band sits about 16 dB below the 4 000–6 768 Hz band (E-03; Figure 4.1).
- **Levels**: the room floor in the feature band was −52.9 dBFS (≈ 52 int16 units RMS, with a desk fan running); spoken words at about 0.5 m peaked at −38 to −42 dBFS per 100 ms (E-06). Low-frequency fan rumble (61–70 Hz lines, about −30 dBFS) dominates the broadband level (C-12) — this later forced a 150 Hz high-pass filter on the server (Chapter 11). DOCUMENTED

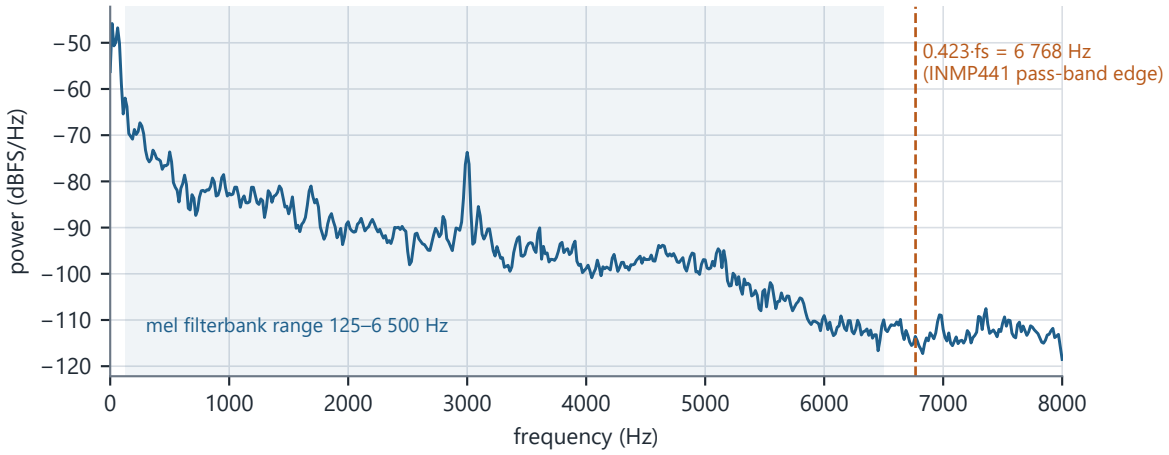
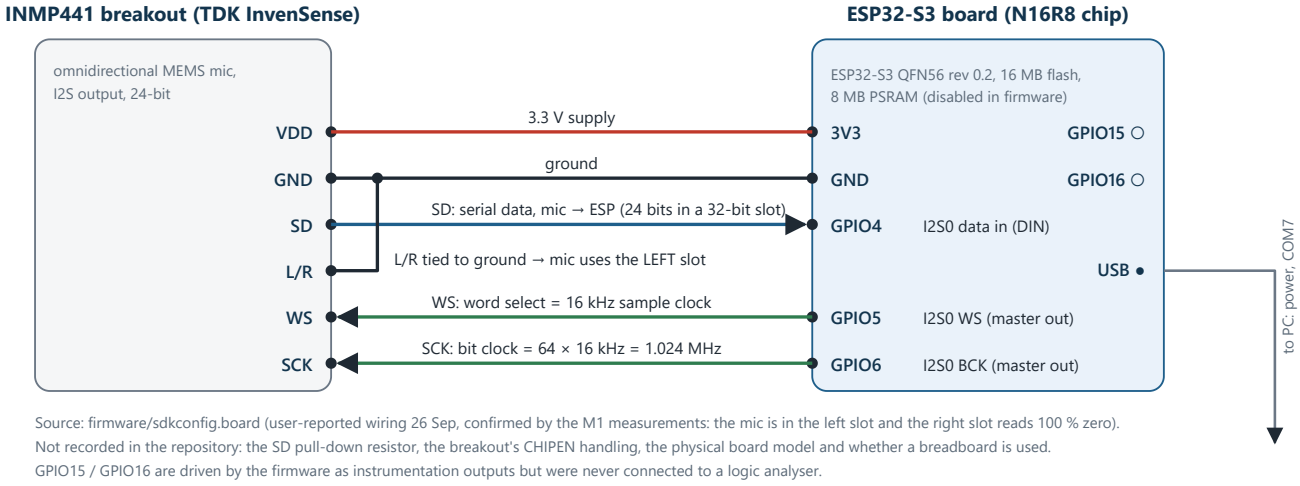


Figure 4.1. Spectrum of 18 s of room sound recorded through the INMP441 on this board (run `20260926_214931_m1_msb_settle_warm`, no speech). The drop above 6 768 Hz is the microphone's own filter; the shaded band is the range the mel filterbank uses. Generated from the saved WAV by `docs/report/make_figures.py`.

4.4 Wiring



Source: firmware/sdkconfig.board (user-reported wiring 26 Sep, confirmed by the M1 measurements: the mic is in the left slot and the right slot reads 100 % zero). Not recorded in the repository: the SD pull-down resistor, the breakout's CHIPEN handling, the physical board model and whether a breadboard is used. GPIO15 / GPIO16 are driven by the firmware as instrumentation outputs but were never connected to a logic analyser.

Figure 4.2. Wiring between the INMP441 and the ESP32-S3 as recorded in `firmware/sdkconfig.board`.

Table 4.5. Connection table

Component / signal	Controller pin	Purpose	Notes
INMP441 SCK (bit clock)	GPIO6	I2S0 BCK, driven by the ESP32-S3 at 1.024 MHz	the architecture's pin map had BCK on GPIO4
INMP441 WS (word select)	GPIO5	I2S0 WS, 16 kHz frame clock	as planned
INMP441 SD (serial data)	GPIO4	I2S0 data in; 24-bit samples, MSB-justified	the plan had SD on GPIO6; pull-down not recorded
INMP441 L/R	GND	selects the left slot	confirmed by measurement
INMP441 VDD	3V3	supply	decoupling not recorded
INMP441 GND	GND	ground	—
(none)	GPIO15	`INST_DETECT`: 10 ms pulse on each detection	driven, never measured
(none)	GPIO16	`INST_ACTIVE`: high while a 10 ms hop is processed	driven, never measured

Component / signal	Controller pin	Purpose	Notes
PC	native USB (USB-Serial-JTAG)	power, flashing, console, capture	COM7 on the PC

Planned versus actual. The study's blueprint (§B5) put BCK on GPIO4 and SD on GPIO6. The board had already been wired the other way round (SCK→GPIO6, SD→GPIO4) before the project started, and the firmware was made to follow the real wiring through `firmware/sdkconfig.board` instead of rewiring (conflict C-05). This wiring was **reported by the user**; it is consistent with every measurement (live audio in the left slot, empty right slot), which is as strong a confirmation as software can give. VERIFIED by measurement

4.5 The interfaces, explained

4.5.1 I2S (the microphone link)

I2S is a three-wire serial audio bus. The **bit clock (BCK/SCK)** ticks once per data bit. The **word select (WS)** line changes level at the start of each channel's slot and so runs at the sample rate. The **data line (SD)** carries the sample bits, most significant bit first. The ESP32-S3 is the *master*: it generates both clocks, and the microphone clocks its data out on them.

The only subtlety that mattered is *when* the first bit appears. In "Philips" framing the MSB arrives one bit clock after WS changes; in "MSB-justified" framing it arrives on the WS edge. Pick the wrong one and every sample is shifted by one bit: the sound is still recognisable (only a factor of two and a lost bit), which is exactly why this kind of fault is dangerous — it looks almost right. The M1 alignment sweep tested four combinations and picked the one where the data behaved like real audio (Chapter 11).

4.5.2 USB-Serial-JTAG (the evidence link)

The S3's native USB port appears on the PC as a serial port. The project uses it as a framed, CRC-checked binary link (`serial_link.c`): every message — text telemetry, raw audio, features, model outputs — is wrapped in a frame with a 4-byte sync word, type, length and a CRC-16. This made it possible to record hours of audio *through the real microphone* and to compare the device's internal numbers with the PC's, which is where most of the project's evidence comes from.

4.5.3 Wi-Fi and TCP (the product link)

The board joins the home network as a Wi-Fi station and keeps one TCP connection to the server. Chapter 6 describes the protocol.

4.5.4 GPIO (instrumentation)

Two pins are toggled by the firmware purely for measurement, following the MLPerf Tiny method the study recommended: GPIO16 is high while a 10 ms hop is being processed (its duty cycle *is* the pipeline's CPU share) and GPIO15 pulses on each detection. Without a logic analyser they were never read. VERIFIED code; measurement NOT RUN

4.6 Clocking and data-rate arithmetic

- Sample rate 16 000 Hz × 64 bit clocks per frame = **1.024 MHz** bit clock.
- Per 10 ms hop: 160 frames. In mono mode each frame is one 32-bit word, so the DMA delivers 640 bytes per hop (64 000 B/s); in stereo it would be 1 280 bytes (128 000 B/s).
- The firmware keeps only 16 bits per sample: 320 bytes per hop, **32 000 B/s** — the same rate that goes over Wi-Fi during a stream.
- DMA buffering: 4 descriptors × 160 frames = 40 ms of audio, so the processing task can be late by up to ~30 ms without losing samples. The driver counts overflows (`i2s_ovf`), and every product run reported **0**

(e.g. E-60). VERIFIED

4.7 Power

No current or energy measurement was made. Two design choices increase power on purpose: the CPU runs at 240 MHz (the study showed that halving the clock would roughly double the CPU percentage, and CPU is a hard gate while power has no target), and Wi-Fi modem sleep is disabled (`WIFI_PS_NONE`) so that packets from the server are not delayed to beacon intervals. The microphone's own 2.2 mA is negligible. Any statement about the device's power consumption would currently be UNVERIFIED.

4.8 What remains unverified about the hardware

- Board maker/model; breadboard or soldered; antenna type.
- The SD pull-down resistor and mic decoupling capacitor (the empty slot reading zero suggests the line is held low, but does not prove a resistor is fitted).
- How the breakout handles the microphone's CHIPEN (enable) pin.
- Supply quality and current draw.

CHAPTER 5

Software Stack and Development Environment

5.1 Four environments

The project runs in four software environments on one PC. Knowing which is which explains most of the file layout.

Table 5.1. The four environments

Environment	Where	Used for	Python
Firmware toolchain	Windows, `D:\esp\esp-idf-v5.5.5`, tools in `D:\esp\.espressif`	building and flashing the ESP32-S3 firmware	CPython 3.12 (IDF's own virtual environment)
Training environment	WSL2 Ubuntu, virtual env `~/.venvs/ps26172`	dataset building, training, int8 export, offline evaluation, the host-twin front end	CPython 3.12.14 via uv 0.12.19
Host tools	Windows, `D:\ps26172\.venv-host`	capture tool, analysis, report figures, forbidden-dependency gate, code generators	CPython 3.12
ASR server	Windows, the same `.venv-host` + vosk	the ingest server and its analysis scripts	CPython 3.12

Why two operating systems? ESP-IDF builds natively on Windows; TensorFlow's CPU build and the host-twin C library are simplest on Linux. The repository lives at `D:\ps26172` because ESP-IDF cannot build from a path containing a space (`D:\sih final` has one, DEC-02), and the IDF wrapper forces Python 3.12 because the PC's default Python 3.14 broke IDF's environment script. VERIFIED `tools/idf.ps1`, memory note

5.2 Firmware toolchain and libraries

Table 5.2. Firmware software with verified versions

Component	Version	Licence	Purpose	Source of the version
ESP-IDF	**v5.5.5** (git tag)	Apache-2.0	RTOS, drivers, Wi-Fi, TCP/IP, build system	`@BOOT idf=v5.5.5`, `dependencies.lock` VERIFIED
Xtensa GCC	**14.2.0**	GPL (tool only)	compiler	`CMakeCCompiler.cmake` in the build folder VERIFIED
CMake	3.30.2	BSD	build	path in the build log VERIFIED
esptool	4.12.0	GPL-2.0+ (tool only)	flashing, chip identification	logs VERIFIED
FreeRTOS	as shipped in IDF 5.5.5 (SMP)	MIT	tasks, queues, run-time statistics	VERIFIED
lwIP	as shipped in IDF 5.5.5	BSD-3	TCP/IP	VERIFIED
esp_wifi / esp_phy / esp_coex	as shipped in IDF 5.5.5	**Espressif binary licence (closed)**	Wi-Fi radio	link map, SBOM VERIFIED
esp-tflite-micro	**1.4.1** (component hash `bdb2625e…`)	Apache-2.0	TensorFlow Lite Micro interpreter	`firmware/dependencies.lock` VERIFIED
esp-nn	**1.4.1** (component hash `3b3411b0…`)	Apache-2.0	optimised int8 convolution kernels for the S3	`dependencies.lock` VERIFIED

Component	Version	Licence	Purpose	Source of the version
TFLM microfrontend	tflite-micro commit `aab6e2fd` (vendored, unmodified)	Apache-2.0	log-mel feature extraction	`firmware/components/microfrontend/PROVENANCE.md` VERIFIED
KissFFT	from esp-tflite-micro v1.4.1, commit `5d505b5f`	BSD-3-Clause	the FFT inside the microfrontend (16-bit fixed point)	`PROVENANCE.md` VERIFIED

How the versions are pinned. `firmware/main/idf_component.yml` requests `espressif/esp-tflite-micro: "==1.4.1"`; the ESP component manager resolves that to esp-nn 1.4.1 and records both with content hashes in `firmware/dependencies.lock`. The study warned why this matters: an unpinned esp-nn update once enlarged the tensor arena needed on the S3 and broke a working wake-word product (X7). The microfrontend is *copied into the repository* because esp-tflite-micro 1.3.3.1 to 1.4.1 do not include it (C-03). VERIFIED

Build configuration layers. `tools/idf.ps1` assembles the configuration from several files, in order: `sdkconfig.defaults` (policy: 240 MHz, PSRAM off, 1 kHz tick, run-time stats, performance optimisation, cache sizes, heap hooks) → `sdkconfig.board` (pins, I2S framing) → `sdkconfig.console_<uart|usbjtag>` → `sdkconfig.model` (generated operating point) → `sdkconfig.wifi` (local credentials, git-ignored) → an optional extra file such as `sdkconfig.product`. It also regenerates the `sdkconfig` whenever any of these is newer, because ESP-IDF otherwise silently ignores a changed defaults file. VERIFIED

5.3 Training environment

Table 5.3. Training software (from `model/env/requirements.lock`, 47 pinned packages)

Package	Version	Purpose
tensorflow (CPU)	2.21.0	training and int8 conversion (also recorded in every `metrics.json`)
keras	3.15.1	model definition
numpy / scipy	2.5.3 / 1.18.1	signal processing, mixing, filters
soundfile	0.14.0	reading FLAC/WAV
piper-tts	1.8.0 (GPL-3.0, run as a separate tool)	generating synthetic keyword audio
onnxruntime	1.30.0	Piper's neural voice runtime
matplotlib	3.11.2	plots

Data assets used by training: four Piper voices (`en_US-libritts_r-medium`, `en_GB-vctk-medium`, `en_US-l2arctic-medium`, `hi_IN-pratham-medium`, 63–79 MB each, downloaded 27 Sep) and LibriSpeech `dev-clean`. The host-twin library `libpsfe.so` is compiled in WSL with the system `gcc -O2` from the same C files as the firmware (`model/frontend/build_twin.sh`); the Ubuntu release and host gcc version were not recorded. VERIFIED except the last point, UNVERIFIED

5.4 Host tools and server

Table 5.4. Host and server software

Package	Version	Used by
Python	3.12	all host tools and the server
numpy / scipy / matplotlib	2.5.3 / 1.18.1 / 3.11.2	capture analysis, server audio conditioning and VAD, report figures
pyserial	3.5	`host/capture.py` (USB link)
vosk	0.3.45 (Apache-2.0)	speech recognition in `server/ingest.py`

Package	Version	Used by
vosk-model-en-in-0.5	971 186 187 B zip, SHA-256 `0365c52f…`	final ASR model (Indian English; publisher's WER 36.12 %)
vosk-model-small-en-in-0.4	37 573 330 B zip	first ASR model (publisher's WER 49.05 %), replaced 28 Sep

Sources: `docs/SBOM.md`, `docs/PROJECT_STATE.md` §6. DOCUMENTED

5.5 How the pieces talk to each other

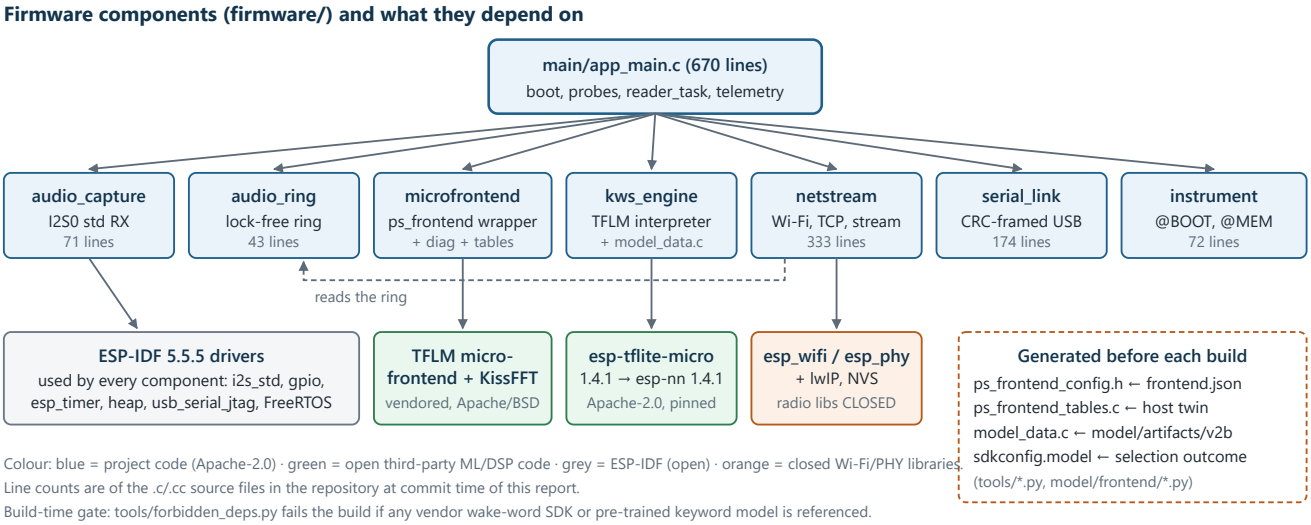


Figure 5.1. Firmware components and their dependencies. Four source files are generated from the model and front-end definitions before every build.

The software stack is linked by four **generated files** and two **wire formats**, and understanding these is the quickest way into the code:

- `model/frontend.json` → `tools/gen_frontend_config.py` → `ps_frontend_config.h` (firmware) — and the same JSON is read at run time by `model/frontend/psfe.py` (training).
- The host twin's maths tables → `model/frontend/export_tables.py` → `ps_frontend_tables.c` (firmware).
- A trained model folder `model/artifacts/<run>/` → `tools/gen_model_source.py` → `model_data.c` (model bytes, SHA-256s, input lookup table) and `sdkconfig.model` (threshold, averaging, stride, refractory).
- The **USB frame format** (`serial_link.h`) connects the firmware to `host/capture.py`.
- The **TCP stream format** (`netstream.h`) connects the firmware to `server/ingest.py`.

5.6 Compliance tooling

- `tools/forbidden_deps.py` scans every source, manifest and lock file for ESP-SR, WakeNet, MultiNet, `srmodels.bin`, Picovoice/Porcupine, Sensory, Cyberon, Alango and known pre-trained wake-word model names, and fails the build on any hit. It runs before every firmware build. VERIFIED
- `docs/SBOM.md` lists every component, version and licence, discloses the closed Wi-Fi/PHY libraries, the GPL tools that are *run* but not linked (esptool, piper-tts), and two training voices whose datasets are licensed non-commercially (L2-ARCTIC, Pratham). VERIFIED
- The project's own code is Apache-2.0 (`LICENSE`, team decision 29 Sep). VERIFIED

5.7 Versions that could not be confirmed

WSL Ubuntu release; the host gcc used for `libpsfe.so`; the exact FreeRTOS and lwIP versions bundled in IDF 5.5.5 (not printed anywhere in the logs); the Wi-Fi router's firmware. None affects a measured result, but they are listed so that no version in this report is guessed. UNVERIFIED

CHAPTER 6

Complete System Architecture

6.1 The big picture

The system has three subsystems connected by two links:

1. **The edge device** (ESP32-S3 firmware): captures audio, computes features, runs the keyword model, decides, keeps a short audio history, and streams on demand.
2. **The network**: one Wi-Fi association and one TCP connection on the home LAN.
3. **The ASR server** (Python on a PC): receives streams, timestamps them, ends them when the speaker stops, recognises the command, and records everything.

A fourth element, the **host capture tool** on the same PC, reads the device's USB telemetry. It is not needed for the system to work, but almost all the evidence in this report was collected through it.

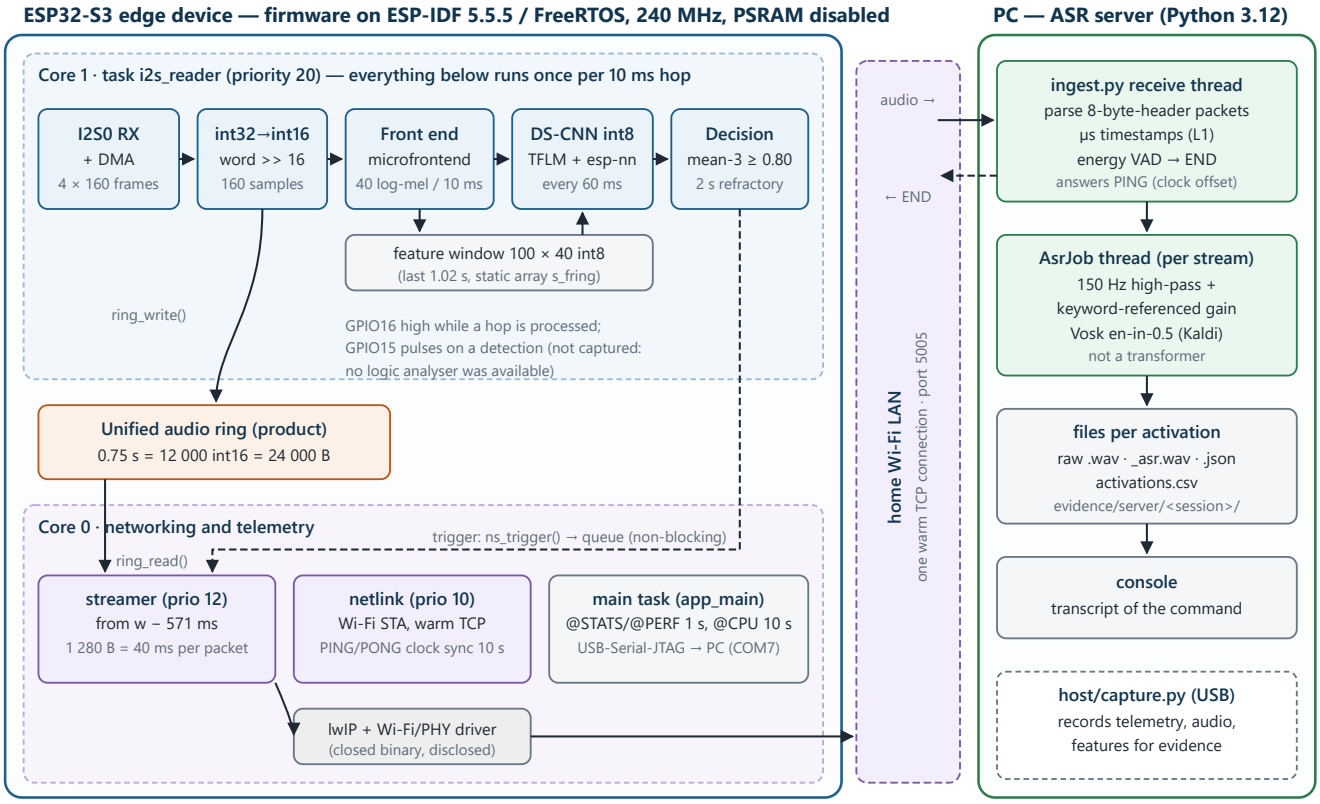


Figure 6.1. Final architecture. Core 1 runs the always-on audio path inside one task; core 0 runs networking and telemetry. The ring buffer is the only data shared between the two cores.

6.2 Subsystems and their responsibilities

Table 6.1. Subsystems of the final system

Subsystem	Implemented in	Responsibility	Runs where
Audio capture	`firmware/components/audio_capture`	configure I2S0 (MSB framing, 32-bit mono slot, DMA), read 160-sample hops, count overflows	core 1 (called by i2s_reader)
Audio ring	`firmware/components/audio_ring`	0.75 s int16 history; one writer, one reader; detects being overwritten	memory shared by both cores
Front end	`firmware/components/microfrontend`	30 ms window, FFT, 40 mel bands, noise reduction, gain normalisation (PCAN), log → 40 numbers per 10 ms	core 1

Subsystem	Implemented in	Responsibility	Runs where
KWS engine	`firmware/components/kws_engine`	TFLite Micro interpreter with the int8 model; one inference per call	core 1
Decision logic	`firmware/main/app_main.c` (`kws_on_frame`)	schedule inference every 6 frames, average 3 scores, threshold, refractory, trigger	core 1
Streamer	`firmware/components/netstream` (`streamer_task`)	on a trigger, send START, pre-roll and live audio, END; detect overrun; enforce timeout	core 0
Network link	`netstream` (`net_task`, Wi-Fi events)	Wi-Fi station, TCP connection at boot, reconnect, clock pings, receive END	core 0
Telemetry	`firmware/components/serial_link`, `instrument`, main loop	framed USB messages: `@BOOT @CFG @MEM @KWS @KW @STREAM @STATS @PERF @CPU @NET`	core 0
Ingest + endpointing	`server/ingest.py` (`Session`, `VadEndpointer`)	parse packets, timestamp, answer pings, decide end of speech, write files	PC, receive thread
ASR	`server/ingest.py` (`AsrJob`, `AsrFrontEnd`)	condition the audio, run Vosk, produce the transcript	PC, one thread per stream

6.3 Tasks and cores

Table 6.2. FreeRTOS tasks in the product firmware

Task	Core	Priority	Stack	Created by	Work
`i2s_reader`	1 (pinned)	20	3 584 B	`app_main`	the whole audio pipeline, once per 10 ms hop
`streamer`	0 (pinned)	12	4 096 B	`ns_start`	one stream at a time, waits on a 1-slot queue
`netlink`	0 (pinned)	10	4 096 B	`ns_start`	TCP connect/reconnect, pings, receive loop
`main` (`app_main`)	0	IDF default	3 584 B	ESP-IDF	boot, then telemetry every 1 s / 10 s and the `T` test hook
`wifi`	0 (pinned by IDF config)	IDF	IDF	ESP-IDF	Wi-Fi driver
`tcpip_thread`	either core	18	IDF	ESP-IDF	lwIP
`esp_timer`, `ipc0/1`, `IDLE0/1`	system	—	—	ESP-IDF	system services and idle accounting
`link_tx`	0	5	4 096 B	`app_main`	**capture builds only**: sends binary audio/feature frames over USB

Sources: `app_main.c`, `netstream.c`, product `sdkconfig`. VERIFIED

Why this split. Everything that must happen every 10 ms without fail is on core 1 at high priority, in a single task, so there is no hand-off between tasks, no extra stacks, and Wi-Fi interrupts on core 0 cannot delay it. The architecture study proposed exactly this split (§4.8), but with three tasks on core 1 (reader, front end + inference, control). The implementation merged them into one: they run strictly one after another on each hop, and splitting would only add context switches and stack memory. VERIFIED code; rationale DOCUMENTED in the study §B2

6.4 Data flow

Follow one piece of sound through the system:

1. **Microphone → DMA.** The INMP441 shifts out a 24-bit sample every 62.5 µs. I2S0's DMA collects 160 of them (10 ms) into one of four buffers.
2. **DMA → int16.** `reader_task` wakes when a buffer is full, takes the 160 32-bit words and keeps bits 31:16 (the top 16 bits of the 24-bit sample).

3. **int16 → ring.** The 160 samples are appended to the 0.75 s ring. Nothing reads them there unless a detection happens.
4. **int16 → features.** The same 160 samples go to the front end, which slides its 30 ms window by 10 ms and outputs 40 numbers. They are converted to int8 and stored as row *f mod 100* of a 100-row feature array.
5. **features → score.** On every sixth frame the 100 rows are copied, oldest first, into the model's input tensor (through a 256-entry lookup table that maps the front end's int8 scale onto the model's), and the model is invoked. Its int8 output becomes a probability *p*.
6. **score → decision.** The mean of the last three *p* values is compared with 0.80, and a trigger is allowed only if at least 2 000 ms have passed since the last one.
7. **decision → stream.** A trigger puts a small record (frame number, ring write position, detection time, raw score) into the streamer's one-slot queue — without waiting. If the streamer is already busy or the network is down, the trigger is logged with `streamed=0`.
8. **ring → network.** The streamer reads the ring from 571 ms before the recorded write position and sends it in 1 280-byte packets; then it keeps up with new audio.
9. **network → server.** The server timestamps every packet, feeds the audio to its voice-activity detector and to the ASR thread, and tells the device to stop when the speaker has finished.

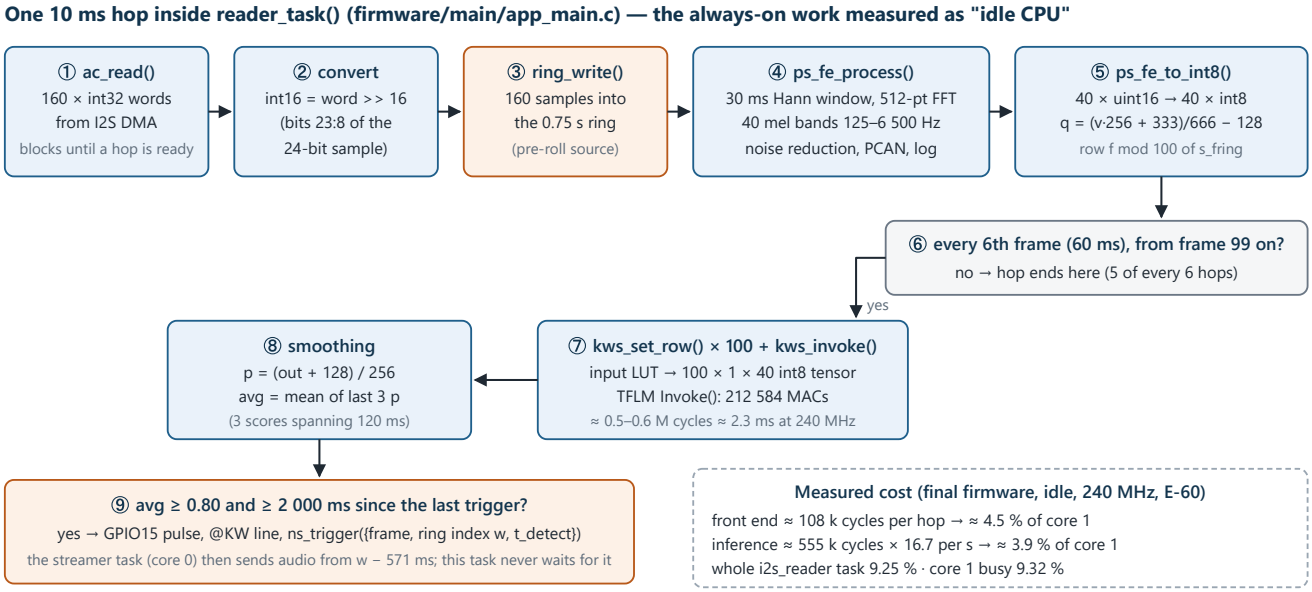


Figure 6.2. What happens inside one 10 ms hop on core 1 (steps ①–⑨ correspond to steps 2–7 above).

6.5 Control flow, initialisation and runtime behaviour

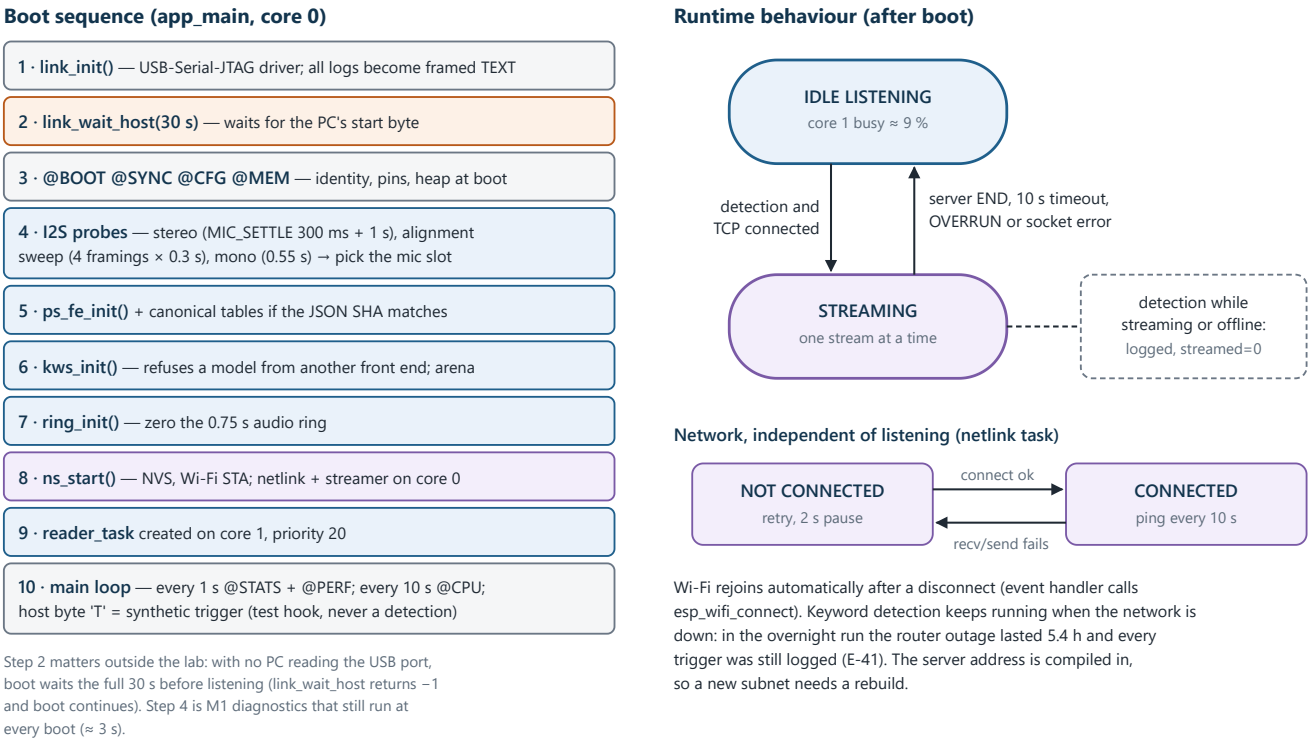


Figure 6.3. Left: the boot sequence in `app_main()`. Right: runtime states. Listening never stops; streaming and networking run beside it.

Two design points in the boot sequence deserve attention:

- **The firmware waits for the PC before starting audio** (`link_wait_host(30000)`), because on the native USB port anything printed before the PC opens the port is lost, and the M1 boot diagnostics were the evidence. If no PC opens the port, the wait times out after 30 s and boot continues. For a stand-alone device this is a 30-second delay after every power-up. VERIFIED code
- **The M1 microphone probes still run at every boot** — a 1.3 s stereo probe, four 0.3 s alignment captures, and one or two 0.55 s mono probes — and they decide which slot mask to use. They cost about 3 s of boot time and make each boot print a fresh microphone health check. VERIFIED code

6.6 Communication between device and server

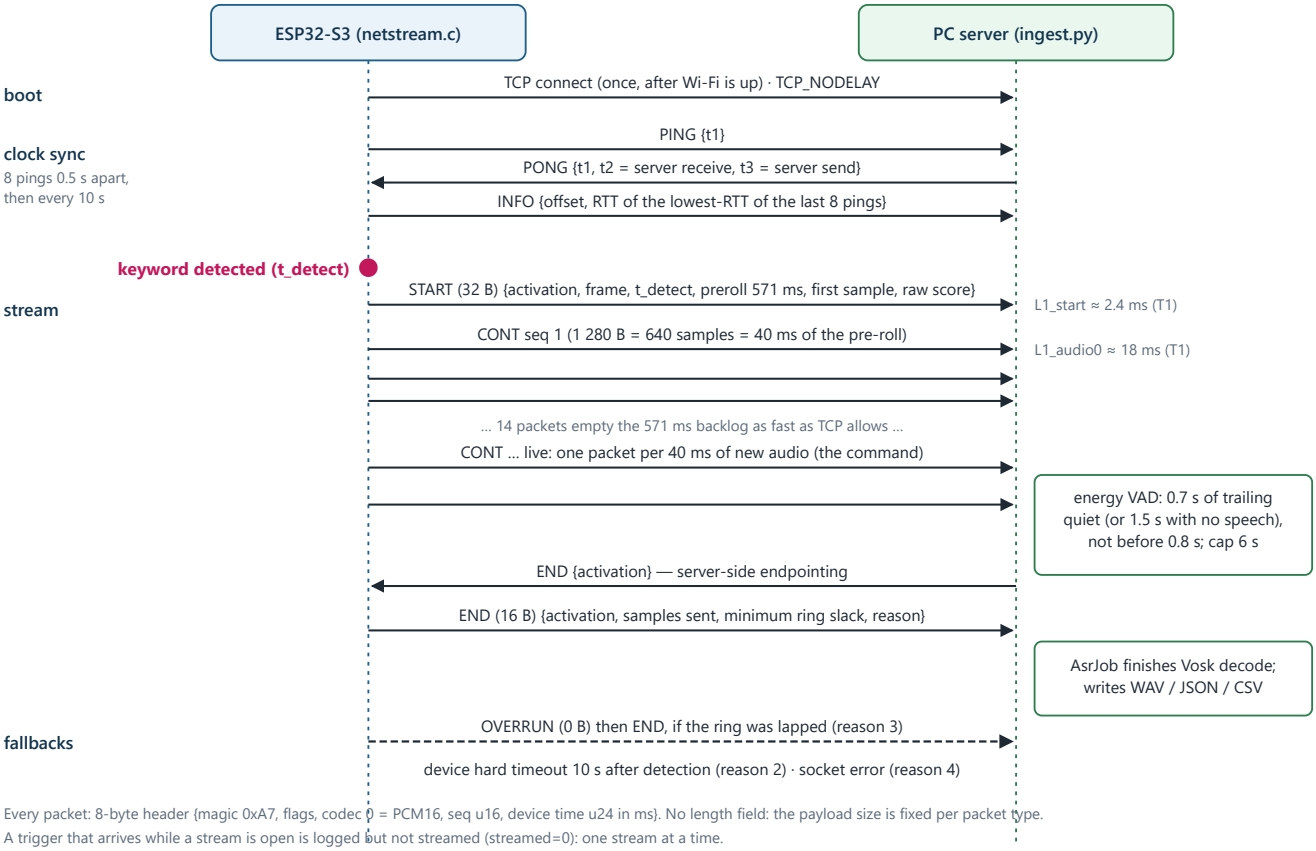


Figure 6.4. The TCP message sequence between the device (`netstream.c`) and the server (`ingest.py`), with the latency points measured in T1.

The protocol is binary and minimal. Every message starts with the same 8-byte header — `magic 0xA7` , `flags` (message type), `codec` (always 0 = PCM16), a 16-bit sequence number, and a 24-bit device time in milliseconds. The architecture study's header had no length field, so each message type has a **fixed payload size**, and both sides use a lookup table (`PAYLOAD` in `ingest.py`). The receiver resynchronises on the magic byte if it ever loses track. VERIFIED

Table 6.3. Message types

Flag	Name	Direction	Payload	Meaning
0x01	START	device → server	32 B	activation number, keyword frame, detection time (µs), pre-roll (571 ms), first ring sample, raw model output
0x02	CONT	device → server	1 280 B	640 PCM16 samples = 40 ms of audio
0x04	END	device → server	16 B	activation, samples sent, minimum ring slack, reason (1 server end, 2 device timeout, 3 overrun, 4 socket error)
0x04	END	server → device	4 B	activation to stop (server-side endpointing)
0x08	OVERRUN	device → server	0 B	the ring was overwritten before it could be sent; followed by END
0x20	PING	device → server	8 B	device time t1
0x80	PONG	server → device	24 B	t1, server receive time t2, server send time t3
0x40	INFO	device → server	16 B	current clock offset, round-trip time, number of pings

Clock alignment. Latency across two computers needs a common clock. After each PONG the device computes the offset `((t2 - t1) + (t3 - t4)) / 2` and the round-trip time, and keeps the offset from the lowest-RTT exchange among the last eight (the least disturbed one). It sends that offset with INFO; the server converts the device's detection time into its own clock and reports the uncertainty as half the RTT (1.2–1.8 ms in T1). VERIFIED `handle_pong()`

6.7 External dependencies at runtime

- A Wi-Fi access point with the SSID and password compiled into the firmware (`sdkconfig.wifi`, never committed).
- A server listening on the compiled-in IPv4 address, port 5005.
- For the ASR: the 1.0 GB Vosk model directory on the PC and about 30 s to load it.
- For telemetry and evidence: a PC with the capture tool on the USB port (optional for operation, apart from the 30 s boot wait).

6.8 Failure handling that is actually implemented

Table 6.4. What happens when something goes wrong

Failure	Mechanism	Behaviour	Evidence
Streamer falls 0.75 s behind	`backlog >= RING_SAMPLES` or `ring_read` detects it was lapped	sends OVERRUN, stops, END reason 3 — a short correct stream instead of corrupted audio	3/31 overruns in E-58, all flagged VERIFIED
Server never ends a stream	device hard timeout	END reason 2 after 10 s	code VERIFIED
Server always ends a stream	server cap `--stream-s-max 6`	END after 6 s of post-trigger audio	code VERIFIED
Detection while a stream is open	`ns_trigger()` refuses	logged `streamed=0`; not queued	E-48, E-49 DOCUMENTED
TCP connection lost	`net_task` loop	closes, waits 1 s, reconnects (2 s pause after each failed connect)	E-41 DOCUMENTED
Wi-Fi lost	event handler	`esp_wifi_connect()` again	E-41 DOCUMENTED
Network down for hours	detection is independent of the network	keeps detecting; triggers logged	5.4 h outage, E-41 DOCUMENTED
DMA not read in time	driver callback	counted as `i2s_ovf` in `@STATS` (0 in final runs)	E-60 VERIFIED
Model built for a different front end	`kws_init()` SHA check	refuses to start (`ESP_ERR_INVALID_VERSION`)	code VERIFIED
USB host not reading	1 s write timeout	frame dropped and counted (`link_drops`); audio path unaffected	E-20 DOCUMENTED
Server address unreachable (new subnet)	none	retries forever; needs a rebuild	E-41 — **not handled** VERIFIED
Heap exhausted	none beyond IDF defaults	not handled; peak measured instead	— VERIFIED

6.9 Operational sequence (final system)

1. **Power and boot.** USB power; the firmware waits for the capture tool (≤ 30 s), prints `@BOOT` (IDF version, firmware hash, chip, PSRAM off), runs the microphone probes (≈ 3 s), initialises the front end with canonical tables and the model (`@KWS … run=v2b … arena_used=13404`), zeroes the ring, starts Wi-Fi.
2. **Connect.** Wi-Fi gets an address (`@NET wifi=up`), the TCP connection opens (`@NET tcp=connected`), eight quick pings establish the clock offset.

3. **Listen.** Core 1 processes 100 hops per second; the model runs 16.7 times per second; the main task prints `@STATS` / `@PERF` every second and `@CPU` every ten.
4. **Detect.** The averaged score crosses 0.80: GPIO15 pulses, `@KW` is printed, the streamer is woken.
5. **Stream.** START, then the 571 ms backlog in about 14 packets, then live 40 ms packets.
6. **End.** The server's VAD decides the command is over and sends END; the device replies with END and its statistics (`@STREAM … reason=1 … min_slack_ms=179`).
7. **Recognise.** The server's ASR thread finishes decoding, prints the command and writes `<stem>.wav` , `<stem>_asr.wav` , `<stem>.json` and a row in `activations.csv` .
8. **Continue.** Listening never paused; the next keyword can be detected two seconds after the previous one.

CHAPTER 7

Implementation and Source-Code Walkthrough

This chapter is a map for reading the repository. It starts with the directory structure, then follows the firmware in the order it executes, and then covers the model pipeline, the server, the host tools and the test scripts. Line counts and names are taken from the files at commit `9a6e700`. VERIFIED throughout unless marked

7.1 Repository structure

```
D:\ps26172\
├── firmware\                    ESP-IDF project (the edge software)
│   ├── CMakeLists.txt           project "ps26172_fw"
│   ├── sdkconfig.defaults       policy: 240 MHz, PSRAM off, 1 kHz tick, run-time stats, caches, -O2
│   ├── sdkconfig.board          wiring (BCK 6, WS 5, SD 4) and I2S framing (MSB)
│   ├── sdkconfig.console_*      console on UART or native USB
│   ├── sdkconfig.model          GENERATED operating point (threshold 80 %, N 3, stride 6, 2 000 ms)
│   ├── sdkconfig.product        EXP-1 product configuration (DEC-22, DEC-24)
│   ├── sdkconfig.wifi           local credentials — git-ignored, never committed
│   ├── dependencies.lock        esp-tflite-micro 1.4.1 / esp-nn 1.4.1 with hashes
│   ├── main\  app_main.c (670), Kconfig.projbuild (122), idf_component.yml
│   └── components\
│       ├── audio_capture\       I2S0 receive (71)
│       ├── audio_ring\          unified ring buffer (43)
│       ├── microfrontend\       wrapper (99 + 94) + GENERATED config/tables + vendored TFLM/KissFFT
│       ├── kws_engine\          TFLM interpreter (79) + GENERATED model_data.c (the model bytes)
│       ├── netstream\           Wi-Fi, TCP, streaming, clock sync (333)
│       ├── serial_link\         CRC-framed USB link (174)
│       └── instrument\          @BOOT / @MEM lines (72)
├── model\
│   ├── frontend.json            SINGLE SOURCE OF TRUTH for the features
│   ├── frontend\                host twin: psfe.py, build_twin.sh, parity_test.py, export_tables.py …
│   ├── data\                    sources.json (data manifest + declared splits), gen_tts.py, build_dataset.py
│   ├── train\                   train.py, export.py, run_v4.sh
│   ├── eval\                    eval_stream.py (device-identical replay) + 10 analysis scripts
│   ├── env\                     requirements.lock, check_converter.py
│   └── artifacts\               v0 … v4 models, metrics, logs, evaluation JSONs, selection rules
├── server\                      ingest.py (453) + latency_report, stream_integrity, replay_session, ASR
tests
├── host\                        capture.py (501), analyze_audio.py, prompter.py, t1_session.py,
│                                scripts\*.txt (recording protocols), demo recording tools
├── tools\                       idf.ps1, forbidden_deps.py, gen_frontend_config.py, gen_model_source.py,
setup scripts
├── docs\                        PROJECT_STATE.md, EVIDENCE_LEDGER.md, SBOM.md, experiments\, submission\,
report\
└── evidence\                    raw data: runs\<time>_<label>\, server\<session>\, build/size logs
```

The **source of truth** for anything numerical is in three places: `model/frontend.json` (features), `model/artifacts/<run>/metrics.json` + `selection_*_outcome.json` (model and operating point), and `firmware/sdkconfig.*` (everything else on the device). Generated files are never edited by hand; each says so in its first line.

7.2 Build entry points

The firmware is always built through `tools/idf.ps1`, never with a bare `idf.py`. The wrapper does four things before calling ESP-IDF:

1. puts Python 3.12 first on the PATH and sources IDF's `export.ps1`;
2. runs `tools/forbidden_deps.py` — the build stops on any hit;

3. runs `tools/gen_frontend_config.py` — regenerates `ps_frontend_config.h` from `frontend.json` only if it changed (keeping the file's timestamp stable avoids needless rebuilds);
4. assembles the `SDKCONFIG_DEFAULTS` list and deletes the build's `sdkconfig` if any defaults file is newer, because ESP-IDF would otherwise keep the old values silently.

Build variants live in separate folders: `build_usbjtag` (capture/demo build), `build_usbjtag_product` (the final product build, `-ExtraDefaults firmware\sdkconfig.product -BuildTag product`), `build_usbjtag_nonet` (no Wi-Fi), `build_uart`.

The model enters the firmware through `tools/gen_model_source.py <artifact folder>`: it checks that `metrics.json` describes exactly that `.tflite` (SHA-256), writes the bytes as a 16-byte-aligned `const` array (so they stay in flash), writes the model's SHA-256 and the front-end SHA-256 its training data used, computes the 256-entry input lookup table *exactly* as the evaluation code does, and writes `sdkconfig.model` from the matching selection outcome.

7.3 Firmware, in execution order

7.3.1 `app_main()` — boot (firmware/main/app_main.c)

`app_main` runs once on core 0. In order:

1. `link_init()` installs the USB-Serial-JTAG driver and redirects every `ESP_LOG` call into framed TEXT messages, so logs can never corrupt binary audio frames.
2. `link_wait_host(30000)` blocks until the PC sends a byte (the capture tool sends `S` when it opens the port) or 30 s pass.
3. `inst_boot_report()` prints `@BOOT` (IDF version, first 9 hex digits of the firmware ELF hash, build date, cores, chip revision, PSRAM flag, flash size, CPU MHz, reset reason); `@CFG` prints the pins, framing and DMA layout; `inst_mem_report("boot")` prints heap totals.
4. **Microphone probes.** `probe()` opens I2S in a given slot mode, discards a settling period, then accumulates per-slot statistics (`slot_stats_t`): mean (DC), RMS, peak, the fraction of words with any of the low 8 bits set (`lowbyte_nz`, which must be ~0 because the INMP441 drives only 24 bits), the fraction whose 24-bit LSB is 1 (`lsb1`, ~50 % when correctly aligned), zeros, and the first non-zero sample after the clock started. The stereo probe runs first so it sees the first clock after boot; `alignment_sweep()` repeats it for Philips/MSB × normal/inverted bit clock and prints `@ALIGN`; the mono probe (EXP-10) then confirms which slot mask carries the microphone.
5. `ps_fe_init()` builds the front end from the constants generated from `frontend.json`; then, only if `ps_fe_tables_fe_sha256` equals the firmware's `PS_FE_JSON_SHA256`, `ps_fe_set_tables()` overwrites the filterbank weights and PCAN lookup table with the host twin's canonical values, and `@FETAB` prints eight table hashes for comparison with the PC.
6. `kws_init()` loads the model (next section) and `@KWS` prints its identity: `run=v2b model_sha256=f7c913efdbb28f64 model_bytes=28024 arena_bytes=16384 arena_used=13404 win=100 bands=40 stride=6 avg_n=3 thr_pct=80 refractory_ms=2000`.
7. `ring_init()` zeroes the ring; `ns_start()` starts NVS, Wi-Fi and the two network tasks on core 0.
8. `xTaskCreatePinnedToCore(reader_task, "i2s_reader", CONFIG_PS_READER_STACK, …, 20, …, 1)` starts the audio pipeline on core 1 at priority 20.
9. The main loop wakes every second: prints `@STATS` (samples, drop counters, I2S overflows, level in dBFS, heap free and heap minimum) and `@PERF` (stack high-water marks, front-end and inference cycle counts, trigger count); every tenth second calls `cpu_report()`; and polls the USB link for a `T` byte, which starts a stream exactly like a detection but marked `synthetic=1` — the hook that made memory and transport tests possible without speaking (EXP-1).

7.3.2 `reader_task()` — the always-on pipeline

This is the most important function in the firmware. Its loop body, lightly abridged:

```
if (ac_read(raw, sizeof raw, &got, 1000) != ESP_OK || got != sizeof raw) { s_short_reads++; continue; }
gpio_set_level(CONFIG_PS_INST_ACTIVE_GPIO, 1);          /* I-1 duty-cycle pin: start of this hop's work */
for (int i = 0; i < HOP; i++) i16[i] = (int16_t)(raw[i] >> 16);   /* bits 23:8 of the 24-bit sample */
ring_write(i16, HOP);                                    /* pre-roll source for the streamer */
const uint32_t c0 = esp_cpu_get_cycle_count();
while (off < HOP) {                                      /* at most one feature frame per hop */
    const int produced = ps_fe_process(i16 + off, HOP - off, &used, feat);
    off += used; if (produced) { fi = fe_frames++; new_frame = 1; }
}
const uint32_t dc = esp_cpu_get_cycle_count() - c0;     /* front-end cycles (I-2) */
if (new_frame) kws_on_frame(fe_frames - 1, feat);       /* inference + decision, timed separately */
/* … level meter, capture-build framing … */
gpio_set_level(CONFIG_PS_INST_ACTIVE_GPIO, 0);
```

Problem it solves: do all periodic work in one place so its cost is exactly the "idle listening" CPU. *Inputs:* 160 × int32 words per call. *Outputs:* ring contents, one feature frame, possibly a trigger. *Why this design:* the hop is the natural clock; one task on one core cannot be pre-empted by Wi-Fi; the cycle counter brackets give per-stage attribution. *Tested by:* feature parity (E-21), posterior parity (E-22), live-equals-replay (E-26), and the CPU runs (E-43, E-56, E-60).

The `>> 16` deserves a note. The microphone's 24-bit sample sits in bits 31:8 of the 32-bit I2S word; shifting right by 16 keeps bits 31:16, i.e. the top 16 bits of the sample. The host capture tool derives its int16 files as `int24 >> 8`, which is the same value, so device and PC always see identical 16-bit audio. The measured room floor was about 52 units RMS in this representation, so quantisation noise is about 45 dB below it and the truncation costs nothing measurable (DEC-09).

7.3.3 `kws_on_frame()` — scheduling, smoothing, decision

```
if (f < KWS_WIN - 1 || (f - (KWS_WIN - 1)) % CONFIG_PS_KWS_STRIDE_FRAMES) return;   /* every 6th frame */
for (int i = 0; i < KWS_WIN; i++) kws_set_row(i, s_fring[(f - (KWS_WIN - 1) + i) % KWS_WIN]);
const float p = kws_invoke(&raw, &cyc);
ring_p[n_inf % CONFIG_PS_KWS_AVG_N] = p; n_inf++;
/* avg = mean of ring_p[0..N-1] */
if (avg >= CONFIG_PS_KWS_THRESHOLD_PCT / 100.0f &&
    (!any_trig || f - last_trig_frame >= CONFIG_PS_KWS_REFRACTORY_MS / PS_FE_HOP_MS)) {
    /* trigger: GPIO15 high for one frame, @KW line, ns_trigger({frame, ring_w(), t_detect, raw}) */
}
```

Three details make the device's behaviour reproducible on the PC:

- The 100-row feature array `s_fring` is a circular buffer of int8 rows; `kws_set_row` writes each row straight into the model's input tensor through the lookup table (the "zero-copy" change from EXP-1 that removed a separate 4 KB window buffer).
- The three-score average starts from zeros, exactly like `np.convolve(p, ones(3)/3, "full")` in `eval_stream.py`, so the first detections after boot match the offline replay (E-26: 27 of 27 trigger times identical).
- `t_ms = 30 + 10·f` — the time of the last sample in frame *f*'s window — is printed with every `@KW` so detections can be located in recorded audio.

7.3.4 `audio_capture.c` — I2S

`ac_open()` creates an I2S0 receive channel in master role with `dma_desc_num = 4` and `dma_frame_num = 160`, a standard-mode slot configuration for 32-bit data (MSB or Philips template from `sdkconfig.board`), the three GPIOs, no MCLK, and registers `on_recv_q_ovf` to count DMA overflows. `ac_read()` is a thin wrapper around

`i2s_channel_read()`. Keeping this component small made it easy to reopen the channel with different framings during the M1 sweep.

7.3.5 `audio_ring.c` — the unified ring buffer

```
static int16_t s_buf[RING_SAMPLES];           /* CONFIG_PS_RING_MS * 16 samples = 12 000 in the product */
static _Atomic uint32_t s_w;                  /* monotonic write index (wraps at 2^32) */

void ring_write(const int16_t *x, size_t n) {
    uint32_t w = atomic_load_explicit(&s_w, memory_order_relaxed);
    for (size_t i = 0; i < n; i++) s_buf[(w + i) % RING_SAMPLES] = x[i];
    atomic_store_explicit(&s_w, w + (uint32_t)n, memory_order_release);   /* publish after the data */
}
int ring_read(uint32_t idx, int16_t *dst, size_t n) {
    /* 0 if not written yet, or if already overwritten (lapped); copy; re-check after copying */
}
```

The indices are *sample counts since boot*, not array positions, so "571 ms before the trigger" is simply `w - 9136`, and "has the writer overtaken me?" is `w - idx > RING_SAMPLES`. The release/acquire atomics make the one-writer (core 1), one-reader (core 0) pattern safe without a mutex on the audio path, and the reader checks again after copying because the writer may lap it mid-copy. It implements the architecture's "retention is a property of not overwriting yet" idea (§8.4). The front end does not need a ring cursor: it consumes the fresh hop directly.

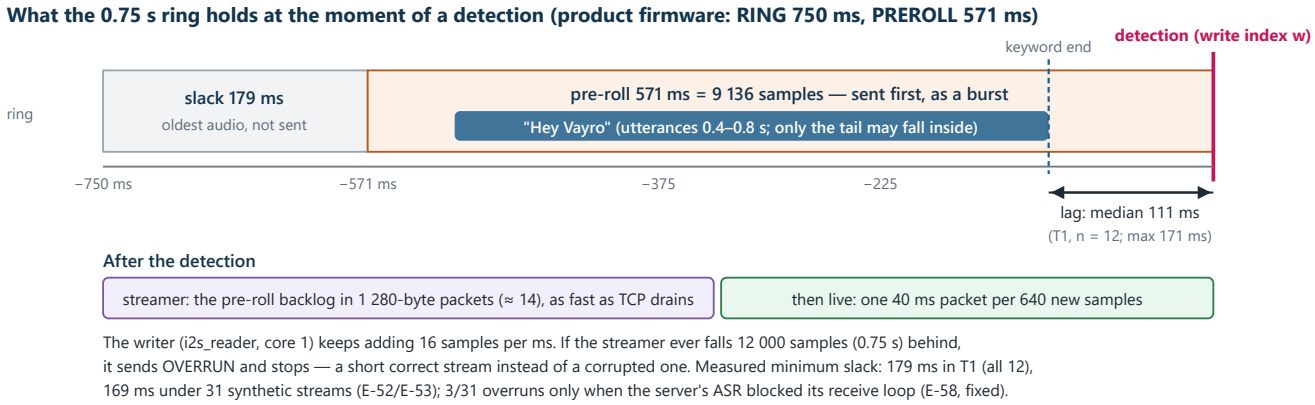


Figure 7.1. The ring at the moment of a detection, and what the streamer does with it.

7.3.6 `microfrontend/` — features

`ps_frontend.c` is a 99-line wrapper around the vendored TensorFlow Lite Micro "microfrontend". `ps_fe_init()` fills a `FrontendConfig` with **every** parameter from `frontend.json` — window 30 ms, step 10 ms, 40 channels, 125–6 500 Hz, filterbank output shift 7, noise-reduction smoothing (10 bits, 0.025 / 0.06, minimum remaining 0.05), PCAN on (strength 0.95, offset 80, 21 gain bits), log scale shift 6 — because the upstream defaults are different (25 ms window, PCAN off) and relying on them would be a silent mismatch. `ps_fe_process()` returns at most one frame per call. `ps_frontend_diag.cc` computes an FNV-1a hash of every table the library builds at init (window, filterbank tables, PCAN lookup, FFT twiddles) and can read or overwrite the two libm-dependent tables — the mechanism behind the M2 parity fix. The int8 conversion used by the model,

```
static inline int8_t ps_fe_to_int8(uint16_t v) {
    int32_t q = (((int32_t)v * 256) + 333) / 666 - 128;       /* TFLM micro_speech mapping */
    return (int8_t)(q < -128 ? -128 : (q > 127 ? 127 : q));
}
```

is defined in the header so that the firmware and `psfe.py` (`to_int8`) use literally the same arithmetic.

7.3.7 `kws_engine.cc` — TensorFlow Lite Micro

```
constexpr size_t kArenaBytes = CONFIG_PS_KWS_ARENA_BYTES;          /* 16 384 in the product build */
alignas(16) uint8_t g_arena[kArenaBytes];                           /* static internal RAM */
tflite::MicroMutableOpResolver<6> g_resolver;                        /* only the ops the model uses */
…
if (std::strcmp(g_kws_model_fe_sha256, PS_FE_JSON_SHA256) != 0) return ESP_ERR_INVALID_VERSION;
g_resolver.AddConv2D(); g_resolver.AddDepthwiseConv2D(); g_resolver.AddReshape();
g_resolver.AddFullyConnected(); g_resolver.AddLogistic();
g_interp = new (g_interp_mem) tflite::MicroInterpreter(model, g_resolver, g_arena, kArenaBytes);
```

The interpreter object is placed in static memory with placement `new`, so nothing comes from the heap at run time. Only five operator kernels are registered, which keeps the flash image small and makes it impossible for a model with any other operator to load. `kws_invoke()` brackets `Invoke()` with the CPU cycle counter and converts the int8 output to a probability with the output tensor's scale and zero point (1/256 and −128 for every model here). Because esp-tflite-micro links esp-nn, the convolutions run on the S3's vector-optimised kernels without any code here asking for them.

7.3.8 `netstream.c` — Wi-Fi, TCP and the streamer

- `wifi_init()` initialises NVS (erasing it once if the partition holds an incompatible layout — the pre-existing Arduino image left one), creates the default station interface, registers an event handler that reconnects on every disconnect, applies the SSID/password from Kconfig, starts Wi-Fi and disables modem sleep.
- `net_task()` waits for an IP address, opens the TCP socket to `CONFIG_PS_SERVER_HOST:5005`, sets `TCP_NODELAY` and a 100 ms receive timeout, then loops: send a PING when due (8 at 0.5 s intervals, then every 10 s), receive, resynchronise on the magic byte, handle PONG (`handle_pong`) and server END. On any error it closes the socket, waits, and reconnects.
- `streamer_task()` blocks on a one-slot queue. For each trigger:

```
uint32_t r = t.trig_sample - (uint32_t)CONFIG_PS_PREROLL_MS * 16;   /* first sample to send */
send_pkt(F_START, 0, &st, sizeof st);
while (ok) {
    if (s_server_end) { reason = 1; break; }                          /* server endpointing */
    if (esp_timer_get_time() > t_stop) { reason = 2; break; }         /* 10 s hard timeout */
    const uint32_t backlog = ring_w() - r;
    if (backlog >= RING_SAMPLES) { send_pkt(F_OVERRUN, seq, NULL, 0); reason = 3; break; }
    if (backlog < PKT_SAMPLES) { vTaskDelay(pdMS_TO_TICKS(5)); continue; }   /* wait for 40 ms of audio */
    if (!ring_read(r, pcm, PKT_SAMPLES)) { /* lapped while copying: OVERRUN */ }
    send_pkt(F_CONT, seq++, pcm, sizeof pcm); r += PKT_SAMPLES;
}
send_pkt(F_END, seq, &en, sizeof en);   /* activation, samples sent, minimum slack, reason */
```

It records the minimum ring slack seen during the stream and the device time of the first audio packet, both printed in `@STREAM`. `ns_trigger()` is non-blocking and refuses when not connected or already streaming — the "one stream at a time" policy.

7.3.9 `serial_link.c` and `instrument.c` — the evidence channel

Frames are `A5 5A 26 17 | type | flags | length (u16) | payload | CRC-16/CCITT-FALSE`. Types: 1 audio (24-bit samples), 2 text, 3 feature frames, 4 model outputs (capture builds only). The USB write uses a 1 s timeout — a 100 ms timeout once lost a frame when Windows stalled — and a mutex so tasks cannot interleave frames. Text lines are limited to 240 characters (one telemetry line once lost its tail, so `@STATS` was split into `@STATS` and `@PERF`). `instrument.c` produces `@BOOT` and `@MEM`; `@MEM` reports internal-heap total, free, minimum-ever free and largest block, plus `psram=disabled_in_sdkconfig`.

7.3.10 `cpu_report()` — how the CPU figure is produced

Every 10 s the main task calls `uxTaskGetSystemState()`, which returns each task's accumulated run time. The difference since the last call, divided by the elapsed run-time ticks, gives each task's share; **core busy = 100 % − idle task share** for `IDLE0` and `IDLE1`. The same line gives the shares of `i2s_reader`, `streamer`, `netlink`, `wifi` and `tcpip_thread`. Interrupt time is charged to whichever task was interrupted, so time spent in interrupts during idle is slightly under-counted — a documented limitation (E-43).

7.3.11 Configuration options (Kconfig)

Table 7.1. Project configuration options: defaults and the final product values

Option	Default	Product	Meaning
`PS_I2S_BCK/WS/SD_GPIO`	4 / 5 / 6	6 / 5 / 4	wiring (sdkconfig.board)
`PS_I2S_FORMAT`	Philips	**MSB**	measured framing
`PS_CAPTURE_STREAM`	y	**n**	binary audio/feature/posterior frames over USB
`PS_LINK_TX_RING`	16 384	2 048	USB driver transmit buffer (bytes)
`PS_KWS_ARENA_BYTES`	40 960	**16 384**	tensor arena
`PS_KWS_STRIDE_FRAMES`	6	6	inference every 60 ms
`PS_KWS_AVG_N`	3	3	scores averaged
`PS_KWS_THRESHOLD_PCT`	90	**80**	generated from the selection outcome
`PS_KWS_REFRACTORY_MS`	2 000	2 000	minimum time between triggers
`PS_RING_MS`	1 000	**750**	ring length
`PS_PREROLL_MS`	624	**571**	audio sent from before the trigger
`PS_READER_STACK`	4 096	3 584	i2s_reader stack (measured high-water mark allowed it)
`PS_STREAM_MAX_MS`	10 000	10 000	device-side stream timeout
`PS_WIFI_PS_NONE`	y	y	modem sleep off
`PS_NET_ENABLE`	n	y	Wi-Fi streaming (enabled when `sdkconfig.wifi` exists)
`PS_SERVER_HOST` / `PORT`	192.168.1.100 / 5005	the PC's LAN address / 5005	compiled-in server address
`PS_MIC_SETTLE_MS`	300	300	settle time before the stereo probe measures

The **capture (demo) build** keeps the defaults for `PS_CAPTURE_STREAM`, `PS_LINK_TX_RING`, `PS_KWS_ARENA_BYTES`, `PS_RING_MS`, `PS_PREROLL_MS` and `PS_READER_STACK`, and has none of the product memory trims (DEC-22, DEC-24): it streams audio, features and model outputs over USB so that sessions can be recorded and replayed, with a 1.0 s ring, a 624 ms pre-roll and a 40 KiB arena. This matters when reading any number: E-60 and E-63 are product-build numbers; the demonstration run (E-61) and most recording sessions used the capture build, whose idle CPU is about 1–2 points higher because of the USB audio traffic.

7.4 Model pipeline code (summary; details in Chapter 8)

Table 7.2. Training and evaluation scripts

File	Role
`model/frontend/psfe.py`	ctypes wrapper of `libpsfe.so`; `Frontend().features(int16)` gives the device's features; `to_int8()` the device's mapping
`model/frontend/build_twin.sh`	compiles the firmware's front-end C files into `libpsfe.so` in WSL

File	Role
`model/data/gen_tts.py`	Piper TTS generation with spelling fixes, speed/prosody randomisation, speaker-disjoint split
`model/data/build_dataset.py`	mixes clips into device-recorded room audio, computes twin features, cuts 1 s windows, labels by keyword end, adds real, hard and household negatives
`model/train/train.py`	builds the DS-CNN, trains with SpecAugment masks and class weighting, first int8 export, metrics
`model/train/export.py`	re-exports with a static batch-1 graph so only 5 operator types appear
`model/eval/eval_stream.py`	replays recordings exactly as the device would (reference kernels, same LUT, stride, averaging, refractory); keyword-end location; prompt scoring
`model/eval/prompt_hits.py`, `household_compare.py`, `event_scan.py`, `fa_context.py`	per-prompt hits, household false activations, event listing
`model/eval/posterior_parity.py`, `eval_device_triggers.py`	device vs host model outputs; live triggers vs offline replay

7.5 Server code

`server/ingest.py` (453 lines) is built from five pieces:

- **`AsrFrontEnd`** — conditions audio for the recogniser only: an 8th-order 150 Hz high-pass filter (removes fan rumble) and a *fixed* gain per stream chosen so that the loudest speech in the pre-roll (the keyword itself, 95th percentile of 50 ms levels in 300–4 000 Hz) sits at −20 dBFS, clamped to 0.5–40×. It holds back the first 571 ms until the gain is known; because the pre-roll arrives as one burst, this adds no measurable delay.
- **`VadEndpointer`** — decides when the command is over: 20 ms frames in 300–4 000 Hz; threshold = keyword level − 20 dB (not below −60 dBFS); speech starts after 3 loud frames; END after 0.7 s of quiet, or after 1.5 s if speech never started.
- **`AsrJob`** — one thread per activation with its own Vosk recogniser, fed through a queue; produces the transcript after the device's END. This is the 29 Sep fix that took ASR off the receive path (E-58).
- **`Session`** — per connection: parses packets, answers PING with PONG using `time.perf_counter_ns()`, tracks sequence gaps, computes `L1_start` and `L1_audio0` in server time, requests END (only after `--min-after-s` 0.8 s of post-trigger audio, or at the 6 s cap), and writes files under a lock. Files are named `c<HHMMSS>_act<NNNN>` with the connection time because the device restarts activation numbers at every reboot and plain names once overwrote ten activations (E-36).
- **`serve()`** / **`main()`** — accept loop with a 1 s timeout (so Ctrl+C works on Windows), one thread per connection, model loading, and `--print-style compact` / `--status-file` for supervised sessions.

The analysis scripts next to it are the measurement instruments: `latency_report.py` (L1, lag and L2 per activation — see Chapter 10), `stream_integrity.py` (finds each server WAV inside the device's own USB recording and checks it sample by sample), `replay_session.py` (feeds a recorded session through the *real* `Session` class to test server changes offline), and `asr_level_test.py` / `make_asr_control.py` / `asr_offline.py` (the controlled ASR experiments of E-45).

7.6 Host tools

- **`host/capture.py`** (501 lines) — opens the USB port without toggling reset lines, resets the board reliably (a Windows driver quirk required esptool's DTR/RTS sequence, E-20), parses frames, writes `audio_24bit.wav`, `audio_16bit.wav`, `features_device.bin`, `posteriors_device.bin`, `device_log.txt`, `device_events.jsonl` (every `@` line as JSON with a host timestamp) and `capture_summary.json`. It detects missing samples from the sample index in each audio frame and **writes zeros for them rather than hiding**

them, rotates files every hour for long recordings, starts a new file set if the device reboots, runs at high process priority, auto-resets after a link stall, shows scripted prompts (`--script-file`), and can send timed bytes such as the synthetic trigger (`--send "T@20"`).

- **`host/analyze_audio.py`** — levels, clipping, zero runs, Welch PSD with the pass-band edge marked, spectrograms (M1 evidence).
- **`host/prompter.py`**, **`host/t1_session.py`** — prompt windows with wall-clock logs; T1 starts the server and the capture hidden, waits for the board to connect, then shows the prompts.
- **`host/scripts/*.txt`** — the recording protocols: keyword sessions v1/v2 (25 keyword prompts at 0.5/1/2 m including fast, slow and soft, plus 16 confusable words and free talk), command sessions v1–v3, and the T1 latency script.
- Demo-recording tools (`demo_take.py`, `demo_cut90.py`, `term.py`, `prompt_window.py`, …) produce the demonstration video; they do not affect the system and are not described further.

7.7 Where the tests live

There is no unit-test framework and no CI server; testing is done by scripts that compare two independent computations of the same thing, run on recorded or live data.

Table 7.3. Test and verification scripts

Script	Compares	Kind of test	Main result
`firmware` probes + `@ALIGN`	four I2S framings on live audio	hardware validation	E-18
`model/frontend/parity_test.py`	device features vs host twin on the same audio	integration (train/deploy parity)	E-21
`model/frontend/compare_tables.py`	device vs host init-table hashes	diagnostic	E-21
`model/eval/posterior_parity.py`	device int8 outputs vs TFLite reference on the PC	integration (deployment parity)	E-22
`model/eval/eval_device_triggers.py`	live device triggers vs offline replay	integration (decision logic)	E-26
`model/eval/eval_stream.py`, `prompt_hits.py`, `household_compare.py`	detections vs scripted prompts / hours of audio	accuracy and reliability	E-25…E-59
`server/stream_integrity.py`	server WAVs vs device USB recording	end-to-end integrity	E-44
`server/latency_report.py`	server receive times vs device detection times + audio	performance	E-37, E-62, E-63
`server/replay_session.py`	server code on recorded sessions	regression	E-48, E-58
`model/frontend/selftest_twin.py`, `model/env/check_converter.py`	smoke tests	environment	—

CHAPTER 8

Data, Algorithms, Models and the Processing Pipeline

This chapter follows the numbers from raw sound to a transcript, explains each algorithm in plain terms first, and then gives the exact parameters used. Everything here is VERIFIED against `model/frontend.json`, the training and evaluation scripts, and `model/artifacts/*/metrics.json`, unless marked.

8.1 The pipeline at a glance

Table 8.1. Every processing stage, with its parameters

Stage	Where	Input → output	Key parameters
Acquisition	INMP441 + I2S DMA	sound → 24-bit samples	16 kHz, MSB framing, left slot
Conversion	`reader_task`	24-bit → int16	`word >> 16` (bits 23:8)
Framing	microfrontend	160 new samples → one 30 ms window	window 480 samples, hop 160
Spectrum	microfrontend	window → 257 frequency bins	Hann window, 512-point real FFT, 16-bit fixed point (KissFFT)
Mel filterbank	microfrontend	bins → 40 band energies	125–6 500 Hz
Noise reduction	microfrontend	subtract a slowly tracked noise floor per band	smoothing bits 10; even 0.025, odd 0.06; min signal 0.05
PCAN gain control	microfrontend	normalise each band by its own recent level	strength 0.95, offset 80, gain bits 21
Log	microfrontend	→ 40 × uint16 (observed range 0–631)	scale shift 6
Quantise	`ps_fe_to_int8`	uint16 → int8	q = (v·256 + 333)/666 − 128
Window	`kws_on_frame`	last 100 frames → 100 × 1 × 40 tensor	1.02 s context, every 6 frames
Model	TFLite Micro	tensor → one int8 score	DS-CNN, 10 665 parameters
Smoothing + decision	`kws_on_frame`	scores → trigger	mean of 3 ≥ 0.80; 2 s refractory
Stream	`streamer_task`	ring → TCP	571 ms pre-roll, 40 ms packets
Endpointing	`VadEndpointer`	stream → END	300–4 000 Hz energy, keyword level − 20 dB, 0.7 s hang
ASR conditioning	`AsrFrontEnd`	stream → clean, level-normalised audio	150 Hz 8th-order high-pass, keyword → −20 dBFS
Recognition	Vosk en-in-0.5	audio → words with times	Kaldi-based, open vocabulary

8.2 Feature extraction: turning sound into 40 numbers

8.2.1 The idea in simple words

A neural network this small cannot learn from raw samples efficiently. Instead, each 10 ms the device describes the last 30 ms of sound by *how much energy there is in each of 40 frequency bands*, spaced the way human hearing is (finer at low frequencies, coarser at high ones — the "mel" scale), on a logarithmic scale (so a quiet and a loud "Hey Vayro" look similar). A sequence of these 40-number frames is a picture of the sound — a spectrogram — and the model learns what "Hey Vayro" looks like in that picture.

8.2.2 What the microfrontend does, step by step

1. **Window.** Take 480 samples (30 ms) and multiply them by a Hann window, which tapers the ends to zero so the frequency analysis is not polluted by the sharp edges of the cut.
2. **FFT.** A 512-point fast Fourier transform (the 480 samples padded to 512) gives the strength of 257 frequencies from 0 to 8 kHz, 31.25 Hz apart. This implementation uses 16-bit fixed-point arithmetic (KissFFT).
3. **Mel filterbank.** 40 overlapping triangular filters, from **125 Hz to 6 500 Hz**, sum the FFT energies into 40 bands. The 125 Hz floor keeps most of the desk fan's 61–70 Hz rumble out; the 6 500 Hz ceiling stays below the microphone's 6 768 Hz pass-band edge, so training audio and device audio see the same band (the study's option O1).
4. **Noise reduction.** For each band, a slowly moving estimate of the background level is subtracted (different smoothing for even and odd bands, keeping at least 5 % of the signal). A steady noise such as a fan fades from the features after a few seconds.
5. **PCAN (per-channel energy normalisation).** Each band is divided by a smoothed version of its own recent energy raised to a power (strength 0.95, offset 80). This acts like an automatic gain control per band, so the same word at 50 cm and at 2 m gives similar features.
6. **Log.** A fixed-point logarithm compresses the range; the output is 40 unsigned 16-bit numbers. On this device they stayed within 0–631.

The microfrontend is the front end of TensorFlow's own "micro_speech" example, used unmodified. Its noise reduction and PCAN state *adapt over time* — which is why the training data is built as continuous streams that start 2.5 s before each clip (see Section 8.4.4): the features of a clip depend on the audio before it, exactly as on the always-on device.

> **KEY IDEA**
>
> The design principle is **one definition, one implementation, two builds.** The same C files produce the device's features (compiled for the Xtensa core) and the training features (compiled for the PC as `libpsfe.so`), and both read the same `frontend.json`. The parity test showed the outputs are bit-identical — after one fix (Chapter 11).

8.2.3 From 16-bit features to the model's int8 input

Two mappings happen in sequence. First, `ps_fe_to_int8()` maps each uint16 feature to int8 with the micro_speech formula `(v·256 + 333)/666 − 128` (so the value 666 maps to +128, i.e. ≈ saturation). Second, because the trained model expects its input in its *own* quantisation (for v2b: scale 0.007567, zero point 4), a 256-entry lookup table, generated from the model's metadata, converts each int8 feature to the model's int8 scale: `lut[q] = clip(round(q/128/0.007567 + 4))`. Training fed the network `q/128` as floats; the lookup table is exactly that scaling, done in integer form. `eval_stream.py` uses the identical table, so the PC and the device feed the network the same bytes.

8.3 The model: a small depthwise-separable CNN

8.3.1 The concept

A normal convolution layer that turns 24 channels into 32 channels with a 5-tap filter needs 24 × 32 × 5 = 3 840 weights and as many multiply-accumulates per output step. A **depthwise-separable** convolution splits that into two cheap steps: a *depthwise* convolution filters each channel on its own (24 × 5 = 120 weights), then a *pointwise* 1 × 1 convolution mixes channels (24 × 32 = 768 weights) — 888 weights instead of 3 840, with little loss of accuracy. This is the idea behind the DS-CNN keyword-spotting models of Zhang et al. (2017) and many microcontroller models since.

8.3.2 This model

The input is 100 time steps × 1 × 40 values. Although the code uses Keras `Conv2D` layers, every kernel is *k × 1*, so the network convolves along **time only**, and the 40 mel bands act as 40 input **channels**. In effect it is a 1-D temporal DS-CNN — the "frequency as channels" layout that the study's TC-ResNet discussion describes. VERIFIED `build_model()`

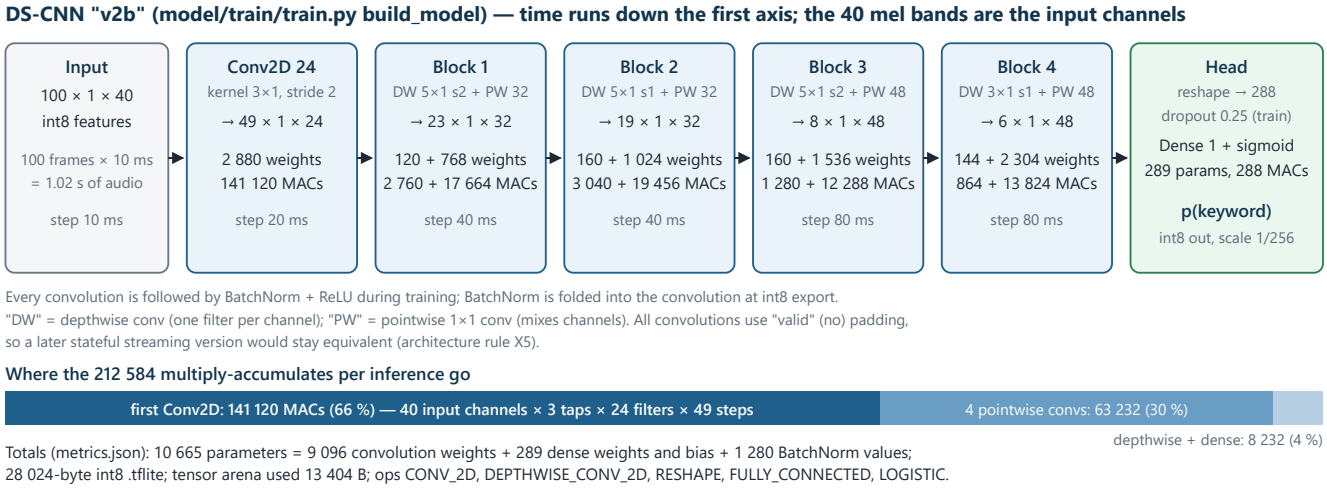


Figure 8.1. Layer-by-layer structure of the shipped model, with output shapes, weights and multiply-accumulates computed from `build_model()`; the totals match `metrics.json` exactly.

Points worth understanding:

- **Strides halve time three times** (10 → 20 → 40 → 80 ms per step), so the final 6 × 48 feature map summarises the whole 1.02 s window; a single dense layer then produces the score.
- **Two-thirds of the arithmetic is in the first layer** (141 120 of 212 584 MACs), because it is the only layer that mixes all 40 input channels at full time resolution.
- **"Valid" (no) padding everywhere.** The study's rule X5: a model with padding behaves differently when later converted to a stateful streaming form. By avoiding padding, the same trained weights could later be run incrementally without retraining (DEC-11). VERIFIED code; the conversion itself was not done
- **BatchNorm after every convolution, then ReLU;** dropout 0.25 before the output during training only. BatchNorm is folded into the convolutions at export, which is why the int8 file contains only 5 operator types.
- **Training from scratch.** No weights are ever loaded; Keras' default initialisers with seed 26172 start every run. This is how the PS's ban on models pre-trained on assistant keywords is met by construction. VERIFIED E-14

8.3.3 Why a windowed model instead of a streaming one

The architecture proposed a *stateful streaming* model: each new frame updates internal state and the network does only the new work every 30 ms, costing about 1 % of a core. The implementation instead re-runs the network on the **whole 100-frame window every 60 ms** (DEC-11, approved by the user on 27 Sep). The reason was schedule risk: int8 streaming state in TensorFlow Lite Micro was the least-proven step in the plan and three days remained. The cost is CPU — inference takes about 3.9 % of core 1 instead of ≈ 1 % — which was measured and still fits under 10 %. DOCUMENTED decision; VERIFIED — cost

8.4 Data

8.4.1 The keyword

"Hey Vayro", pronounced "Hey VAI-ro" (three syllables), was chosen by the user on 27 Sep (DEC-08); "Vayro" alone was considered and declined. The study recommended 3–4 syllables and phonetic distance from common words; the confusable list below shows which words were considered close.

A practical problem appeared immediately: the synthetic voices mispronounced the spelling. Phoneme probing (`model/data/tts_probe.py`) showed all three English voices said /veɪɹoʊ/ ("VAY-ro"); the spelling **"Hey Vyro"** gives /vaɪɹoʊ/, and for the Hindi voice **"हे वाइरो"** is correct. The keyword is unchanged; only the text given to the synthesiser differs (DEC-14). DOCUMENTED

8.4.2 Sources

Table 8.2. Training data sources

Source	What	Used from	Role
Piper TTS, 4 voices	`en_US-libritts_r-medium` (multi-speaker US), `en_GB-vctk-medium` (UK), `en_US-l2arctic-medium` (non-native English), `hi_IN-pratham-medium` (Hindi)	v0	synthetic positives and confusable negatives
LibriSpeech dev-clean	read English speech, first 5 s of each utterance	v0	generic speech negatives; in v4 also the "command" after the keyword
Device-recorded room audio	recordings made through this INMP441 in this room ("beds")	v0	background for mixing, and ambient negatives
Real speaker S01, session 1	23 located keywords + confusables + talk	v2	real positives (3 windows each) and real negatives, plus 6 copies with extra room noise
Night recording, hours 1–5	5 h of the household at night (declared training hours)	v3	extra beds, general negatives, and **hard negatives**
Household evening recording, training part	TV, conversation, prayer, 2 sessions	v4	household negatives and hard negatives; extra beds

8.4.3 Synthetic speech (`gen_tts.py`)

Each clip picks a random speaker of the voice, a random speaking rate (`length_scale`), and random prosody noise. The **v2** set (used by v2a, v2b, v3, v4) was regenerated after the first live tests with two changes, both driven by measurements: faster speech (length scale 0.6–1.2 instead of 0.8–1.35, because the real speaker's keywords measured 0.42–0.78 s against a synthetic median of 0.79 s), and an expanded negative list (1.8× as many negatives) built around the words that had actually fooled v0 on the device — "Hey" alone, "Hey Vayu", "Hey bro", "Hero", "Pyro" — plus many "Hey + name" phrases. VERIFIED code comments, E-27

Table 8.3. TTS v2 plan per voice (clips)

Voice	Positives	Negatives	Note
en_US-libritts_r-medium	1 400	1 980	many speakers
en_GB-vctk-medium	400	540	
en_US-l2arctic-medium	300	450	dataset licensed non-commercial
hi_IN-pratham-medium	120	162	single speaker; training only; non-commercial share-alike
Total	**2 220**	**3 132**	5 352 clips; 4 736 train / 616 validation

Speaker-disjoint split. For each multi-speaker voice, the highest 12 % of speaker IDs go to validation only; LibriSpeech's last 8 speakers are validation only; the real speaker S01 was *excluded entirely* from v0/v1 training (so v0/v1 results on S01 are speaker-independent) and from v2 on only session 1 is used for training, session 2 for validation, session 3 for testing. The study's rule X15 ("split by speaker first, augment second") is followed: every split is by recording, time block or speaker, never by random window. VERIFIED `gen_tts.py`, `sources.json`

8.4.4 Mixing and labelling

`build_dataset.py` never uses a clip on its own. Every example is built as a **continuous stream**: 2.5 s of a randomly chosen device-recorded bed (scaled by a random −10 to +4 dB, to imitate quieter and louder rooms), then the clip added into the continuing bed at a random signal-to-noise ratio of **0–25 dB** (measured in the 125–6 500 Hz band), then 0.4 s more bed. The host twin computes features over the whole stream from a reset state — so, as on the always-on device, noise reduction and PCAN have settled before the clip starts.

Windows of 100 frames are then cut and labelled by the **end of the keyword** (the study's end-of-keyword labelling, X13):

- **positive**: 3 windows whose last frame is 0–160 ms after the keyword's end (chosen at random among the 17 possible);
- **negative "prefix"**: windows ending 200 ms and 350 ms *before* the keyword's end — they contain "Hey Vy…" but not the whole word, which teaches the model not to fire early;
- windows in between are not used (masked);
- for negative clips: windows ending at, and 80/160 ms after, the clip end, and 200 ms before it;
- LibriSpeech: a window every 80 ms across the utterance; ambient: every 100 ms across 8 s of bed.

For real recordings (`add_real_run`) the keywords are located by energy inside each prompt window, positives are taken the same way, and negatives are every 4th window further than 0.3 s before and 1.2 s after any keyword — which includes the spoken confusables and free talk.

Hard-negative mining (v3, v4) is the one step that uses the deployed model's own mistakes: during the overnight and household recordings the device logged its score for every inference; every moment where v2b's live score was ≥ 0.30 in the *training* hours became a negative example (the window at that frame and ±3 frames, repeated 3 times). These are precisely the sounds the shipped model found keyword-like.

Keyword-then-speech positives (v4 only): half of the synthetic keywords were followed, after a 0–250 ms gap, by 0.5–1.5 s of LibriSpeech speech, because v2b missed keywords that were said straight before a command (single high scores suppressed by the 3-score average, E-51). Labels still mark the keyword's end.

8.4.5 Dataset composition

Table 8.4. Windows per source in the training sets (from `metrics.json` data_meta)

Source	v0 / v1	v2a / v2b	v3	v4
TTS keyword (positive)	11 898	11 832	11 832	5 916
TTS keyword + continuation (positive)	—	—	—	5 916
real S01 keyword (positive)	—	483	483	483
TTS prefix (negative)	7 932	7 888	7 888	7 888
TTS confusables (negative)	6 132	11 056	11 056	11 056
LibriSpeech (negative)	121 608	121 608	121 608	121 608
room ambient (negative)	34 000	34 000	34 000	34 000
real S01 non-keyword (negative)	—	16 758	16 758	16 758

Source	v0 / v1	v2a / v2b	v3	v4
night hard negatives (×3)	—	—	8 832	8 832
night general negatives	—	—	29 975	29 975
household hard negatives (×3)	—	—	—	6 501
household general negatives	—	—	—	9 175
Total (positives)	**181 570 (11 898)**	**203 625 (12 315)**	**242 432 (12 315)**	**258 108 (12 315)**
Bed audio available for mixing	115 s / 1 912 s	3 709 s	21 694 s	25 530 s
Validation windows (positives)	44 815 (1 422)	45 569 (1 488)	45 569 (1 488)	45 569 (1 488)

Only about **6 %** of the training windows are positive; the class weighting below compensates.

8.5 Training

Table 8.5. Training configuration (`model/train/train.py`)

Setting	Value
Input scaling	int8 feature q → q / 128 (float)
Augmentation	SpecAugment-style: one frequency mask of 0–3 bands and one time mask of 0–7 frames per example, masked cells set to the feature floor (−1.0)
Loss	binary cross-entropy
Class weight	v0, v1, v2a: balanced (positives weighted ≈ 14–16×); **v2b, v3, v4: positive weight 4**
Optimiser	Adam, cosine-decay learning rate from 1e-3 (v0: 3e-3, which oscillated) over all steps
Batch / epochs	256 / up to 30, early stopping on validation PR-AUC with patience 8, best weights restored
Seed	26172 (TensorFlow, NumPy and the dataset builder)
Hardware	the PC's CPU in WSL (no GPU); v2b trained in 421.6 s (16 epochs)

The validation metric (area under the precision-recall curve on 1 s windows) is used only for early stopping. It is **not** the model-selection criterion — per-window scores say little about detections per hour of continuous audio, so selection was always done on real recordings with the device's own decision logic (next sections).
VERIFIED

8.6 Quantisation and export

The trained float model is converted to **full-integer int8** with TensorFlow Lite's post-training quantisation, calibrated on 500 training windows (250 positive, 250 negative, fixed seed). Inputs and outputs are int8 too. `export.py` then rebuilds the same network with a **fixed batch size of 1** and copies the weights across, because Keras 3's dynamic batch dimension made the converter emit extra SHAPE/STRIDED_SLICE/PACK operators; the static graph contains only CONV_2D, DEPTHWISE_CONV_2D, RESHAPE, FULLY_CONNECTED and LOGISTIC, and gives identical outputs (DEC-17).

Table 8.6. The shipped model's quantisation facts (v2b, `metrics.json`)

Property	Value
File size	28 024 B (SHA-256 `f7c913efdbb28f64…`)
Input quantisation	scale 0.007567, zero point 4
Output quantisation	scale 1/256, zero point −128 (so p = (out + 128)/256, in steps of 1/256)
Largest float-vs-int8 difference on validation windows	0.151 in probability
Tensor arena needed on the device	13 404 B (`arena_used_bytes()`)

The largest float-vs-int8 difference of 0.15 is on individual windows; it is the int8 model that was evaluated everywhere below, so this difference does not affect any reported result.

8.7 Runtime inference and decision

The device runs the model every 6 feature frames (60 ms), i.e. 16.7 times per second, on the most recent 100 frames. The output *p* is averaged over the last **three** inferences (a 120 ms span), and a trigger fires when the average is at least **0.80** and at least **2 000 ms** have passed since the previous trigger. These four numbers (0.80, 3, 6, 2 000) are the "operating point"; they are stored with the model's selection outcome and compiled in through `sdkconfig.model`.

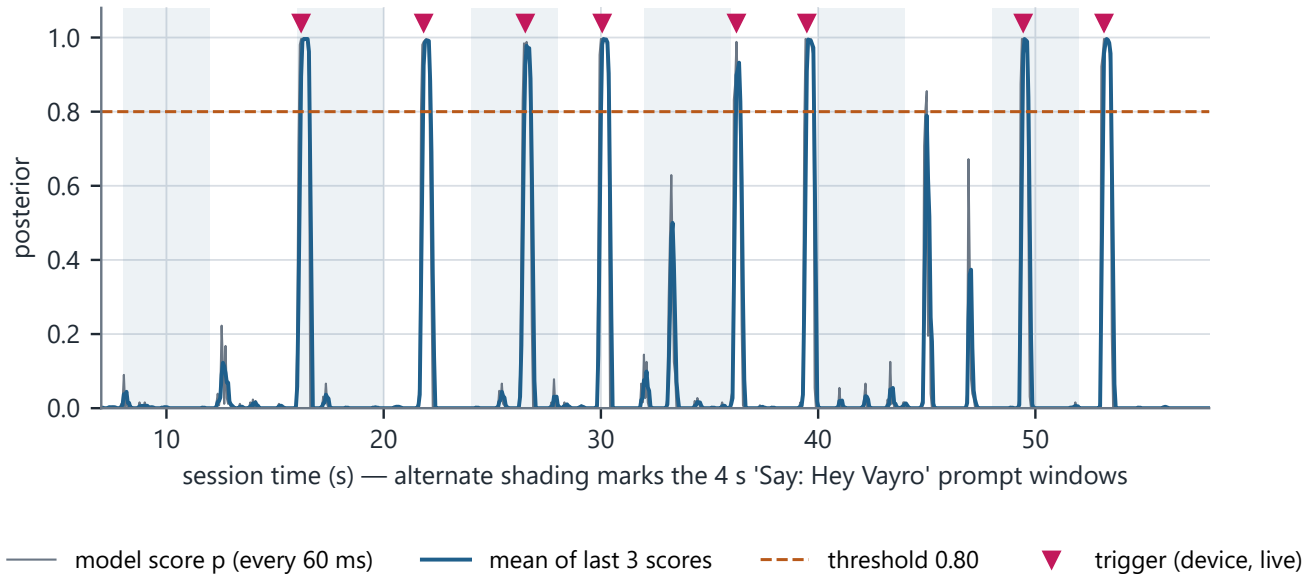


Figure 8.2. The shipped model's scores during the first 50 s of the held-out session S01-s3, recorded live on the device (`posteriors_device.bin`). Each "Hey Vayro" produces a sharp peak; the averaged score must stay above 0.80 across three inferences. The peaks near 33 s and 45 s did not stay high across three inferences and produced no trigger — most likely missed keywords (this part of the session scored 4/6 and 2/3).

Why average three scores? A single high score can come from a keyword-like fragment of other speech; a real keyword gives high scores on several consecutive windows as it slides through the 1 s context. Averaging trades a little latency (≈ 60–120 ms) for fewer false triggers — the same trade the study analysed as "smoothing window" (§8.2). It also causes a known failure: a keyword spoken *immediately* before a command sometimes produced only one high score and was missed (E-51) — the motivation for v4's continuation data.

8.8 Evaluation methodology

8.8.1 Device-identical replay

`model/eval/eval_stream.py` reproduces the device on the PC: host-twin features over the whole recording from a reset state, the same lookup table, TensorFlow Lite's *reference* integer kernels (the closest match to the device's integer arithmetic), inference every 6 frames, the same zero-initialised 3-score average, the same threshold and refractory. Because of parity (E-21, E-22), its detections equal the device's live detections — checked explicitly: 27 of 27 trigger times identical on S01 session 2 (E-26), 39 of 39 on the household recording (E-51), and 12 of 12 on the command session (E-57). This is what makes it legitimate to evaluate several models on the same recording, or to replay a recording through a new model. VERIFIED

8.8.2 Scoring keyword sessions

Recording sessions follow a script of prompts (`host/scripts/kw_session_v2.txt`: 25 "Say: Hey Vayro" prompts at 0.5, 1 and 2 m including fast, slow and soft, then 16 confusable words and 12 s of free talk). Two scorers exist:

- **Prompt score** (primary): a keyword prompt is a *hit* if any detection falls between 0.3 s after the prompt starts and 1.5 s after it ends; a detection inside a confusable, talk or silence prompt (and not in that 1.5 s spill-over) is a *false activation*. It needs no labelling and tolerates speakers who start late. It was introduced after session 2 showed speakers drifting into the next prompt (DEC-20).
- **Labeller score** (secondary): locates each spoken keyword by energy in 300–4 000 Hz (20 ms frames, threshold = the prompt window's 20th percentile + 9 dB, gaps up to 0.3 s bridged, at most 2 s long) and measures the detection lag after the keyword's acoustic end.

8.8.3 False activations per hour

Continuous recordings in which nobody said the keyword are replayed and every detection is a false activation. The result is always reported with its **exposure** (hours) and, where the count is small, an upper bound (for zero events in T hours, the "rule of three": fewer than $3/T$ per hour at 95 % confidence). Household recordings that include a test where the keyword *was* said have those minutes excluded.

8.8.4 Declared splits and selection rules

Before each new model, the data split and the rule for choosing it were written to a file and committed *before* the results existed: `sources.json` (the night split declared before the night was recorded; the household split declared before v4 was built) and `model/artifacts/selection_rule_v2/v3/v4.json`. The outcomes are recorded next to them. Chapter 10 tells the history, including the two occasions when a rule was not followed and why.

8.9 Model versions

Table 8.7. The six models and what distinguished them

Model	Built	Training data	Class weight	Epochs	Best val. PR-AUC	Main result
v0	27 Sep 14:01	TTS v1 + LibriSpeech + short room beds	balanced, lr 3e-3	7	0.928	first on-device model; S01 22/23 offline; fired on "Hey", "Hero", "Pyro" (E-25, E-27)
v1	27 Sep 14:45	as v0, more room beds (1 912 s)	balanced	23	0.949	better validation score but worse on the real speaker (18/23 vs 22/23) → DEC-18: never select on synthetic validation
v2a	27 Sep 18:36	TTS v2 + S01 session 1 + beds	balanced	30	0.921	19/25 at 0.80 but 5/16 confusables on validation
v2b	27 Sep 18:43	same as v2a	**positive weight 4**	16	0.925	**shipped** at 0.80: 22/25 held-out session, 20/25 new speaker
v3	28 Sep 07:01	v2 + night beds + night hard negatives	4	17	0.937	household 19.9 FA/h vs v2b 15.8 → not shipped (E-51)
v4	28 Sep 22:56	v3 + keyword-then-speech + household negatives	4	19	0.921	fewer false triggers (9 vs 13 held-out) but new speaker 14/25 → not shipped (DEC-26)

Build times are the UTC timestamps in `metrics.json` converted to IST. All six share the architecture, 10 665 parameters, 212 584 MACs and 28 024 bytes. VERIFIED

8.10 The recogniser side

The server does three things to the audio before and around recognition:

1. **Conditioning** (`AsrFrontEnd`): device speech arrives quiet (around −40 dBFS at 30–50 cm) on top of fan rumble at about −30 dBFS. A controlled test showed the recogniser turned a clean LibriSpeech sentence at that level with the room's background into the single word "that"; after an 8th-order 150 Hz high-pass

filter and gain it decoded almost perfectly (E-45). The gain is set once per stream from the keyword's level in the pre-roll (speech → −20 dBFS), because a moving gain changed transcripts between near-identical takes (E-48).

2. **Endpointing** (`VadEndpointer`): an energy detector in the speech band ends the stream after 0.7 s of quiet following speech, or 1.5 s if no speech follows the keyword; never before 0.8 s; at most 6 s. It replaced Vosk's own end-of-utterance rule, which on this audio waited for its 5 s silence timeout and kept streams open so long that the next keywords could not be streamed (E-48).
3. **Recognition**: Vosk with the `en-in-0.5` Indian-English model (Kaldi-based, not a transformer), open vocabulary, word timestamps. The words that start before the end of the pre-roll are the keyword; the rest is reported as "after trigger" — the command. A 12-phrase grammar mode exists (`--grammar server/commands.json`) but was a development experiment and is not the validated configuration.

CHAPTER 9

Hardware–Software Integration and Deployment

9.1 Preparing the hardware

1. **Identify and preserve.** Before anything was flashed, esptool identified the chip (ESP32-S3 rev 0.2, 8 MB in-package PSRAM, 16 MB flash, native USB) and the whole 16 MB flash — an unrelated Arduino-ESP32 sketch built in March 2024 — was backed up to `D:\ps26172_backup\preexisting_flash_16MB.bin`. DOCUMENTED PROJECT_STATE §6
2. **Keep the existing wiring.** The INMP441 was already wired (SCK→GPIO6, WS→GPIO5, SD→GPIO4, L/R→GND). Rather than rewire to the study's pin map, the pins were recorded in `firmware/sdkconfig.board` (C-05).
3. **Choose the link.** The board's USB connector is the S3's native USB-Serial-JTAG (vendor ID 0x303A), not a UART bridge. It became the console and data link (DEC-07); it enumerates as COM7.

9.2 Installing the software

Table 9.1. Installation steps and dates

When	What	How / where	Evidence
26 Sep 20:37	ESP-IDF v5.5.5 (git tag) + toolchain	`D:\esp\esp-idf-v5.5.5`, tools in `D:\esp\.espressif`, installed with Python 3.12	PROJECT_STATE DEC-01 DOCUMENTED
26 Sep	host venv	`D:\ps26172\.venv-host` (numpy, scipy, matplotlib, pyserial)	DOCUMENTED
27 Sep 07:33 UTC	training environment	`tools/setup_train_env.sh` in WSL Ubuntu: uv 0.12.19 → CPython 3.12.14 → `~/.venvs/ps26172` → packages from `requirements.lock`	`evidence/m2/setup_train_env.log` VERIFIED
27 Sep 07:44 UTC	Piper voices (4 × 63–79 MB)	`tools/get_piper_voices.sh`	`evidence/m2/piper_voices.log` VERIFIED
27 Sep	LibriSpeech dev-clean	WSL home folder	`sources.json` DOCUMENTED
28 Sep	Vosk 0.3.45 + small en-in-0.4 model	host venv; `D:\ps26172_ext\vosk\`	`evidence/m6_vosk_install.log` DOCUMENTED
28 Sep evening	Vosk en-in-0.5 (971 186 187 B)	`D:\ps26172_ext\vosk\vosk-model-en-in-0.5`	E-49 DOCUMENTED

Every download was approved individually by the user before it happened; the approval list is part of the project's working record.

9.3 Building

```
# capture / demo build (streams audio, features and scores over USB)
powershell -ExecutionPolicy Bypass -File D:\ps26172\tools\idf.ps1 -Console usbjtag -Port COM7 build flash

# product build — the configuration all final numbers refer to
powershell -ExecutionPolicy Bypass -File D:\ps26172\tools\idf.ps1 -Console usbjtag -Port COM7 `
    -ExtraDefaults D:\ps26172\firmware\sdkconfig.product -BuildTag product build flash

# memory report of the last build (static internal RAM, flash)
powershell -ExecutionPolicy Bypass -File D:\ps26172\tools\idf.ps1 -Console usbjtag `
    -ExtraDefaults D:\ps26172\firmware\sdkconfig.product -BuildTag product size
```

Before compiling, the wrapper runs the forbidden-dependency gate and regenerates the front-end header; the model source and `sdkconfig.model` must already have been generated with `tools/gen_model_source.py model/artifacts/v2b`. VERIFIED `tools/idf.ps1`

Table 9.2. Final product build outputs (`evidence/final_product_size_v2b.log`)

Item	Value
Application binary `ps26172_fw.bin`	0xdc030 = 901 168 B (idf_size "total image size" 901 059 B)
App partition (single-app "large" table)	0x177000 = 1 536 000 B → 41 % free
Bootloader	0x5170 = 20 848 B (36 % of its slot free)
Flash code (.text) / flash data (.rodata)	669 272 B / 157 212 B (the model's 28 024 B are in .rodata)
Internal RAM, static (DIRAM)	139 695 B: .bss 65 408 + IRAM .text 54 591 + .data 18 668 + vectors 1 028
Flash size configured / chip size	4 MB / 16 MB

The app partition was enlarged from the default 1 MB when the first Wi-Fi + TFLM build reached 926 247 B and left only 12 % free (DEC-21). VERIFIED

9.4 Flashing

ESP-IDF's `flash` action calls esptool with the addresses from the partition table; the equivalent command printed by the build is:

```
python -m esptool --chip esp32s3 -b 460800 --before default_reset --after hard_reset write_flash
    --flash_mode dio --flash_size 4MB --flash_freq 80m
    0x0 bootloader.bin   0x8000 partition-table.bin   0x10000 ps26172_fw.bin
```

esptool verifies a hash of each region after writing ("Hash of data verified" in `evidence/m3_flash_v0.log`). The final product build's own log contains the build and size steps but **not** the flash step, so the evidence that the measured firmware is the built firmware is the firmware's own identity line: every boot prints the first nine hex digits of its ELF's SHA-256.

Table 9.3. Which firmware produced which final result

Run	`@BOOT elf_sha`	Built	Model / threshold / arena / pre-roll	Result
E-60 final confirmation	`6294c691f`	29 Sep 20:21	v2b / 80 / 16 384 / 571	RAM, CPU, 30 streams
E-61 demonstration (capture build)	`f94cc7e9b`	29 Sep 22:12	v2b / 80 / 40 960 / 624	demo run
E-63 T1 latency	`424ec7782`	30 Sep 08:54	v2b / 80 / 16 384 / 571	latency, 12/12

The T1 firmware was rebuilt on 30 Sep from unchanged sources (no commit touches `firmware/` after 29 Sep 20:20); its ELF hash differs from E-60's because the build time is embedded in the image. Its `@KWS` and `@CFG` lines are identical to E-60's, and the current `build_usbjtag_product\ps26172_fw.elf` hashes to `424ec77828ffb70e…`, matching T1. VERIFIED

9.5 What exactly is deployed

- **Device**: product build of the firmware at commit `9ee1a42` sources (unchanged since), with `sdkconfig.defaults` + `sdkconfig.board` + `sdkconfig.console_usbjtag` + `sdkconfig.model` (v2b: 80 %, N 3, stride 6, 2 000 ms) + `sdkconfig.wifi` + `sdkconfig.product`; model v2b embedded in flash.
- **Server** (T1 command line, from `evidence/server/20260930_085548/server_config.json`):

```
python -u server/ingest.py --port 5005 --asr D:\ps26172_ext\vosk\vosk-model-en-in-0.5
       --min-after-s 0.8 --stream-s-max 6 --status-file <run folder>\status.txt      (endpoint: vad)
```

The model loaded in 11.7 s on the PC in that run. VERIFIED

9.6 Configuration and credentials

The Wi-Fi SSID and password are set in `firmware/sdkconfig.wifi`, which is git-ignored and never committed; `idf.ps1` includes it only if it exists (`-NoNet` builds without networking). Two consequences worth understanding: the credentials end up **inside the firmware image** in flash (anyone with the binary or physical access can read them), and the server address is a compile-time constant (`CONFIG_PS_SERVER_HOST`), so a change of network requires a rebuild (E-41). Traffic is plain TCP on the LAN with no encryption or authentication — the study's deliberate choice to keep a TLS handshake off the latency path and out of the RAM budget. VERIFIED

9.7 Memory allocation: where the 245 KiB goes

Static allocations (decided at link time) include the audio ring (`s_buf`, 24 000 B), the tensor arena (16 384 B), the 100 × 40 feature array (4 000 B), the interpreter object, the USB TX pool and all driver globals; the code copied into IRAM (54 591 B) is counted too. Heap allocations happen once at boot. The `@MEM` lines of the final confirmation run show them step by step: VERIFIED

`runs/20260929_202327_final_v2b_confirm/device_events.jsonl`

Table 9.4. Internal heap during boot and streaming (final firmware, E-60)

After step	Heap free (B)	Change	What was allocated
total internal heap	289 708	—	internal RAM left after static data, IRAM code and 96 KB of cache
boot	264 032	−25 676	USB driver, event queues, IDF start-up
front-end init	252 404	−11 628	microfrontend state (FFT buffers, filterbank, noise and PCAN state)
model init	252 404	0	nothing — the arena and interpreter are static
network start	202 248	−50 156	NVS, Wi-Fi driver and buffers, lwIP, netlink and streamer stacks
reader task started	194 480	−7 768	i2s_reader stack, I2S DMA buffers
lowest while streaming	178 504	−15 976	Wi-Fi TX buffers and lwIP segments during 30 streams

Peak heap use = 289 708 − 178 504 = 111 204 B; S4 = 139 695 + 111 204 = **250 899 B**.

9.8 Debugging and observability interfaces

The system was built to explain itself. Every line the firmware prints over USB starts with a tag and consists of `key=value` pairs; `host/capture.py` turns each into a JSON record with a host timestamp.

Table 9.5. Telemetry lines

Tag	When	Contents
`@BOOT`	boot	IDF version, ELF hash, build time, cores, revision, PSRAM flag, flash size, CPU MHz, reset reason
`@SYNC`, `@CFG`	boot	time waited for the host; pins, framing, DMA, front-end SHA and parameters; network config
`@MEM`	boot steps	internal heap total, free, minimum-ever free, largest block, DMA-capable free, PSRAM policy
`@DIAG`, `@ALIGN`	boot	microphone probe statistics, alignment sweep, mono decision
`@FETAB`	boot	whether canonical tables were applied; eight table hashes
`@KWS`	boot	model name, SHA, size, arena, window, operating point
`@NET`	events	Wi-Fi up (IP), TCP connected / failed (errno) / closed
`@KW`	each detection	frame, time, score, average, raw output, streamed or not, synthetic flag
`@STREAM`	each stream end	reason, audio sent, minimum slack, detection / start / first-audio times, clock offset, RTT

Tag	When	Contents
`@STATS`	every 1 s	samples, drops, I2S overflows, short reads, level (dBFS), peak, heap free and minimum
`@PERF`	every 1 s	stack high-water marks, front-end and inference cycles (mean, max), inference and trigger counts
`@CPU`	every 10 s	core 0 / core 1 busy, and the shares of reader, streamer, netlink, wifi, tcpip

Other debugging hooks: the `T` byte (synthetic trigger, EXP-1), the capture tool's `--send "T@20,T@35"` schedule, stack high-water marks (the reader's measured headroom allowed shrinking its stack from 4 096 to 3 584 B), and the server's `replay_session.py`, which reproduces a live session through the real server code.

9.9 Deployment restrictions (as built)

- The ASR server must run on a PC on the same LAN, at the compiled-in address, awake (a sleeping PC killed the server once, E-58).
- Without a PC reading the USB port, boot waits 30 s before listening.
- No over-the-air update, no provisioning interface, no persistent configuration other than the compiled-in values.
- One stream at a time per device; the server accepts several connections (one thread each) but was only ever tested with one device.
- Plain TCP without authentication: suitable for a trusted LAN only.

9.10 A working-deployment checklist

A correctly deployed product firmware prints, in order: `@BOOT idf=v5.5.5 … emb_psram=0 … cpu_mhz=240`; `@FETAB canonical_tables_applied=1`; `@KWS init=ESP_OK run=v2b model_sha256=f7c913efdbb28f64 … arena_used=13404 … thr_pct=80`; `@CFG net=1 … preroll_ms=571`; `@NET wifi=up`; `@NET tcp=connected`; and the server prints `device connected`. Every `@STATS` line should show `i2s_ovf=0`, and every `@CPU` line core-1 busy below 10 %. VERIFIED against the E-60 and T1 logs

CHAPTER 10

Complete Experimental and Validation History

This chapter reconstructs, in time order, every test that shaped the final system — including the failures and the results that were later corrected. Each test is described in the same format. Dates and times come from commit times, run-folder names and the evidence ledger; results marked VERIFIED were re-computed from the raw files for this report, the rest are DOCUMENTED ledger entries whose raw files exist but were not re-analysed.

Development and validation timeline, 26–30 Sep 2026 (from git commit times and run folders)

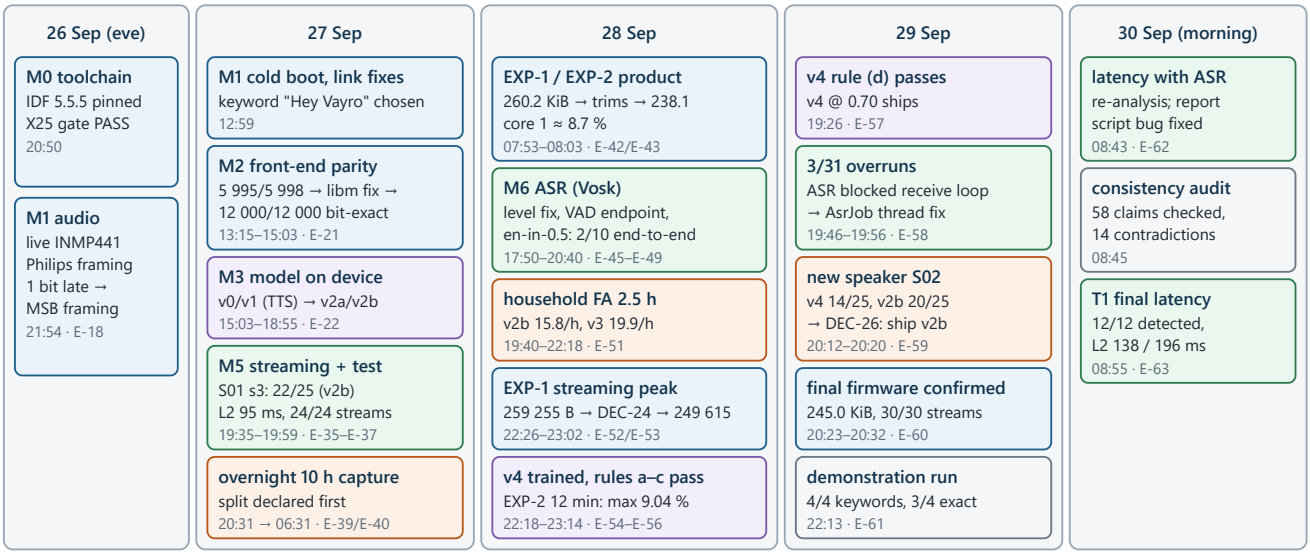


Figure 10.1. The development and validation timeline. Each box is a group of tests or a decision; the ledger IDs point to Appendix A.

10.1 Test categories used in this project

Table 10.1. Kinds of testing that were actually done

Category	What it means here	Examples
Hardware validation	does the physical part behave as its datasheet says, on this board?	I2S alignment sweep, mono mode, start-up, pass band, levels
Integration (parity)	do two independent implementations of the same step agree bit for bit?	device vs host features, device vs host model outputs, live vs replayed triggers, server WAV vs device audio
Functional	does a subsystem do its job?	Wi-Fi/TCP bring-up, streaming with pre-roll, ASR endpointing
Performance	how much memory, CPU and time does it take?	EXP-1 memory, EXP-2 CPU, latency L1/L2
Accuracy and reliability	how often does it detect, and how often does it fire wrongly?	scripted keyword sessions, new speaker, overnight and household exposure
End-to-end	keyword → stream → transcript on the real chain	M6 command sessions, T1
Demonstration	a recorded run of the whole system	E-61

There were **no unit tests** in the software-engineering sense and no continuous-integration server; the architecture's CI gates (§B9) were implemented as scripts run by hand, except the forbidden-dependency gate, which runs automatically on every build.

10.2 Summary of all tests

Table 10.2. Every significant test, in order

When	Test	Category	Key result	Outcome
26 Sep 20:50	M0 build + X25 gate	build	image 241 095 B, gate PASS	pass
26 Sep 21:45	I2S alignment sweep	hardware	Philips 1 bit late; MSB aligned	design changed (MSB)
26 Sep 21:45	EXP-10 mono slot	hardware	63 976 vs 127 963 B/s	pass
26–27 Sep	start-up / settle, levels, integrity	hardware	no zero blanking; DC transient; link faults found and fixed	settle kept at 300 ms
27 Sep 13:42	M2 feature parity	integration	5 995/5 998 frames	**fail** → fixed
27 Sep 14:57	M2 parity after fix	integration	12 000/12 000; 8/8 tables	pass
27 Sep 14:57	M3 model output parity	integration	1 984/1 984 bit-exact	pass
27 Sep 14:57	M3 per-stage cycles	performance	front end 4.66 %, inference 3.11 %	informative
27 Sep 15:11	v0 live on S01 session 2	accuracy	20/25, 6/16 confusables; live = replay 27/27	baseline
27 Sep 18:46	v2 selection	accuracy (validation)	declared rule → v2b@0.98 (12/25); amended → v2b@0.80 (17/25)	rule amended, documented
27 Sep 19:27	"live" v2b test	accuracy	first read as a failure; no keyword had been spoken	**corrected**
27 Sep 19:32	Wi-Fi on/off noise A/B	diagnostic	Wi-Fi hypothesis rejected	pass
27 Sep 19:52	S01 session 3 (held-out)	accuracy	**22/25**, 2/16 confusables	headline recall
27 Sep 19:52	streaming + integrity	functional / integration	24/24 streams, 0 overruns; 23/23 bit-exact	pass
27 Sep 19:52	first latency	performance	L1 1.66 ms; L2 95 ms median	superseded by T1
27–28 Sep night	10 h false-activation exposure	reliability	29 FA / 10 h; 1 in the 5 h held-out block	informative
28 Sep 07:53	EXP-1 non-streaming	performance	260.2 KiB → trims → 238.1 KiB	pass after DEC-22
28 Sep 08:01	EXP-2 first CPU	performance	core 1 8.6–8.9 %	pass
28 Sep 17:50	first ASR session	end-to-end	empty transcripts — no command had been spoken	**corrected**
28 Sep 19:27	M6 command test, small ASR	end-to-end	edge works; ASR 1/4	fail at ASR
28 Sep 20:18	M6 v2, en-in-0.5 + VAD	end-to-end	6/10 detected, 2/10 exact end to end	partial
28 Sep 22:18	household FA, v2b vs v3	reliability	v2b 15.8 /h, v3 19.9 /h	v3 rejected
28 Sep 22:26	EXP-1 streaming peak	performance	259 255 B — fails 256 000	**fail** → DEC-24
28 Sep 22:55	EXP-1 after DEC-24	performance	249 615 B	pass
28 Sep 23:01	EXP-2 12 min	performance	core 1 max 9.04 %	pass

When	Test	Category	Key result	Outcome
28 Sep 23:05	v4 selection rules a–c	accuracy	pass	pending (d)
29 Sep 19:22	v4 rule (d), fresh commands	accuracy	10/10 both	v4 selected
29 Sep 19:46	final v4 firmware + ASR server	performance / functional	3/31 overruns	**fail** → server fix
29 Sep 20:12	new speaker S02	accuracy	v4 14/25, v2b 20/25	v2b shipped (DEC-26)
29 Sep 20:23	final v2b confirmation	performance / functional	245.0 KiB, 30/30 clean, idle 9.32 %	pass
29 Sep 22:13	demonstration run	demonstration	4/4 keywords, 3/4 commands exact	recorded
30 Sep 08:43	latency re-analysis with ASR	performance	L1_audio0 7–11 ms; L2 81 ms (n = 9)	informative
30 Sep 08:55	T1 on the final configuration	performance / accuracy	12/12; L2 138 / 196 ms	headline latency

10.3 Phase 0 — Toolchain (26 Sep)

10.3.1 M0: pinned build and the compliance gate

- **Objective:** a reproducible firmware build from a pinned toolchain, with the ESP-SR gate in place before any code exists.
- **Setup and method:** ESP-IDF v5.5.5 installed natively (not PlatformIO, which pinned IDF 6.1 with GCC 15, DEC-01); `tools/idf.ps1 -Console uart build`.
- **Result:** green build, image 241 095 B, `forbidden_deps: PASS`; the M1 build's static internal RAM was 93 371 B, of which 60 015 B was **code placed in internal RAM (IRAM)** (E-01, E-17). DOCUMENTED
- **Interpretation and next step:** that IRAM line is not in the architecture's memory budget at all. It was logged immediately as conflict C-10 — "this could move the headroom to negative" — and it became the central problem of EXP-1 two days later.

10.4 Phase 1 — Physical audio (26–27 Sep)

10.4.1 I2S alignment sweep (E-18)

- **Objective:** confirm that the samples arriving at the chip are the microphone's samples, bit for bit.
- **Hypothesis:** Philips framing, as the study's compatibility table specified (P-02).
- **Method:** at every boot the firmware captures 0.3 s with each of four configurations (Philips / MSB-justified × normal / inverted bit clock) and reports, per slot, the fraction of samples whose 24-bit least significant bit is 1 (≈ 50 % for real audio; 0 % if the data arrives one bit late) and whether the empty slot stays empty.
- **Result:** Philips: LSB = 1 in 0.00 % of samples, and 3.3 % of the empty slot's words non-zero (sign bits leaking in). MSB-justified: LSB = 1 in 50.85 %, empty slot 100 % zero. Inverted clocks gave the same verdicts. Repeated on two boots. VERIFIED `runs/20260926_214456_m1_align_sweep_warm/device_log.txt`
- **Status:** hypothesis rejected; **design changed** to MSB-justified framing (DEC-06). Root cause (sampling edge or WS timing) not isolated.
- **Why it matters:** with the wrong framing every sample is halved and loses its lowest bit — the audio still *sounds* right, so the error would have survived into training and deployment unnoticed.

10.4.2 EXP-10: does mono mode halve DMA traffic? (E-05)

- **Method:** measure bytes per second delivered by DMA in stereo and in mono mode.
- **Result:** stereo 127 963–127 965 B/s; mono 63 976 B/s (4 boots, identical). VERIFIED
- **Status:** pass — mono is used, which also removes the empty slot from the capture path.

10.4.3 Start-up blanking and settling (E-04, E-19)

- **Hypothesis:** the datasheet's 2^18-clock zero output after power-up (≈ 256 ms).
- **Result:** no zeros — the first non-zero word came 0.56 ms after the clock started, on 3 warm boots and 1 cold power-up. Instead, a DC offset of up to 26 % of full scale decays over ≈ 1.5 s; in the 125–6 500 Hz feature band the level is at the room floor after 100 ms. DOCUMENTED
- **Status:** the study's 300 ms settle time is enough for the features; a proposed 2–3 s settle (C-11) was withdrawn (DEC-10). The sample (one cold boot) is small.

10.4.4 Levels and pass band (E-02, E-03, E-06)

- **Method:** scripted cold-boot session A: silence, speech at 0.5 m, a steady "shhh", speech at 1 m; spectra and 100 ms levels.
- **Result:** feature-band room floor −52.9 dBFS (fan on); words at −38…−42 dBFS peaks; the 6 768–8 000 Hz band 15.8 dB below 4 000–6 768 Hz. DOCUMENTED spectrum re-plotted in Figure 4.1
- **Interpretation:** the level is low but usable; keeping the top 16 bits (`>> 16`) is fine (DEC-09); the mel ceiling at 6 500 Hz is justified by the measured roll-off.

10.4.5 Capture-link integrity (E-07, E-20, E-40)

- **Objective:** make sure recordings used for training and testing have no hidden gaps.
- **Findings and fixes:** (1) a 100 ms USB write timeout lost frames → 1 s; (2) the Windows USB serial driver ignored a bare RTS pulse, so the board was not actually reset and a stalled link persisted → esptool's DTR/RTS sequence and a check that `@BOOT` arrives before audio is accepted; (3) a "USB connected?" check in the firmware briefly read false while the host was reading and dropped frames → removed; (4) a host-side stall watchdog that auto-resets into a new session. DOCUMENTED
- **Result after fixes:** 60 s clean, then the overnight run: 36 000 s, 0 CRC errors, 0 I2S overflows, 1 gap of 20 ms (320 samples), 0 auto-resets. VERIFIED `ambient_night_20260927/capture_summary.json`

10.5 Phase 2 — Front-end parity (27 Sep)

10.5.1 M2 golden-vector parity: first attempt fails (E-21)

- **Objective:** prove the device computes exactly the features the model will be trained on (study §7.3; failure modes X3, X4).
- **Method:** the device streams its int16 audio *and* its 40-value feature frames over USB; the PC recomputes the features from the same audio with the host twin (same C sources); frames are compared value by value, and the front-end SHA-256 must match.
- **Result (60 s):** 5 995 of 5 998 frames identical; 3 values in 3 frames differed (bands 34–35, max difference 26). **FAIL.** VERIFIED `runs/20260927_134200_m2_parity_60s/parity_report.json`
- **Investigation:** a per-table hash diagnostic (`@FETAB`) showed the **filterbank weights differ**: they are computed at start-up with `log1pf()`, and the chip's C library (newlib) rounds differently from the PC's (glibc). The PCAN lookup table's first comparison was inconclusive because the hash included never-initialised slots — the hash was fixed to cover only the entries the code reads.
- **Change:** the host twin's tables are exported to `ps_frontend_tables.c` and copied into the device's state after initialisation, if their recorded JSON hash matches (DEC-16).

- **Result after the fix:** 12 000/12 000 frames bit-exact over 120 s and 3 001/3 001 in a second run; 8/8 table hashes equal. **PASS.** VERIFIED

10.6 Phase 3 — Model on the device (27 Sep)

10.6.1 Deployment parity and footprint (E-22, E-24)

- **Method:** the device sends every raw int8 model output; the PC runs the same `.tflite` with TensorFlow Lite's reference kernels on the host-twin features of the same audio.
- **Result:** 1 984 of 1 984 outputs bit-exact (esp-nn on the device vs reference kernels on the PC). Model 28 024 B; arena used 13 404 B of 40 960 reserved; 10 665 parameters. VERIFIED
- **Meaning:** combined with feature parity, *any* recording can be evaluated on the PC with the certainty that the device would have produced the same scores.

10.6.2 Per-stage CPU with cycle counters (E-23)

- **Result (capture build, v0):** front end 111 758 cycles per 10 ms hop = 4.66 % of core 1 (upper bound: includes queuing the USB frame); inference 448 178 cycles on average (756 202 max) × 16.7 per second = 3.11 %; 0.47 multiply-accumulates per cycle (the study assumed 0.5). VERIFIED `posterior_parity.json`
- **Interpretation:** the pipeline would take ≈ 8 % of one core — three times the study's 2.8 % estimate, but inside 10 %. The windowed model (DEC-11) and the fixed-point microfrontend explain the difference.

10.6.3 v0 — first model, first real speaker (E-25, E-26, E-27, E-28)

- **Setup:** v0 trained on synthetic speech only; S01 was completely excluded from training, so this is a speaker-independent test.
- **Results:** offline on S01 session 1: 22 of 23 located keywords at threshold 0.9 (N = 3); median detection 0.19 s after the keyword's end. Live on the device, S01 session 2: 20/25 keyword prompts, but **6 of 16 confusables fired** ("Hey", "Hero", "Hey bro", "Vayu", "Hey Vayu", "Pyro"). The device's 27 live triggers were **identical** to an offline replay (E-26). 0 false activations in 401 s of held-out daytime audio (too short to mean much: < 27 /h at 95 %). VERIFIED E-26; others DOCUMENTED
- **Consequence:** the confusable failures drove the v2 synthetic data (faster speech, many "Hey + X" negatives). The live-equals-replay result made offline evaluation trustworthy.

10.6.4 v1 and the validation lesson (DEC-18)

v1 scored better than v0 on synthetic validation (PR-AUC 0.949 vs 0.929) but worse on the real speaker (18/23 vs 22/23). Choosing models by synthetic validation would have picked the worse model; choosing by S01 would have been selecting on the test speaker. **Decision:** create a real-speech validation set separate from the test set — S01 session 2 for validation, later sessions for testing (DEC-19). DOCUMENTED

10.6.5 v2 selection — a declared rule, and its amendment (E-30)

- **Declared rule (18:18, before results):** minimise false activations on the 16 confusable prompts of S01 session 2; then maximise recall; ties → the higher threshold. Candidates: v2a (balanced class weight) and v2b (positive weight 4), thresholds 0.80/0.90/0.95/0.98.
- **Outcome:** the rule selected **v2b @ 0.98 — 12/25 recall** (2/16 confusables).
- **Amendment (18:46, before any held-out test):** "highest recall subject to ≤ 3/16 confusables" → **v2b @ 0.80 — 17/25 recall, 3/16**. The recorded reason: the validation set is tiny (differences of 1–2 confusables are not significant), a pure false-activation-first rule collapsed recall to 48 %, and confusable prompts are only a proxy for what really matters (false activations per hour on held-out audio). Both outcomes are recorded in `selection_v2_outcome.json`. VERIFIED

- **Honest reading:** this is a post-hoc choice on validation data. Its protection is that every *test* afterwards (sessions recorded later, the night, the household, the new speaker) was untouched by it.

10.7 Phase 4 — Streaming and the first real test (27 Sep evening)

10.7.1 Network bring-up (E-32)

Wi-Fi station up, warm TCP connection about 1.5 s after boot; internal heap free with Wi-Fi up 64 456 B in the capture build. DOCUMENTED

10.7.2 A "failed" live test that was not a test (E-33, E-34)

- **What happened:** a 150 s live session produced no detection, and the first interpretation was that the model failed in the noisier evening room.
- **Investigation:** a controlled A/B (30 s with Wi-Fi off, 30 s on, same minute) rejected a Wi-Fi interference hypothesis — Wi-Fi *on* was not louder in any band (E-34). The evening room *was* 10–15 dB louder at 5–8 kHz than the afternoon (insects/appliances, cause unconfirmed).
- **Correction:** the user then confirmed that the keyword had **not been spoken** in that session — the prompt was not seen in time. The session was reclassified as 150 s of non-keyword audio with 0 false activations. DOCUMENTED
- **Lesson recorded:** never interpret a result before confirming what was actually said. The same kind of correction happened again on 28 Sep (E-45).

10.7.3 S01 session 3 — the held-out keyword test (E-35)

- **Setup:** recorded after v2b was frozen; protocol v2 (25 keyword prompts at 0.5/1/2 m including fast, slow and soft; 16 confusables; free talk); evening room; Wi-Fi and streaming on; v2b @ 0.80 live on the device.
- **Result: 22/25 keyword prompts** (0.5 m normal 4/6, fast 2/3, slow 3/3; 1 m 6/6, soft 2/2; 2 m 5/5); **2/16 confusables** ("Hey Vayu", "Tyre"); median detection 80 ms after the energy-located keyword end. For information: v0 21/25 & 4/16, v2a 21/25 & 1/16. VERIFIED `model/artifacts/v2b/eval_test_S01_s3.json`
- **Limit:** S01 also recorded the training session 1, so this is **speaker-dependent**; confusable prompts are adversarial, not a false-activation rate.

10.7.4 Streaming integrity (E-36, E-44)

- **Result:** 24/24 detections streamed with pre-roll, each ended by the server, 0 overruns, 0 sequence gaps, minimum ring slack 376 ms (1.0 s ring, 624 ms pre-roll); the keyword's end lay inside the pre-roll in 22/24. Each server WAV was located inside the device's own USB recording and compared sample by sample: **23/23 bit-exact** (the 24th came after the USB capture ended). VERIFIED `server/20260927_192112/stream_integrity.json`
- **Side finding:** the files of 10 earlier, unattended activations had been overwritten because the device restarts activation numbers at each reboot; server files are now named by connection time (E-38 remains unresolved: whether those 10 detections were real is unknown).

10.7.5 First latency measurement (E-37)

- **Method** (used again in every later latency test): the device sends its detection time; the server timestamps the START message and the first audio packet on its own clock; the ping-based offset converts one into the other (uncertainty = half the round-trip time). To get from the *keyword's end* rather than the detection, `latency_report.py` finds the keyword's acoustic end inside the streamed audio itself (speech-band energy, 20 ms resolution) — possible because the stream starts 624/571 ms before the detection — and measures how long before the detection point it lies. The remaining piece, from the last sample in the ring to the detection decision, cannot be seen in the audio; it is taken as **10 ± 7 ms (estimated)**.

- **Result:** detection → START 1.66 ms median (p95 2.18); → first audio 2.12 ms; keyword end → detection 84 ms median (max 144); **L2 ≈ 95 ms median, 136 ms p95** (n = 22). VERIFIED
  `server/20260927_192112/latency_report.json`, re-run 30 Sep with identical numbers
- **Conditions:** capture build, a server without ASR, LAN, one session. Later superseded as the headline by T1.

10.8 Phase 5 — The overnight run (27–28 Sep)

10.8.1 False activations by household activity (E-39, E-40, E-41)

- **Setup:** v2b @ 0.80 live on the device for 10 h (20:31–06:31) with Wi-Fi and streaming on; the user confirmed the keyword was never spoken. **Before recording**, the split was declared in `sources.json`: hours 1–5 (c000–c004) for training v3, hours 6–10 (c005–c009) held out.
- **Result:** 29 false triggers in 10 h, but very unevenly spread: 27 in the first two hours while the household was awake (≈ 13.5 /h), 2 in the 8 h asleep; **1 in the 5 h held-out block (0.2 /h; 95 % upper bound ≈ 0.95 /h)**. VERIFIED Figure 10.2, from `posteriors_device.bin`
- **Network outage (E-41):** the router dropped at 00:44 and came back at 06:08 on a *different subnet*; the device kept detecting and logging throughout, retried TCP every ~20 s, but could not reach the server because its address is compiled in.
- **Interpretation:** "false activations per hour" depends overwhelmingly on whether people are talking. The quiet-night number must never be quoted alone (the ledger's own rule).

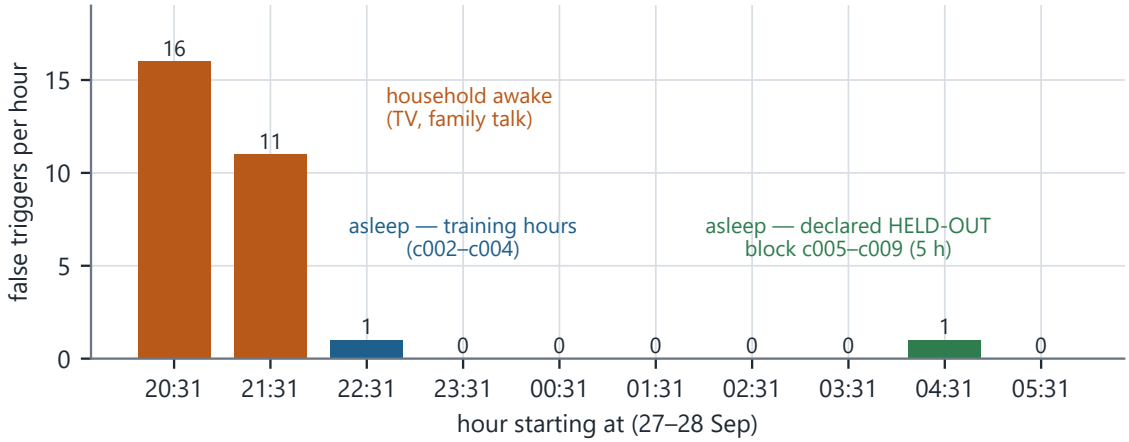


Figure 10.2. False triggers per hour in the overnight run (v2b live on the device). The declared held-out block is the last five hours. Generated from `posteriors_device.bin`.

10.9 Phase 6 — Product configuration (28 Sep morning)

10.9.1 EXP-1: peak memory, first product build (E-42)

- **Objective:** the study's kill-condition experiment — does the whole system fit in 256 KB with PSRAM off?
- **First measurement:** the first product build measured **S4 = 266 483 B (260.2 KiB) — over both limits.** DOCUMENTED
- **Changes (DEC-22):** USB capture streams off (the 64-slot transmit pool alone was ~30 KB of `.bss`); tensor arena 40 → 16 KiB (13.4 KiB used); ring 1.0 → 0.75 s and pre-roll 624 → 571 ms, using the study's own table row for 5-score smoothing because the measured worst-case detection lag was 144 ms (E-37); Wi-Fi buffers trimmed (static RX 4, dynamic RX 16, TX 16 at this point); lwIP send/window buffers 4 096; Wi-Fi, FreeRTOS, ring-buffer and heap functions moved out of IRAM into flash; model input written in place (no separate window buffer); reader stack 3 584.

- **Result:** static 139 123 B + peak heap 104 644 B = **243 767 B (238.1 KiB)**, *not streaming* (the PC and the board were on different subnets after the outage, so no stream could be opened). VERIFIED recomputed

10.9.2 EXP-2: first idle CPU (E-43)

Three 10 s windows on the product build, Wi-Fi associated (TCP not connected): core 1 busy 8.63–8.92 %, core 0 0.74–1.08 %, both cores ≈ 4.8–5.0 %; cycle counters: front end 4.36 %, inference 3.46 % of core 1. VERIFIED Short, but the first product-configuration CPU number.

10.9.3 v3 declared and trained (DEC-23)

v3 = v2 recipe + night beds + hard negatives mined from the *training* night hours; its threshold rule selected 0.60 (20/25 on validation). Ship rule declared first: ship v3 only if it has fewer false activations than v2b on a *new* active-household recording **and** S01 session-3 recall ≥ 21/25.

10.10 Phase 7 — Speech recognition (28 Sep afternoon and evening)

10.10.1 First ASR session, and a second correction (E-45, E-46, E-47)

- **Observation:** with the small Vosk model, live streams produced empty or wrong transcripts, and the first detection → first audio time was ≈ 190 ms.
- **Correction:** the user confirmed that **no command had been spoken** after the keyword in that session; one stream held only devotional singing from another room — a genuine false activation (E-47). The empty transcripts were therefore not evidence of ASR failure.
- **Controlled finding that stood:** a clean LibriSpeech sentence decodes correctly; the same sentence scaled to the device's measured speech level (−40 dBFS) with this room's background decodes to "that"; after an 8th-order 150 Hz high-pass and gain it decodes correctly again. → `AsrFrontEnd`.
- **The 190 ms** was a server artefact: the recogniser was being created inside the receive thread at START; the device had sent the first audio 2.8 ms after the detection. Fixed by pre-creating it (E-46). DOCUMENTED

10.10.2 M6 live command test with the small model (E-48)

- **Setup:** S01, 5 scripted "Hey Vayro … <command>" prompts at 30–50 cm; capture build; Vosk small en-in-0.4, open vocabulary.
- **Result:** edge side worked — 6 triggers, 4 streamed and all 4 bit-exact; L1_start 2.2–2.5 ms, first audio 13.9–17.7 ms; one keyword missed (single-score spike 0.78, average 0.61). **ASR: 1 of 4 correct** ("open the door"). The small model heard the commands as noise, so Vosk ended utterances only on its 5 s silence rule; the long streams then blocked the next keywords (one stream at a time). Offline replay reproduced the live transcripts exactly. DOCUMENTED
- **Status: failing at the ASR stage.** Changes: an energy VAD on the server with a rule fixed before evaluation (`VadEndpointer`), a fixed keyword-referenced gain, and the larger `en-in-0.5` model (user-approved download).

10.10.3 M6 live validation v2 (E-49, E-50)

- **Setup:** 10 new keyword + command prompts (said once each), en-in-0.5 + VAD endpointing; capture build v2b @ 0.80; TV on; household active.
- **Result:** keyword detected in 6/10 prompts (5 streamed; 1 detected but a stream from the user's early repetition was still open); streams: 0 gaps, 0 overruns, L1_audio0 4.9–10.1 ms, VAD END 1.6–2.2 s after the trigger. ASR exact on 2 of the 5 prompted streams ("stop the music", "open the door"); errors "turn off the light" for *turn on*, "sir joseph" for *switch on the fan*; word error rate 7/18 ≈ 39 %. **End to end: 2/10.** The user listened to two streams: the commands were clearly intelligible, so the errors are the recogniser's, not audio damage. DOCUMENTED

- **Capture stalls (E-50):** four USB link stalls happened while the PC was unzipping and loading the 1 GB model (≈ 1 GB free RAM); ≈ 4 minutes affected. Rule adopted: no heavy host work during long captures.

10.10.4 Household false activations: v2b vs v3 (E-51)

- **Setup:** 2.47 h of evening household audio (TV, family talk, prayer), nobody addressing the device; the M6 test minutes excluded. Both models replayed on the same audio with the device-identical pipeline.
- **Result: v2b 39 false triggers = 15.8 /h; v3 49 = 19.9 /h.** v2b's 39 offline detections equal the device's 39 live triggers. On the M6 v2 prompts inside the recording (not part of the rule), v3 detected 9/10 vs v2b 6/10. VERIFIED `household_compare_20260928.json`
- **Decision:** v3's ship rule failed on false activations → **v2b stays** (DEC-23). Analysis of v2b's misses: 3 of the 4 missed prompts had a single high score (0.78/0.74/0.86) but a low average — the keyword was said with no pause before the command.

10.11 Phase 8 — Memory under streaming, and v4 (28 Sep night)

10.11.1 EXP-1 while streaming (E-52 → DEC-24 → E-53)

- **Method:** product configuration, v2b, TCP connected, 31 synthetic triggers (`T` byte) every 10 s, each streaming the 571 ms pre-roll plus 3 s of live audio; static RAM from the size report, peak heap from the lowest free heap during the run.
- **First result:** 141 047 + 118 208 = **259 255 B** — passes 262 144 by 2 889 B, **fails 256 000 by 3 255 B**. Streaming costs ≈ 17 KB of heap at its peak. VERIFIED recomputed
- **Changes (DEC-24, configuration only):** heap poisoning off (a debugging aid), Wi-Fi A-MPDU aggregation off in both directions (a 32 kB/s uplink does not need it, and each block-ack session holds buffers), at most 8 queued Wi-Fi TX buffers, IPv6 off.
- **Result:** 139 695 + 109 920 = **249 615 B** — passes both limits (+6 385 B vs 256 000); 31/31 streams, 0 overruns, minimum slack 169 ms. VERIFIED recomputed

10.11.2 v4 built and selected on its declared rule (E-54, E-55)

- **Declared first** (`selection_rule_v4.json` , commit 3ffc188, amendment before the build): v4 = v3 + keyword-then-speech positives + household training-part negatives and beds. Ship v4 only if (a) fewer false activations than v2b on the held-out household hour, (b) S01 session-3 recall ≥ 21/25, (c) ≤ 2 false activations in the 5 h held-out night, (d) at least as many hits as v2b on a fresh command session recorded after v4 is frozen.
- **Results:** threshold from validation 0.70 (no threshold reached 20/25; best 17/25). (a) held-out household 0.86 h: **v4 9 vs v2b 13** (10.4 vs 15.1 /h) ✓; (b) 21/25 ✓; (c) 0 vs 1 in 5 h ✓. Reported but not in the rule: M6 v2 prompts v4 8/10 vs v2b 6/10. VERIFIED `selection_v4_outcome.json`

10.11.3 EXP-2 for 12 minutes (E-56)

- **Setup:** trimmed product build, v2b, idle listening, Wi-Fi + TCP connected, 72 windows of 10 s, no triggers.
- **Result:** core 1 busy **mean 8.81 %, max 9.04 %**; i2s_reader task 8.78 %; core 0 0.82 %; both cores **4.82 %** mean (max 5.16 %). Cycle counters: front end 4.46 %, inference 3.59 % of core 1. VERIFIED recomputed
- **Status:** pass. Not done: the GPIO/logic-analyser cross-check (no analyser); interrupt time is under-counted by the method.

10.12 Phase 9 — The final decisions (29 Sep)

10.12.1 v4 rule (d): a fresh command session (E-57)

S01 said 10 new commands once each, after v4 was frozen; both models replayed on the USB recording: **v2b 10/10, v4 10/10**, no detections outside the prompts. Rule (d) passed, so by its declared rules **v4 @ 0.70 was selected** and its firmware built. The server could not be reached during this session (the PC's network had not recovered from sleep), so streaming was not measured. VERIFIED `prompt_hits_20260929_192230_m6_v3_S01.json`

10.12.2 Final v4 firmware: overruns with the ASR server (E-58)

- **Result:** RAM 251 495 B and idle CPU (both cores 4.72 %) passed. But with the ASR server in the loop, **3 of 31 streams overran and 1 timed out** — the same configuration had 31/31 clean with the plain server (E-53).
- **Diagnosis:** Vosk decoding ran inside the server's receive loop; it sometimes stalled for up to ≈ 0.6 s; the server stopped reading the socket; TCP flow control backed up into the device; the streamer fell behind; the 0.75 s ring has only ≈ 180 ms of slack beyond the 571 ms pre-roll, so it overflowed. The overruns were correctly flagged — no corrupted audio was delivered.
- **Fix:** `AsrJob` — recognition moved to its own thread per stream; the receive loop only timestamps, runs the cheap VAD and queues audio. Offline replay regression identical. DOCUMENTED fix VERIFIED in E-60

10.12.3 The new speaker (E-59) and the override (DEC-26)

- **Setup:** a second person, S02, recorded the standard keyword session (25 prompts, 16 confusables) on the capture build running v4; neither model had heard S02; no tuning.
- **Result: v4 @ 0.70: 14/25** (0.5 m 10/12, 1 m 3/8, soft 0/2, 2 m 1/5), 0/16 confusables. **v2b @ 0.80: 20/25** (0.5 m 10/12, 1 m 6/8, 2 m 4/5), 1/16. Across all real keyword tests after v4 was built: v2b 58/70, v4 53/70. VERIFIED `eval_newspk_S02_s1.json` for both models
- **Decision (DEC-26, team, 20:20): ship v2b @ 0.80**, overriding v4's declared rule, which had no new-speaker criterion because no second speaker existed when it was written. Accepted cost: more household false triggers (13 vs 9 in the held-out hour). The override is recorded as such in `selection_final.json`.

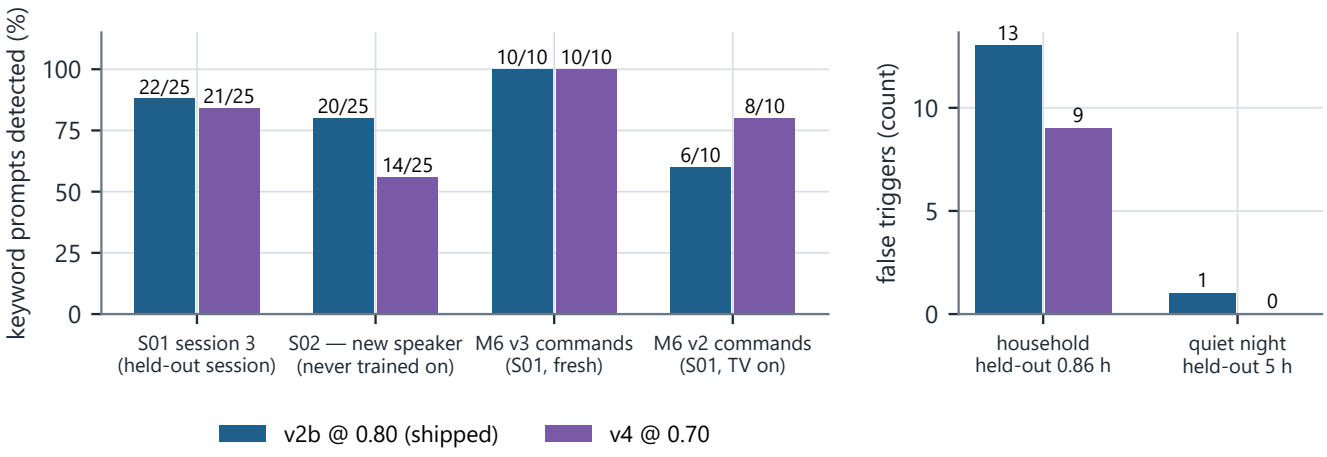


Figure 10.3. The evidence behind DEC-26: keyword detection on four real test sets (left) and false triggers on two held-out exposures (right), v2b vs v4. Generated from the evaluation JSON files.

10.12.4 Final firmware confirmation (E-60)

- **Setup:** product build with v2b @ 0.80 (ELF `6294c691f`), threaded ASR server; 190 s idle, then 30 synthetic streams.
- **Result:** device identity `run=v2b sha f7c913efdbb28f64 thr 80 arena_used 13404`; **RAM 139 695 + 111 204 = 250 899 B (245.0 KiB)**; **30/30 streams ended by the server, 0 overruns**, minimum slack 169 ms (the

server fix works live); **idle CPU core 1 mean 9.32 % (max 9.51 %), core 0 1.74 %, both cores 5.53 %.** VERIFIED recomputed; Figure 10.4

- **Observation made for this report:** during the 30-stream phase core 1 averaged **10.51 % (max 10.83 %)**. The reader task rose from 9.25 % to 9.95 % because inference cycles went up (555 k → 603 k per inference) while the Wi-Fi task was busy — consistent with model weights and Wi-Fi code competing for the flash cache after the IRAM trims INFERRED. The 10 % limit applies to idle listening, so this is not a violation, but it had not been reported before.

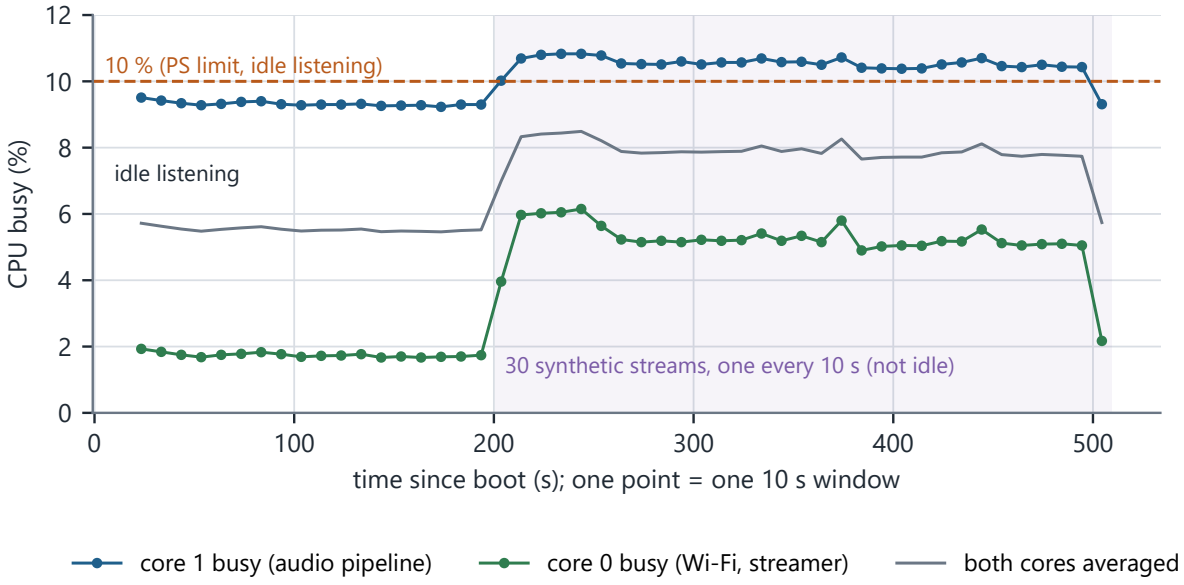


Figure 10.4. CPU per 10 s window in the final confirmation run: idle listening, then one synthetic stream every 10 s. Generated from `device_events.jsonl` (FreeRTOS run-time statistics).

10.12.5 Demonstration run (E-61)

A real-time recording of the real system: capture build (v2b + USB audio for the recording) for the keyword-and-command part, product firmware for the idle part. 4/4 prompted keywords detected (+1 repeat); the server heard "turn on the light", "what time is it", "open the door" exactly and "openly to / place your music" for *play some music* — **3 of 4 commands exact**; 0 triggers while the speaker talked without the keyword; every stream 0 lost packets; first audio 6.5–57 ms after detection; product-firmware idle core 1 8.77 / 8.79 %. The capture build's own CPU lines read 10.6–11.4 % (USB audio + streaming). DOCUMENTED

10.13 Phase 10 — Latency on the final configuration (30 Sep)

10.13.1 Latency with the ASR server in the loop (E-62)

Re-analysis of three 29 Sep sessions after fixing a file-name bug in `latency_report.py` (it had silently skipped every session recorded after the server began naming files `c<time>_act<n>`; E-37 was re-computed identically as a check). Detection → first audio: median 6.7, 7.1 and 10.6 ms in the three sessions (p95 15–42, max 57 ms), versus 2.1 ms without ASR load. Keyword end → server in the draft demo session: 81 ms median, 172 ms p95 (n = 9). The final demo take's L2 was unusable (the command followed the keyword with no pause, so the energy locator found the end of the *command*). DOCUMENTED

10.13.2 T1 — latency and detection on exactly the shipped configuration (E-63)

- **Objective:** replace the 27 Sep latency (different build, no ASR) with a measurement on the final product firmware and the final threaded server.
- **Setup:** `host/t1_session.py` starts the server (Vosk en-in-0.5, VAD endpointing) and the device console hidden, waits for the board to connect, then prompts 12 times "Say: Hey Vayro (then stay quiet)", 6 s apart;

S01 at ~40 cm.

- **Result: 12/12 prompts detected, 0 extra triggers**, 0 lost packets, 0 overruns, all streams ended by the server (minimum slack 179 ms). Detection → START 2.4 ms median; **detection → first audio 18.0 ms median, p95 29.8, max 36.6**; keyword end → detection 111 ms median (p95 171); **L2 = 138 ms median, 196 ms p95, 198 ms max**. Clock uncertainty ±1.2–1.8 ms. The server's transcripts contained only keyword-like words ("cairo", "pyro", …) and nothing after the keyword, as expected; the transcript was ready 43–339 ms (median 135 ms) after the device's END. VERIFIED `server/20260930_085548/latency_report.json` and activation JSONs; Figure 10.5
- **Why it differs from E-37:** the on-device decision lag was 27 ms longer in this session (keyword-end location varies between sessions and speakers; the 60 ms inference cadence quantises it) and the server's receive path added ≈ 16 ms while Vosk was loaded.
- **Limits:** n = 12, one speaker, one quiet session; the capture term is estimated; the study's target of ≥ 100 activations was not reached.

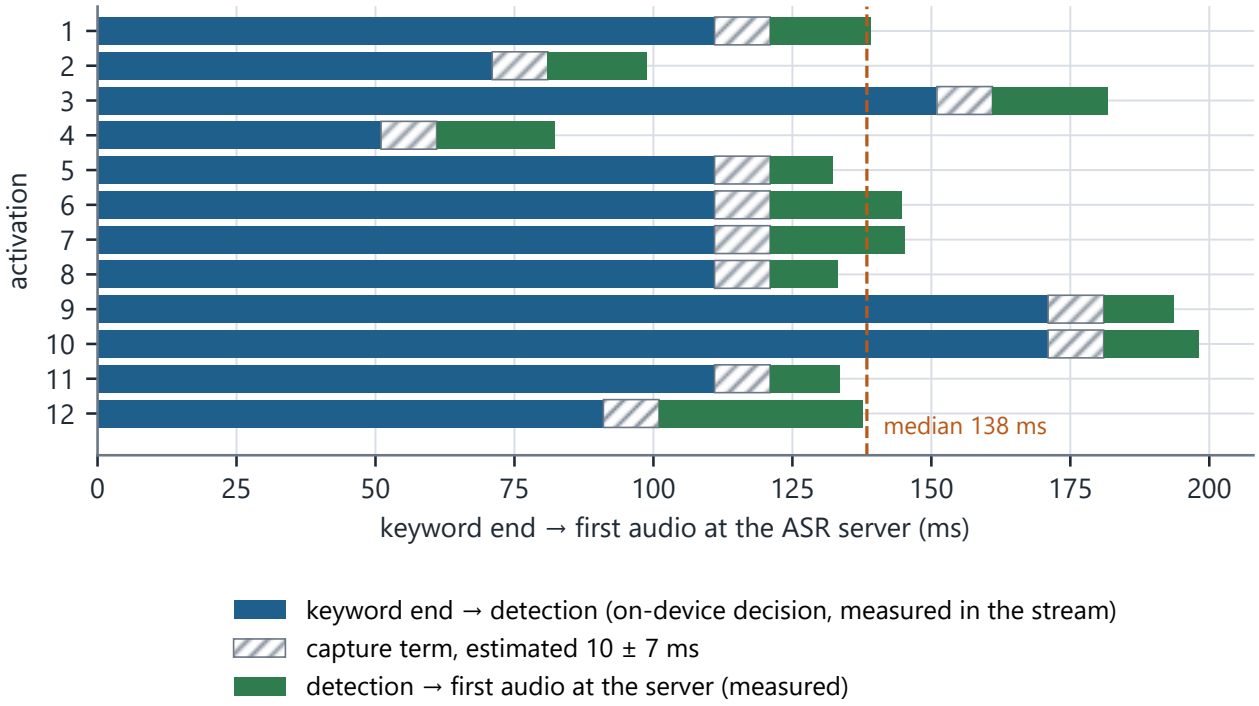


Figure 10.5. T1, per activation: the three parts of the keyword-end → server latency. Generated from `latency_report.json`.

10.14 What cannot be reconstructed reliably

- **E-38:** ten detections on the evening of 27 Sep while no capture was running; their audio was overwritten, and whether anyone said the keyword is unknown.
- **Physical conditions:** speaker distances are nominal (as prompted), room noise levels were not calibrated in SPL, and the TV volume during household recordings was not recorded.
- **Two demo "silent" takes** were discarded because someone spoke to the board; they are not evidence of anything.
- **The capture term** of the latency (≈ 10 ms) was never measured.

CHAPTER 11

Debugging, Failures and Engineering Iterations

Chapter 10 told the story in time order. This chapter takes the most important technical problems one at a time and answers the same eight questions for each: what was observed, under what conditions, the suspected cause, how it was investigated, what evidence confirmed or rejected the cause, what was changed, what happened after, and whether it is resolved. Where the root cause was never proven, it says so.

11.1 Problem 1 — Audio captured one bit late

1. **Observed:** with the planned Philips I2S framing, the lowest bit of every 24-bit sample was always 0, and the supposedly empty right slot contained small non-zero values.
2. **Conditions:** every boot, warm or cold; both bit-clock polarities.
3. **Suspected cause:** the receiver samples the data one bit-clock later than the microphone presents it — a framing mismatch between the INMP441 and the S3's Philips mode on this board.
4. **Investigation:** a built-in sweep over four framings, measuring two signatures that do not depend on what sound is present: the LSB-is-1 fraction (≈ 50 % for real audio) and the empty-slot contents (must be all zero).
5. **Evidence:** Philips 0.00 % LSB, 3.3 % non-zero words in the empty slot; MSB-justified 50.85 %, 0 %. Repeated on two boots. VERIFIED
6. **Change:** MSB-justified framing, locked in `sdkconfig.board` (DEC-06); runs recorded with Philips framing are excluded from training (`sources.json`).
7. **After:** every later recording shows correct statistics.
8. **Status: resolved in behaviour; root cause unconfirmed** — whether it is the S3's sampling edge, the WS timing or the breakout was not isolated (no logic analyser).

11.2 Problem 2 — The evidence link lost data

1. **Observed:** single audio frames missing; once the board was never reset and a stalled link persisted; later, bursts of lost audio during long captures.
2. **Conditions:** Windows host; native USB; heavy PC load.
3. **Suspected causes:** timeouts too short for Windows USB scheduling; the Windows USB serial driver not forwarding a lone RTS change; a firmware "is USB connected?" check reading false while the host was actually reading; host CPU starvation.
4. **Investigation:** sample-index gap detection in `capture.py` (every audio frame carries its first sample index), CRC checks, device-side drop counters (`pool_drops`, `link_drops`), and correlating stalls with host activity.
5. **Evidence:** each fix removed its symptom; the overnight run then had 0 CRC errors and a single 20 ms gap in 10 h; later stalls coincided exactly with unzipping and loading the 1 GB ASR model (E-50). VERIFIED / DOCUMENTED
6. **Changes:** 1 s write timeout; esptool's DTR/RTS reset sequence plus waiting for `@BOOT`; removed the connection gate; 1 MB host receive buffer and high process priority; a stall watchdog that resets and starts a new, separately reported session; missing samples written as zeros *and* reported, never hidden.
7. **After:** clean multi-hour recordings.
8. **Status: resolved**, with an operating rule: no heavy PC work during long captures.

11.3 Problem 3 — Device features did not exactly match training features

1. **Observed:** 3 values in 3 of 5 998 frames differed between the device and the host twin (bands 34–35, up to 26 units).
2. **Conditions:** same audio, same C sources, same `frontend.json`.
3. **Suspected cause:** the only thing not shared is the platform's maths library: tables computed at start-up with floating-point functions (`log1pf` for the filterbank, `powf` for PCAN, `cos` / `sin` for the window and FFT) might round differently on the chip (newlib) and the PC (glibc).
4. **Investigation:** hash every start-up table on both sides and compare (`@FETAB`, `compare_tables.py`); fix a hash that included uninitialised lookup-table slots (it had made the PCAN comparison meaningless).
5. **Evidence:** the filterbank weights differed; the other tables matched once hashed correctly. DOCUMENTED DEC-16; final 8/8 equality VERIFIED
6. **Change:** export the host's tables (`export_tables.py` → `ps_frontend_tables.c`) and overwrite the device's copies after initialisation, guarded by the front-end JSON hash (DEC-16). The host's tables are the "canonical" ones because the training features were computed with them.
7. **After:** 12 000/12 000 and 3 001/3 001 frames bit-exact; model outputs 1 984/1 984 bit-exact.
8. **Status: resolved**. A three-in-six-thousand difference sounds harmless, but it would have made every later "device equals replay" comparison approximate instead of exact.

11.4 Problem 4 — The model's first int8 export contained unsupported-looking operators

1. **Observed:** the converted model contained SHAPE, STRIDED_SLICE and PACK operators besides the expected ones.
2. **Cause (confirmed by the fix):** Keras 3 keeps a dynamic batch dimension through `Reshape` / `Flatten`, and the converter lowers that to shape arithmetic at run time.
3. **Change:** `export.py` rebuilds the network with `batch_size=1`, copies the trained weights and converts that static graph (DEC-17); the Keras `Flatten` was replaced by a `Reshape` with a static size.
4. **After:** only CONV_2D, DEPTHWISE_CONV_2D, RESHAPE, FULLY_CONNECTED, LOGISTIC; identical outputs on S01; the firmware registers exactly these five kernels. VERIFIED
5. **Status: resolved.** A related detail: TensorFlow's CPU depthwise kernel requires equal strides, so the code writes strides `(s, s)`; with a kernel width of 1 and input width 1 this is identical to `(s, 1)` (comment in `train.py`).

11.5 Problem 5 — The synthetic voices said the wrong word

1. **Observed:** Piper's English voices pronounced "Hey Vayro" as "Hey VAY-ro" instead of "Hey VAI-ro".
2. **Investigation:** print each voice's phonemes for candidate spellings (`tts_probe.py`) and write sample WAVs for listening.
3. **Evidence:** "Hey Vayro" → /veɪɹoʊ/ in all three English voices; "Hey Vyro" → /vaɪɹoʊ/; Hindi "हे वाइरो" → /ʊaːɪɾoː/. DOCUMENTED DEC-14
4. **Change:** the synthesiser is given "Hey Vyro" / "हे वाइरो"; the keyword itself is unchanged.
5. **Status: resolved by phonemes**; the planned human listening check was not recorded as done. UNVERIFIED

11.6 Problem 6 — Validation on synthetic data picked the worse model

1. **Observed:** v1 beat v0 on the synthetic validation set (PR-AUC 0.949 vs 0.929) but detected fewer real keywords (18/23 vs 22/23).

2. **Cause:** synthetic speech is a different distribution from a real person through this microphone in this room (failure mode X16 in the study).
3. **Change:** a separate *real* validation session (S01 session 2) for all selection, a *different* real session for testing, and selection rules declared in files before results exist (DEC-18, DEC-19).
4. **Status: resolved as a method.** It is also the root of the v2 rule amendment and the DEC-26 override (Problem 11): with very little real data, every selection decision is fragile.

11.7 Problem 7 — Labelling real keywords automatically was unreliable

1. **Observed:** the first energy-based labeller produced impossible keyword spans (0.02 s, 2.7 s) under fan noise; in session 2, speakers drifted to the end of their 2.5 s prompt windows and into the next prompt, so detections were credited to the wrong prompt (labeller recall 15/25 vs prompt recall 20/25).
2. **Changes:** labeller v2 (speech band 300–4 000 Hz, a threshold relative to each window's own floor, gap bridging, a 2 s cap); recording protocol v2 with 4 s windows and a "next:" preview (DEC-20); a label-free **prompt score** as the primary metric.
3. **Status: mitigated.** Keyword ends located by energy still have 20 ms resolution and can latch onto a command spoken with no pause (which made one latency session unusable, E-62).

11.8 Problem 8 — Results misread before checking what was said

Two separate times, a result was first read as a failure and later corrected once the user confirmed what had actually been spoken: the 27 Sep "no detection in the evening" session (no keyword had been said, E-33), and the 28 Sep "ASR returns nothing" session (no command had been said; one stream held singing from another room, E-45, E-47). In between, a Wi-Fi-interference hypothesis for the evening noise was tested with a controlled on/off A/B and rejected (E-34). **Change:** ground truth is confirmed with the person before a result is interpreted, and both ledger entries were rewritten as explicit corrections that state the original, wrong interpretation. **Status: resolved as practice.** DOCUMENTED

11.9 Problem 9 — The first product build did not fit in 256 KB

1. **Observed:** S4 = 266 483 B (260.2 KiB) — over both readings of the limit.
2. **Conditions:** the first "product" configuration with Wi-Fi, TCP, the model and the ring.
3. **Suspected cause (identified two days earlier as C-10):** the architecture's budget omitted **code placed in internal RAM** — ESP-IDF puts interrupt handlers, parts of FreeRTOS, the heap allocator and, by default, performance-critical Wi-Fi paths in IRAM, and under the strict reading that RAM counts. The capture-mode extras (a 64-slot USB transmit pool, a 40 KiB arena, a 1.0 s ring) added more.
4. **Investigation:** the linker's section sizes (`idf.py size`), per-step heap reports (`@MEM` after each initialisation step), and the stack high-water marks.
5. **Evidence:** IRAM `.text` was 60 015 B in the M1 build and 92 575 B in the first network build; the heap steps attributed 12 040 B to the front end and 53 060 B to Wi-Fi/lwIP/network tasks (E-17, E-29, E-42). DOCUMENTED
6. **Changes (DEC-22):** move Wi-Fi, FreeRTOS, ring-buffer and heap functions from IRAM to flash; capture streams off; arena sized to need (16 KiB); ring and pre-roll shortened using the study's own table (justified by the measured 144 ms worst-case lag); Wi-Fi/lwIP buffer trims; model input written in place; reader stack trimmed after measuring its high-water mark.
7. **After:** 243 767 B not streaming (E-42), then 259 255 B *while* streaming (E-52) — still 3 255 B over 256 000 — then DEC-24 (heap poisoning off, no A-MPDU, 8 TX buffers, no IPv6) → 249 615 B (E-53), final 250 899 B (E-60). VERIFIED

8. **Status: resolved with ≈ 5 kB of margin on the strict reading.** Trade-off: code executed from flash competes with model weights for the cache, which is the most likely reason inference gets slower when Wi-Fi is busy INFERRED (Chapter 12).

11.10 Problem 10 — The speech recogniser heard nothing useful

1. **Observed:** empty or nonsense transcripts; Vosk ended utterances only at its 5 s silence limit; long streams hid the next keyword; transcripts flipped between near-identical takes; the first stream after start-up had an extra ≈ 25–40 ms delay.
2. **Conditions:** device speech around −40 dBFS at 30–50 cm with desk-fan rumble around −30 dBFS; the small `en-in-0.4` model; open vocabulary.
3. **Suspected causes:** level and low-frequency noise far from what the recogniser was trained on; a gain control that moved inside words; a weak model; the recogniser's own endpointing unsuitable; lazy library imports.
4. **Investigation:** a controlled experiment (`make_asr_control.py`, `asr_level_test.py`): one LibriSpeech sentence clean, then at the device's level with this room's recorded background, then with high-pass filtering and gain; and `replay_session.py`, which feeds recorded sessions through the real server code so variants can be compared on identical audio.
5. **Evidence:** clean → correct; device level + background → "that"; + 150 Hz 8th-order high-pass + gain → correct (E-45). Replay reproduced the live transcripts exactly and showed that gain rule, ASR start offset and recogniser persistence each flipped individual results — the small model was fragile (E-48). DOCUMENTED
6. **Changes:** `AsrFrontEnd` (high-pass + one fixed gain per stream, referenced to the keyword in the pre-roll); the larger `en-in-0.5` model; `VadEndpointer` with its rule fixed before evaluation; the recogniser pre-created and numpy/scipy imported at model load.
7. **After:** end to end 2/10 in a noisy live test with the TV on (E-49), 3/4 exact in the demonstration (E-61). The user's listening check showed the delivered audio was intelligible, so the remaining errors are the recogniser's.
8. **Status: partially resolved.** Delivery of audio is solid; recognition accuracy is limited by an off-the-shelf model (publisher's WER 36 %).

11.11 Problem 11 — The ASR server caused stream overruns

1. **Observed:** 3 of 31 streams overran and 1 timed out with the ASR server; 0 of 31 with the plain server on the same firmware.
2. **Suspected cause:** the recogniser ran inside the server's receive loop; while it decoded, the server stopped reading the socket; TCP flow control pushed back to the device; the streamer could not send; the ring filled.
3. **Investigation:** replay timing of Vosk calls (stalls up to ≈ 0.6 s), and the device's own minimum-slack figure per stream (the 0.75 s ring leaves ≈ 180 ms beyond the 571 ms pre-roll).
4. **Evidence:** the arithmetic fits (a 0.6 s stall exceeds 180 ms of slack); the fix removed the symptom. DOCUMENTED cause; fix VERIFIED
5. **Change:** `AsrJob` — one recognition thread per stream fed by a queue; the receive thread only timestamps, runs the VAD and queues.
6. **After:** 30/30 streams clean with the ASR server (E-60); 12/12 in T1. VERIFIED
7. **Status: resolved.** The episode showed that the device's OVERRUN design works: the damaged streams were flagged and ended short, never delivered as corrupted audio.

11.12 Problem 12 — False triggers in an active household

1. **Observed:** ≈ 13.5 false triggers per hour while the family was awake (E-39), 15.8 /h over 2.47 h of TV and conversation (E-51), one from devotional singing (E-47) — against 0.2 /h at night.
2. **Suspected causes:** the training data contained no television speech and almost no real household sound; short keyword-like fragments in continuous speech ("hey", "hero", "pyro"-like sounds).
3. **Attempt 1 (v3):** add the night's training hours as background and hard negatives. **Result:** worse — 19.9 /h (E-51). The night's hard negatives were quiet-room sounds, not speech. VERIFIED
4. **Attempt 2 (v4):** add household TV/talk negatives (hard-mined from v2b's own live mistakes) and keyword-then-speech positives. **Result:** better on held-out household audio (9 vs 13 false triggers in 0.86 h) and on commands, but a new speaker's recall fell from 20/25 to 14/25, collapsing at 1–2 m. VERIFIED
5. **Decision:** v2b ships (DEC-26); the accepted cost is the higher household rate.
6. **Interpretation:** with one training speaker, adding many hard negatives probably taught v4 a narrower idea of "Hey Vayro" INFERRED; the planned fix is more speakers *and* household data together (Chapter 15).
7. **Status: open — the main accuracy limitation of the system.**

11.13 Smaller problems

Table 11.1. Other problems found and fixed (or not)

Problem	Cause	Fix	Status
Firmware would not build from `D:\sih final`	ESP-IDF does not support spaces in project paths	repository at `D:\ps26172` (DEC-02)	resolved
IDF environment script failed	the PC's default Python 3.14	wrapper puts Python 3.12 first	resolved
PlatformIO would pin IDF 6.1 / GCC 15	platform package choice	native ESP-IDF 5.5.5 (DEC-01)	resolved
Changed wiring/policy files silently ignored	ESP-IDF keeps an existing `sdkconfig`	wrapper deletes it when any defaults file is newer	resolved
esp-tflite-micro has no microfrontend	the component's source directory is absent	vendored from tflite-micro `aab6e2fd` (C-03)	resolved
NVS initialisation failed on first network boot	layout left by the pre-existing Arduino image	erase NVS once on a version/space error	resolved
A telemetry line lost its tail	text frames limited to 240 characters	`@STATS` split into `@STATS` + `@PERF`	resolved
Server files overwritten after device reboots	activation numbers restart at 1 on every boot	file names carry the connection time	resolved (10 activations lost, E-38)
Ctrl+C could not stop the server on Windows	`accept()` blocking	1 s accept timeout loop	resolved
Latency report skipped every new session	file-name glob written for the old naming	glob fixed; E-37 reproduced as a check (E-62)	resolved
Server unreachable after the router restarted	server address compiled into the firmware	none	**open** (E-41)
Streams failed when the PC slept	the server process died	operating checklist: PC awake	**open** (procedural)
Boot waits 30 s without a USB host	`link_wait_host(30000)` kept from the evidence build	none	**open**

CHAPTER 12

Performance, Resource Usage and Final Results

Every metric in this chapter is given with its definition, how it was measured, the conditions, the result, the target if one exists, and its limits. Where several measurements exist, the progression is shown and the final validated value is identified. Numbers are VERIFIED (re-computed from raw files) unless marked.

12.1 Memory (RAM)

Definition. S4 = static internal RAM occupied by the application (from the linker: `.bss` + code in IRAM + `.data` + interrupt vectors) + peak heap use (heap total − the lowest free-heap value seen during a run that includes streaming). PSRAM disabled. The 96 KB of internal SRAM configured as CPU cache is excluded as hardware-reserved — a stated interpretation; counting it would add 98 304 B and break any application on this chip.

Method. `idf.py size` on the exact build + `@MEM` / `@STATS` heap minima from the device during 30–31 synthetic streams.

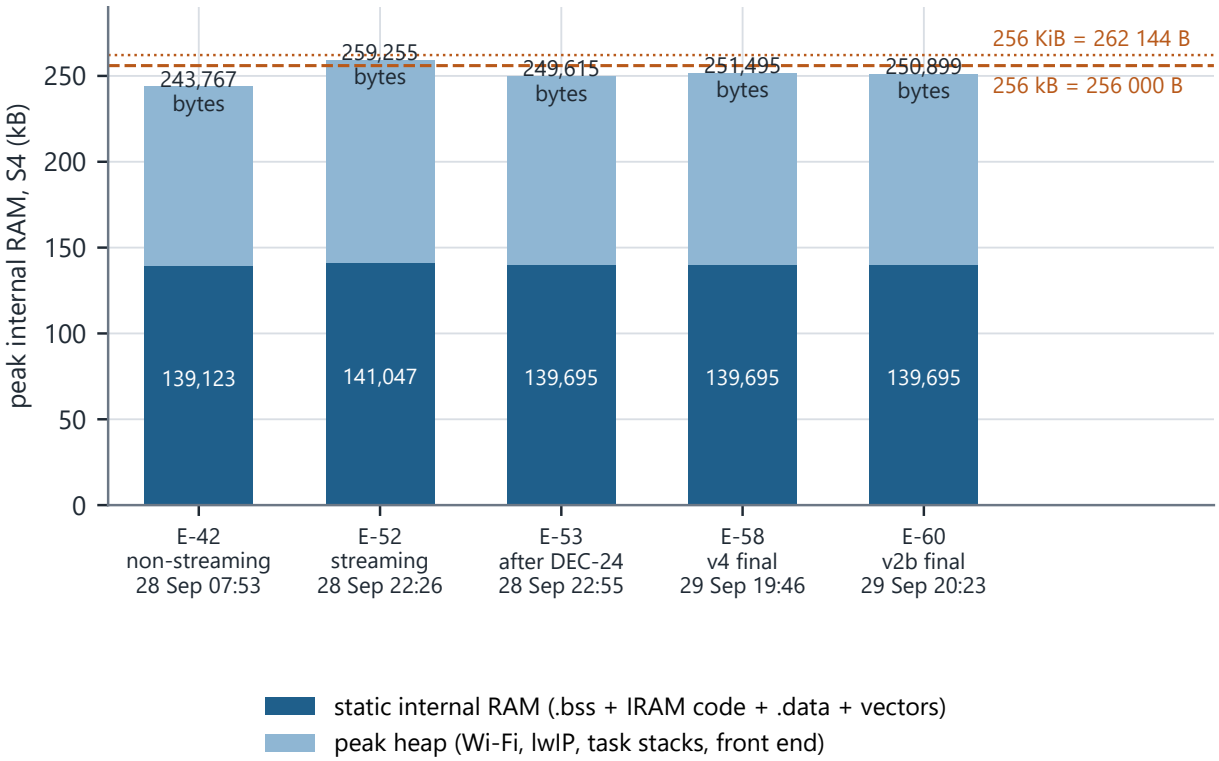


Figure 12.1. Peak internal RAM (S4) at each EXP-1 measurement, recomputed from the size logs and the device's heap minima. The dashed line is 256 000 B, the dotted line 262 144 B.

Table 12.1. Memory progression

Measurement	Static	Peak heap	S4	vs 256 000 B	vs 262 144 B
First product build (E-42 note)	—	—	266 483	−10 483	−4 339
E-42 after DEC-22, not streaming	139 123	104 644	243 767	+12 233	+18 377
E-52 streaming	141 047	118 208	259 255	**−3 255**	+2 889
E-53 after DEC-24	139 695	109 920	249 615	+6 385	+12 529
E-58 final v4	139 695	111 800	251 495	+4 505	+10 649
E-60 final v2b	**139 695**	**111 204**	**250 899**	**+5 101**	**+11 245**

Target: < 256 KB (PS). **Result:** met on both readings. **Interpretation:** the model and its arena are a small part (arena 16 KiB static; weights in flash); the Wi-Fi driver, lwIP, task stacks and IRAM code dominate — exactly as the architecture study predicted, although its total was ≈ 18 KiB too low because IRAM code was missing from its budget. **Limits:** one run per configuration; the heap peak depends on network conditions (streaming TX buffers), so a busier network could raise it; the scopes S0–S3 were not separately measured.

12.2 Flash and model footprint

Table 12.2. Footprint

Item	Value	Source
Model (int8 `.tflite`, in flash)	28 024 B	`metrics.json`, `@KWS`
Parameters / MACs per inference	10 665 / 212 584	`metrics.json`
Tensor arena reserved / used	16 384 / 13 404 B	`@KWS`
Feature window buffer	4 000 B (100 × 40 int8)	code
Firmware image	901 059 B (of a 1 536 000 B partition; 41 % free)	size log
Flash code / read-only data	669 272 / 157 212 B	size log

No flash target exists in the PS; these are reported values.

12.3 CPU utilisation

Definitions. *Core busy* = 100 % − idle-task share, from FreeRTOS run-time statistics every 10 s. *Busiest core* = core 1 (the audio pipeline). *Both cores (C3)* = their mean. *Idle listening* = no keyword, Wi-Fi associated, TCP connected. Cycle counters around the front end and `Invoke()` give per-stage attribution.

Table 12.3. CPU results

Run	Build	Windows	Core 1 busy mean / max	Core 0 mean	Both cores mean	Source
E-43	product (v2b)	3	8.79 / 8.92 %	0.93 %	≈ 4.9 %	first check
E-56	product, DEC-24	72 (12 min)	8.81 / 9.04 %	0.82 %	4.82 %	longest run
E-58	product, v4	18	8.67 / 8.79 %	0.76 %	4.72 %	DOCUMENTED
E-60	**product, v2b (final)**	**18**	**9.32 / 9.51 %**	**1.74 %**	**5.53 %**	final
E-60 streaming phase	same, 1 stream / 10 s	31	10.51 / 10.83 %	5.18 %	7.85 %	not idle
capture build (demo)	USB audio on	822	≈ 10.9 %	—	—	28 Sep household run DOCUMENTED

Per stage on core 1 (E-60 idle): front end ≈ 108 000 cycles per 10 ms hop = **4.5 %**; inference ≈ 555 000 cycles × 16.7/s = **3.9 %**; the rest of the reader task (conversion, ring, level meter) ≈ 0.9 %; total reader 9.25 %.

Target: < 10 % idle (PS). **Result:** met on both readings; the margin is about 0.5 point on the busiest core, 4.5 points on the average. **Interpretation:** the same code measured 8.6–9.5 % on core 1 across runs; inference cycles rise with Wi-Fi activity (492 k → 553 k per inference while the Wi-Fi task went from 0.48 % to 1.31 %), which fits model weights and Wi-Fi code sharing the flash cache after the IRAM trims INFERRED not isolated experimentally. **Limits:** the GPIO duty-cycle method (the study's primary instrument) was not run; interrupt time is charged to the interrupted task, so idle-time interrupts are slightly under-counted; the streaming state exceeds 10 % on core 1.

12.4 Latency

Definitions. *L1_start* = detection → START message read by the server. *L1_audio0* = detection → first audio packet read by the server. *Lag* = keyword's acoustic end → detection, measured in the streamed audio (20 ms resolution). *L2* = keyword end → first audio at the server = lag + capture term (10 ± 7 ms, estimated) + L1_audio0. *L3* (keyword end → server holding audio up to real time) was not computed.

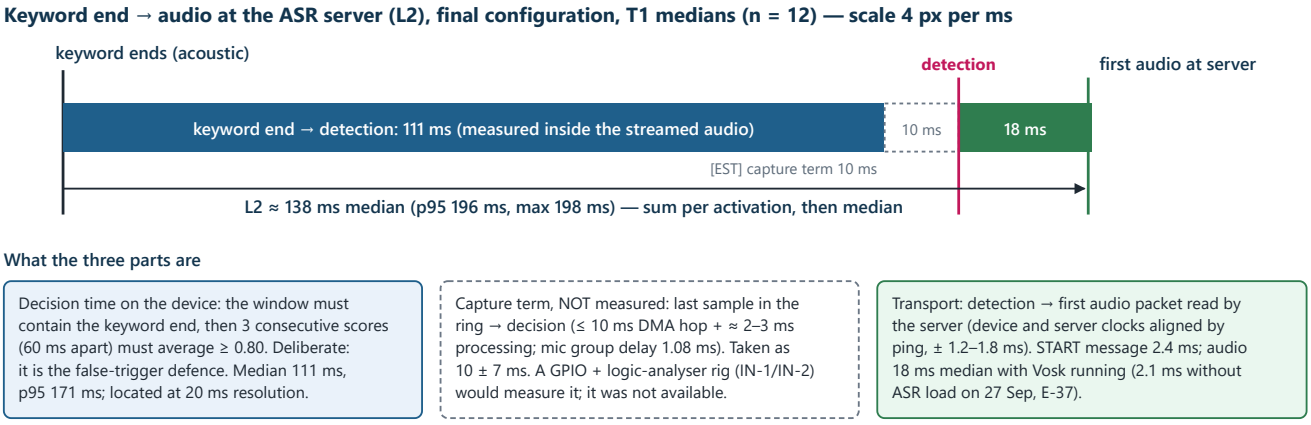


Figure 12.2. How the 138 ms median keyword-end → server latency (T1) is made up.

Table 12.4. Latency measurements (ms)

Session	Build / server	n	L1_start median	L1_audio0 median / p95	Lag median	L2 median / p95 / max
E-37, 27 Sep	capture / no ASR	24 (L2: 22)	1.66	2.12 / 4.27	84	95.4 / 136.3 / 163.7
E-62, 29 Sep draft demo	capture / ASR	9	1.85	6.74 / 15.3	64	81.3 / 171.8 / 179.3
E-62, 29 Sep final confirmation	product / ASR, synthetic triggers	39	1.80	10.6 / 22.0	n/a	n/a (no keyword)
E-63 T1, 30 Sep	**product / ASR (final)**	**12**	**2.38**	**18.0 / 29.8**	**111**	**138.4 / 195.6 / 198.0**

Target: none in the PS. **Final value: 138 ms median, 196 ms p95** (T1). **Interpretation:** about 80 % of the time is the device's deliberate decision time (the keyword must be inside the 1 s window and three consecutive scores must average ≥ 0.80); the network and server add 2–20 ms. Detection → first audio grew from 2 ms to 18 ms when Vosk was running in the server process — server-side scheduling, not the device or the network (the device's own `@STREAM` lines show the first audio leaving 2–3 ms after detection). The architecture study estimated 205 ms for L2; the lower measured value comes from averaging 3 scores rather than 5. **Limits:** n = 12 (the study asked for ≥ 100), one speaker, a quiet home LAN; the 10 ms capture term is an estimate; keyword-end location is energy-based.

12.5 Detection accuracy

Table 12.5. Keyword detection on real speech (v2b @ 0.80 unless noted)

Test set	Speaker	Status of the data	Conditions	Result
S01 session 3	S01 (also in training)	held-out session	0.5 / 1 / 2 m, normal, fast, slow, soft; evening; live	**22/25**; 2/16 confusables
S02 session 1	S02 (never trained on)	new speaker	same protocol	**20/25** (0.5 m 10/12, 1 m 6/8, 2 m 4/5); 1/16
M6 v3 commands	S01	fresh, after v4 froze	keyword + command, quiet	10/10

Test set	Speaker	Status of the data	Conditions	Result
M6 v2 commands	S01	fresh	keyword + command, TV on	6/10
T1	S01	fresh	keyword only, 40 cm	12/12
Demonstration	S01	fresh	keyword + command	4/4
S01 session 2	S01	**validation** (used for selection)	protocol v2	17/25; 3/16 — not a test result

Interpretation. On all real keyword prompts recorded after v4 was built (S01-s3, S02, M6 v2, M6 v3), v2b detected **58 of 70 (83 %)**. The misses are spread over normal, fast and slow speech at 0.5–1 m rather than concentrated at 2 m; a keyword spoken with no pause before a command is a known weak case (E-51). **Limits:** two real speakers, one room; the per-condition cells contain 2–12 prompts each, so differences of one or two are not meaningful; "confusables" are 16 deliberately similar words said once each, not a rate.

12.6 False activations

Table 12.6. False triggers per exposure (v2b @ 0.80)

Exposure	Hours	False triggers	Rate	95 % upper bound	Source
Night, declared held-out block (asleep)	5.0	1	0.2 /h	≈ 0.95 /h	E-39
Night, whole run (2 h awake + 8 h asleep)	10.0	29	2.9 /h	—	E-39
Evening household, TV + talk + prayer	2.47	39	**15.8 /h**	—	E-51
Household held-out hour (rule a)	0.86	13	15.1 /h	—	E-55 (v4: 9, 10.4 /h)
Daytime room, held-out chunk	0.11	0	0	< 27 /h	E-31
12-minute idle CPU run (quiet)	0.20	0	0	< 15 /h	E-56

Target: "near-zero" (PS, no number). **Result:** near zero when nobody is talking; about 16 per hour with television and conversation. **Interpretation:** the false triggers come from speech-like sound, not from noise. The exposures are strongly condition-dependent, so no single "FA/h" describes the system; the two numbers must always be quoted together. **Limits:** one household; about 13 h of exposure in total; TV volume and content not controlled.

12.7 Streaming reliability and integrity

Table 12.7. Stream delivery

Run	Streams	Ended normally	Overruns	Sequence gaps	Bit-exact vs device audio
E-36/E-44 (first test)	24	24	0	0	23/23 checked
E-53 (synthetic, plain server)	31	31	0	0	—
E-58 (synthetic, ASR in receive loop)	31	27	**3** (+1 timeout)	0	—
E-60 (final, threaded ASR)	**30**	**30**	**0**	**0**	—
E-63 T1	12	12	0	0	—

The minimum ring slack is a direct health indicator: 376 ms with the 1.0 s ring (E-36), 169–179 ms with the 0.75 s product ring. **Result:** reliable on a home LAN once the server's receive path was freed. **Limit:** not tested on a congested network, which the study flagged as the main risk for L3.

12.8 Speech-recognition quality

Table 12.8. ASR results (the recogniser is off-the-shelf; these describe it, not the edge system)

Test	Model	Result
Publisher's benchmark	Vosk en-in-0.5	WER 36.12 % (NPTEL) DOCUMENTED SBOM

Test	Model	Result
M6 command test, 28 Sep	small en-in-0.4, Vosk endpointing	1/4 correct (E-48)
M6 v2, 28 Sep (TV on)	en-in-0.5 + VAD	2/5 streamed commands exact; WER 7/18 ≈ 39 %; end to end 2/10 (E-49)
Demonstration, 29 Sep	en-in-0.5 + VAD	3/4 commands exact (E-61)
T1, 30 Sep (keyword only)	en-in-0.5 + VAD	nothing reported after the keyword in 12/12 (correct); transcript ready 135 ms median after the stream ended

12.9 Stability

- **10 hours continuous** listening with Wi-Fi and streaming enabled: no crash, no auto-reset, 0 I2S overflows; it kept detecting through a 5.4 h router outage (E-39–E-41).
- **12 minutes** of idle CPU measurement without a single stray trigger (E-56).
- No long-duration test of the *product* build beyond 12 minutes exists. VERIFIED

12.10 Data overhead

Nothing but 72 bytes of clock-sync data every 10 s is sent while idle. During a stream: 32 000 B/s of audio in 1 288-byte application packets (0.6 % header overhead), about 3.7 % including TCP/IP headers INFERRED calculated, not captured. A keyword with no command costs about 67 kB; with a typical command about 110 kB.

12.11 Power

Not measured. UNVERIFIED

12.12 Final results at a glance

Table 12.9. The final, validated numbers and where each comes from

Metric	Final value	Evidence	Label
Peak RAM (S4)	250 899 B = 245.0 KiB	E-60	VERIFIED
Idle CPU, busiest core	9.32 % mean, 9.51 % max (8.6–9.5 % across runs)	E-60, E-56	VERIFIED
Idle CPU, both cores	5.53 % mean (4.7–5.5 % across runs)	E-60, E-56, E-58	VERIFIED
Model	28 024 B, 10 665 params, 212 584 MACs, arena 13 404 B	E-24	VERIFIED
L2 keyword end → server	138 ms median, 196 ms p95 (n = 12)	E-63	VERIFIED + one INFERRED term
Detection → first audio	18 ms median, 30 ms p95	E-63	VERIFIED
Recall, held-out session / new speaker	22/25 / 20/25	E-35 / E-59	VERIFIED
False triggers, quiet / TV + talk	0.2 /h / 15.8 /h	E-39 / E-51	VERIFIED
Stream delivery	0 overruns, 0 gaps in the final runs	E-60, E-63	VERIFIED
Power	not measured	—	UNVERIFIED

CHAPTER 13

The Most Distinctive Engineering Contributions

13.1 How this chapter judges "innovation"

Using a neural network on a microcontroller, streaming audio over Wi-Fi or running an open-source recogniser is not innovative in itself — thousands of projects do each of these. This chapter looks for **specific engineering decisions in this build that solve a problem that the ordinary approach would handle badly**, and it holds each one to evidence. It makes no claim of global novelty or patentability: the related literature was not surveyed, and several ideas below were proposed by the architecture study and are credited to it.

Table 13.1. Ordinary choices, for contrast (necessary, well-established, not claimed as contributions)

Choice	Why it is ordinary
TensorFlow Lite Micro + esp-nn on an ESP32-S3	Espressif's standard path for int8 inference
DS-CNN keyword model, int8 post-training quantisation	a standard microcontroller keyword-spotting design since 2017
The TFLM microfrontend (log-mel with noise reduction and PCAN)	TensorFlow's own "micro_speech" front end
Vosk (Kaldi) as the recogniser	a common self-hosted open-source ASR
TCP streaming of 16-bit PCM	the simplest reliable transport

13.2 Level A — Conceptual contributions

13.2.1 A1. Making every requirement falsifiable before building

The challenge. The PS's limits ("< 256 KB RAM", "< 10 % CPU", "near-zero false activations", "latency from keyword end") are ambiguous enough that almost any system could be *described* as compliant.

Why the ordinary approach falls short. The usual practice is to report the most favourable number — model size instead of total RAM, one core's inference time instead of system CPU, detection-to-send time instead of keyword end to server, accuracy on a clean test set instead of false triggers per hour.

The choice made. Adopt the strictest defensible reading of each requirement *before* measuring, and measure it with instruments the reader can re-run: S4 includes IRAM code, every stack and the Wi-Fi buffers with PSRAM disabled, and is checked against both kB and KiB; CPU is reported per core *and* averaged; latency starts at the keyword's acoustic end; false triggers are reported per condition with exposure hours; selection rules and data splits are committed before the data they judge exists.

Evidence of usefulness. It changed outcomes: the strict memory reading exposed the IRAM omission and forced real engineering (260 → 245 KiB); the strict latency reading replaced a flattering 95 ms with the measured 138 ms; the declared rules made two departures from them visible and documented instead of silent (the v2 amendment and DEC-26). **What remains unproven:** that evaluators read the requirements the same way. **Credit:** the framing (nested scopes, three latency definitions, kill conditions) is the architecture study's; the execution and the evidence are this project's.

13.2.2 A2. Privacy as a property of the data flow

The device transmits nothing but clock-sync packets until a detection, and the stream is bounded (571 ms before, the command, a hard 10 s cap). This is a design property that can be checked in code (`ns_trigger` is the only path that starts a stream). The project also states the honest corollary: a false trigger sends 2–6 s of room audio, so privacy is exactly as good as the false-trigger rate. VERIFIED

13.3 Level B — Architectural contributions

13.3.1 B1. A front end that is bit-identical on the chip and in training

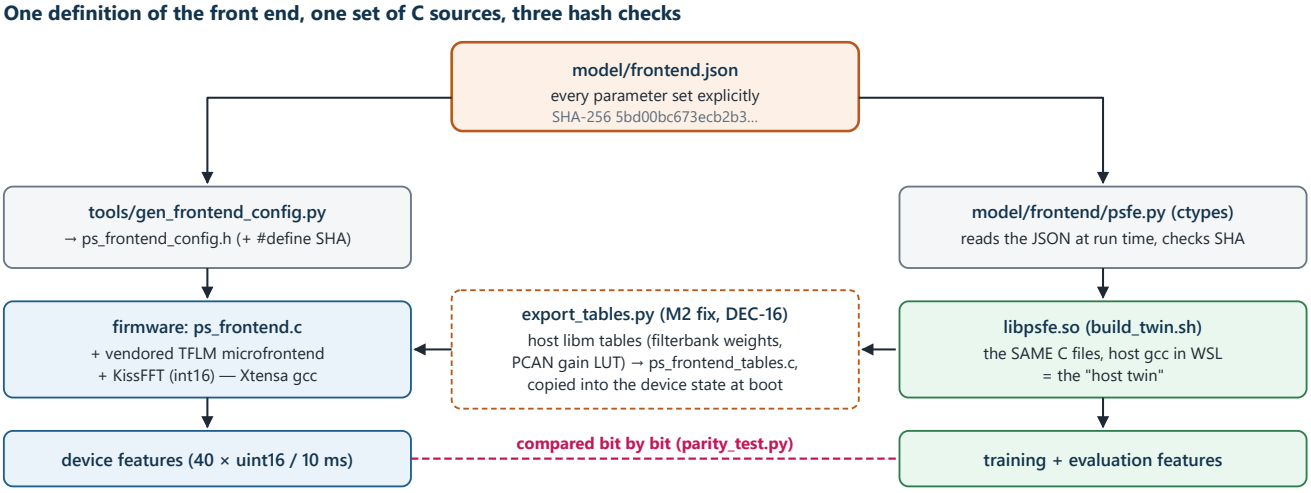


Figure 13.1. How train/deploy parity is enforced: one definition, one set of C sources compiled twice, canonical start-up tables, and three hash checks.

The challenge. A keyword model learns the exact statistics of its input features. If the device computes them even slightly differently from the training pipeline, accuracy degrades silently — the study lists this as failure modes X3 and X4.

Why the ordinary approach falls short. Typical pipelines compute training features with a Python library (librosa, TensorFlow ops) and device features with C code, and check them only by eye or with a tolerance. Even a shared library can differ across platforms, as this project found.

The choice made and how it works. (1) One JSON file defines every feature parameter; the firmware compiles it into a header, the training code reads it at run time, and both carry its SHA-256. (2) The *same* C source files — the vendored microfrontend and KissFFT — are compiled for the Xtensa core and for the PC (`libpsfe.so`, called from Python through ctypes). (3) Because the chip's maths library computes two start-up tables differently from the PC's, the PC's tables are exported and copied into the device at boot, again guarded by the JSON hash. (4) The model records the front-end hash its training data used, and the firmware refuses to run a model built for a different front end.

Components. `frontend.json`, `gen_frontend_config.py`, `psfe.py`, `build_twin.sh`, `export_tables.py`, `ps_frontend_diag.cc`, `parity_test.py`, `kws_init()`.

Trade-offs. It rules out the faster float esp-dsp FFT the study wanted (the FFT must be the same code on both sides), costing CPU; the canonical tables add 1 402 bytes of flash (288 + 288 + 125 16-bit values).

Evidence. 12 000/12 000 feature frames and 1 984/1 984 model outputs bit-exact; live device triggers identical to offline replay on three separate recordings (27/27, 39/39, 12/12). VERIFIED **Unproven:** parity was tested on room audio and speech at normal levels; extreme levels (clipping) were not specifically exercised.

Why it matters. It converts the device into something that can be *simulated exactly*. Every accuracy number in this report — including comparisons of models that were never flashed — rests on this.

13.3.2 B2. One ring buffer, sized from measurement, that fails safe

The challenge. The device knows the keyword was said only after it ends, so it must keep recent audio; memory is scarce; the stream must never deliver corrupted audio if the network stalls.

The choice made. A single int16 ring written once per hop on core 1 and read by the streamer on core 0; positions are sample counts since boot, so the pre-roll start is `w - 9 136` and "was I overtaken?" is one subtraction; release/acquire atomics instead of a lock; an explicit OVERRUN message and a short stream when the reader is lapped. The ring was then *shrunk from 1.0 s to 0.75 s* using the study's own table, once the measured worst-case detection lag (144 ms) justified it.

Trade-offs. Only 179 ms of slack beyond the pre-roll — which is exactly why the server's blocking recogniser caused overruns (Problem 11).

Evidence. 0 overruns in 30/30 and 12/12 final streams; the 3 overruns during the server fault were all flagged, none delivered as corrupted audio; 23/23 streams bit-exact against the device's own audio. VERIFIED
Credit: the unified-ring idea and the pre-roll derivation are the study's (§8.3–§8.5); the measured sizing, index arithmetic and the evidence are this project's.

13.3.3 B3. Measuring "keyword end → server" without laboratory equipment

The challenge. The PS's latency starts at the moment the keyword *acoustically* ends — an event neither computer timestamps. The study's answer was a loudspeaker rig that raises a GPIO at a known sample, plus a logic analyser; neither was available.

The choice made. Split the path into three pieces and measure each where it is observable: (1) the device's detection time converted to server time by an NTP-style offset carried over the *same* warm TCP connection (±1.2–1.8 ms); (2) the server's receive time of the first audio packet; (3) the keyword's end located *inside the streamed audio itself* — possible precisely because the stream starts 571 ms before the detection — and measured back from the detection point. The one unobservable piece (last captured sample → decision, ≈ 10 ms) is stated as an estimate with an error bar rather than ignored.

Evidence. The method reproduced itself exactly when re-run (E-37 recomputed identically on 30 Sep) and gives the full per-activation breakdown in Figure 10.5. **Unproven:** the 10 ± 7 ms term; energy-based keyword-end location fails when a command follows with no pause (one session was unusable, E-62). **Why it matters:** it produces an L2 figure whose every component is either measured or explicitly estimated.

13.3.4 B4. Core partitioning that makes the CPU claim clean

All work that happens every 10 ms — capture, conversion, ring, features, inference, decision — runs in **one** task pinned to core 1 at priority 20; Wi-Fi, TCP, streaming and telemetry run on core 0. Besides protecting the audio path from Wi-Fi interrupt storms, this makes the "idle listening" CPU of the pipeline directly visible as core 1's busy time, and per-stage cycle counters inside that task attribute it. The study proposed this split (§4.8); the implementation simplified it from three tasks to one. VERIFIED **Trade-off:** core 1's margin (≈ 0.5 point) is thin because everything is concentrated there; **unproven:** whether moving the front end to core 0 would help overall.

13.4 Level C — Implementation contributions

13.4.1 C1. A closed-loop data engine built on the device's own mistakes

The challenge. A new keyword has no public dataset, the team had one speaker at first, and household sound caused false triggers that no synthetic data represented.

Why the ordinary approach falls short. Typical custom-wake-word pipelines mix clean clips into generic noise datasets; the model never sees this microphone, this room, or the specific sounds that fool it.

The choice made and how it works.

- **Everything is mixed into audio recorded through the product's own microphone** in the target room (beds), at randomised SNR and room gain.

- **Every example is a continuous stream with a 2.5 s lead-in**, so the front end's adaptive noise reduction and gain control are in the same state as on the always-on device when the keyword arrives.
- **Labels are placed by the keyword's end**, with "prefix" negatives that teach the model not to fire on "Hey Vy...".
- **The deployed model's own live scores** — logged for every inference during hours of real household audio — are used to find the moments it found keyword-like (score ≥ 0.30), and those moments become weighted negatives for the next model. Only declared *training* hours are mined; held-out hours stay untouched.

Evidence — including where it did not work. The live logs made v3 and v4 possible within hours. Mining quiet-night mistakes (v3) made household false triggers *worse* (19.9 vs 15.8 /h); mining household TV/talk mistakes (v4) made them *better* (9 vs 13 in the held-out hour) but hurt a new speaker's recall (14/25 vs 20/25). VERIFIED **Interpretation:** the loop works as a mechanism; its data must contain the right *kind* of negatives and more than one speaker's positives. **Unproven:** that the loop converges with more speakers — the obvious next experiment.

13.4.2 C2. Fitting the strictest RAM reading through configuration engineering

The challenge. The strict S4 reading counts IRAM code and all Wi-Fi/TCP/IP buffers, which the architecture's budget had missed; the first product build was 4.3 KB over 262 144 B and 10.5 KB over 256 000 B.

The choice made. Attribute every byte (linker sections + heap reports after each initialisation step), then remove what the product does not need rather than shrinking the model: move Wi-Fi, FreeRTOS, ring-buffer and heap code from IRAM to flash; size the arena from `arena_used_bytes()`; size the ring from the measured lag; write the model input in place; trim Wi-Fi/lwIP buffers and disable features a 32 kB/s uplink does not use (A-MPDU, IPv6, heap poisoning).

Evidence. 266 483 → 243 767 (not streaming) → 259 255 (streaming, over) → 249 615 → final 250 899 B; every step measured and recomputed. VERIFIED **Trade-offs:** code from flash competes with model weights for cache, making inference time vary with Wi-Fi traffic INFERRED; ≈ 5 kB of margin remains. **Credit:** the "network stack and RTOS, not the model, dominate" insight is the study's; the IRAM finding (C-10) and the trims are this project's.

13.4.3 C3. ASR conditioning referenced to the keyword

The challenge. Device speech arrives quiet and on top of fan rumble; a moving gain changed transcripts between near-identical takes; the recogniser's own end-of-utterance rule did not work on this audio.

The choice made. The pre-roll always contains the keyword, spoken by the same person at the same distance just before the command. The server uses it twice: to set **one fixed gain** for the whole stream (keyword speech → −20 dBFS, after a 150 Hz high-pass), and to set the **VAD threshold** relative to the keyword's level (keyword − 20 dB). Recognition runs in its own thread so it can never block reception.

Evidence. The controlled experiment (a LibriSpeech sentence: correct → "that" at device level → correct after conditioning, E-45); 30/30 clean streams after the thread change; demonstration 3/4 commands exact. **Unproven:** a systematic word-error-rate comparison with and without conditioning on real commands (only small sessions exist).

13.4.4 C4. Compliance as code

A build cannot complete if any vendor wake-word SDK or known pre-trained wake-word model is referenced anywhere in the repository (`forbidden_deps.py`, run by the build wrapper); the closed Wi-Fi/PHY libraries were listed from the linker map, not from memory; "trained from scratch" is checkable in `train.py`; two non-commercial training voices are disclosed with the retraining path. The study recommended the gate (X25); the project automated it. VERIFIED

13.5 How the contributions relate to the research

Table 13.2. Research proposal vs what the build added

Contribution	Proposed by the study	Added by this project
A1 falsifiable requirements	nested scopes, three latency definitions, kill conditions	the measurements; the IRAM correction; declared rules in files; documented departures
B1 parity	single JSON, golden-vector test, hash in model metadata	same-source host twin via ctypes; the libm table discovery and canonical tables; live = replay proofs
B2 ring buffer	unified ring, pre-roll derivation, OVERRUN	index arithmetic with atomics; resizing from measured lag; evidence under failure
B3 latency	L1/L2/L3; playback rig + GPIO	in-stream keyword-end location; explicit estimated term
B4 core split	pipeline on core 1, network on core 0	one task; per-stage cycle attribution
C1 data engine	domain-matched negatives through the demo mic; end-of-keyword labels	streams with front-end warm-up; mining the deployed model's live scores; v3/v4 evidence
C2 memory	"follow the bytes"; PSRAM off; buffer trims	IRAM finding; measured attribution; product/capture build split
C3 ASR conditioning	self-hosted Vosk, server endpointing	keyword-referenced gain and VAD; ASR thread
C4 compliance	ESP-SR gate, SBOM disclosure	automation in the build; link-map evidence

CHAPTER 14

Final System State and Limitations

14.1 Snapshot as of 30 September 2026

The complete chain — microphone, features, model, decision, pre-roll streaming, server endpointing and recognition — exists and has run on physical hardware. The product configuration has been measured for memory, idle CPU, streaming reliability and latency; the model has been tested on two real speakers and about 13 hours of real household audio. The code is at commit `9a6e700`; the firmware sources have not changed since 29 Sep 20:20.

Table 14.1. Feature and subsystem status

Feature / subsystem	Implementation status	Validation evidence	Remaining issue
I2S capture (MSB framing, mono, DMA)	complete	alignment sweep, 10 h capture with 1 gap of 20 ms (E-18, E-40)	root cause of the Philips offset not isolated
Front end (microfrontend, canonical tables)	complete	12 000/12 000 bit-exact (E-21)	CPU cost 4.5 % (fixed-point FFT, NR, PCAN)
KWS model v2b on TFLM + esp-nn	complete	1 984/1 984 outputs bit-exact (E-22)	windowed, not streaming: inference 3.9 % CPU
Decision logic (mean-3 ≥ 0.80, 2 s refractory)	complete	live = replay 27/27, 39/39, 12/12	threshold chosen on one speaker's validation session
Keyword detection accuracy	functional	22/25 held-out session, 20/25 new speaker, 12/12 T1	only 2 real speakers; misses with no pause before a command
False-trigger behaviour	functional, weak in speech	0.2 /h quiet, 15.8 /h TV + talk (E-39, E-51)	**main open problem**
Unified ring + pre-roll 571 ms	complete	min slack 169–179 ms; 0 overruns in final runs	slack ≈ 180 ms: sensitive to server/network stalls
Wi-Fi + warm TCP + clock sync	complete	30/30, 12/12 streams; offset ±1.2–1.8 ms	server address compiled in; no TLS; credentials in flash
Stream protocol (START/CONT/END/OVERRUN)	complete	23/23 bit-exact (E-44)	PCM only (no ADPCM); one stream at a time
Server ingest + VAD endpointing	complete	END by VAD in all T1 streams	rule tuned in one room
ASR (Vosk en-in-0.5, conditioning, thread)	complete	3/4 commands exact in the demo; 2/5 in a noisy test	recogniser accuracy (publisher WER 36 %)
Memory within 256 KB (strict S4)	complete	250 899 B (E-60)	≈ 5 kB margin; heap peak depends on network load
Idle CPU < 10 %	complete	9.32 % busiest core, 5.53 % both (E-60)	≈ 0.5-point margin; 10.5 % while streaming
Latency instrumentation (L1, L2)	complete	T1: 138 / 196 ms (E-63)	10 ms capture term estimated; n = 12; no L3
Telemetry and evidence tooling	complete	used for every result	—
Compliance gate, SBOM, licence	complete	gate PASS in every build	closed Wi-Fi/PHY driver (disclosed); two NC training voices
GPIO duty-cycle / detection-pulse measurement	code only	never captured	needs a logic analyser
Streaming (stateful) model	not implemented	—	PLANNED (DEC-11)
ADPCM, UDP, speculative streaming, TLS	not implemented	—	PLANNED in the study only

Feature / subsystem	Implementation status	Validation evidence	Remaining issue
Bounded-heap enforcement ("arena discipline")	not implemented	—	reporting only
Power measurement	not done	—	UNVERIFIED
Stand-alone operation (no PC on USB)	partially	—	30 s boot wait; the server still needs a PC

14.2 What is fully implemented

The firmware's capture, front end, model, decision, ring, Wi-Fi/TCP client, stream protocol, clock synchronisation and telemetry; the server's ingest, endpointing, conditioning, threaded recognition and recording; the host capture and analysis tools; the complete training pipeline (TTS generation, dataset construction, training, int8 export, device-identical evaluation, selection); and the compliance tooling.

14.3 What is partially implemented

- **Instrumentation:** the GPIO signals for duty cycle and detection exist but were never read; the run-time statistics became the primary CPU instrument.
- **Continuous integration:** the architecture's seven CI gates exist as scripts (parity, forbidden dependencies, memory from the size report) but only the dependency gate runs automatically; there is no CI server.
- **Stand-alone deployment:** the device works without the USB host after a 30 s wait, but it has no way to learn a new server address or network.

14.4 Physically tested versus tested in software only

Physically tested on the device: microphone path and framing; feature parity (device features compared with the PC); model output parity; live detection on scripted sessions (S01 sessions 2 and 3, S02, command sessions, T1, demonstration); 10 h and 2.5 h live listening; memory and CPU on the product build; streaming, overruns, latency; network outage behaviour.

Tested in software only (offline replay): every comparison between models (v0–v4) other than the one running live at the time; false-activation rates of models that were not on the device during the recording; threshold sweeps. These are legitimate *because* live-equals-replay was verified on three recordings — but they are still simulations of the device, not the device.

14.5 What has not been verified

- The capture-term portion of latency (≈ 10 ms, estimated).
- Behaviour on a congested or different network; L3 (catch-up) latency.
- Any speaker other than S01 and S02; any room other than one home; children's voices; accents beyond those represented by the TTS voices.
- Power consumption.
- Long-duration stability of the *product* build (the longest product run is 12 minutes; the 10 h run used the capture build).
- The physical board details (maker, breadboard, pull-down resistor).

14.6 Known limitations

1. **False triggers with television or conversation (15.8 /h)** — roughly one every four minutes in a lively household; each sends a few seconds of room audio to the server.

2. **Speaker coverage** — one speaker in training; one additional speaker tested.
3. **Thin CPU margin on core 1** (≈ 0.5 point idle) and > 10 % while streaming; inference time varies with Wi-Fi activity.
4. **Thin RAM margin** (≈ 5 kB on the 256 000 B reading).
5. **Fixed network configuration** — server address and Wi-Fi credentials are compiled in; no encryption or authentication.
6. **ASR quality** — the recogniser is an off-the-shelf open model; command recognition is not reliable enough for real control.
7. **Latency evidence is modest** — 12 activations, one speaker, one session.
8. **Boot behaviour** — 30 s wait without a USB host; ≈ 3 s of microphone diagnostics on every boot.

14.7 Dependencies required to run the system

Hardware: the ESP32-S3 board with the INMP441 wired as in Table 4.5; a 2.4 GHz Wi-Fi network; a PC on the same LAN. Software: ESP-IDF 5.5.5 and the pinned components to rebuild; Python 3.12 with numpy, scipy and vosk 0.3.45, and the 1 GB Vosk en-in-0.5 model, for the server; the WSL training environment only to retrain.

14.8 What would have to change for larger-scale deployment

- **Robustness of detection** across many speakers, rooms and televisions — a much larger real dataset and a held-out test set of many speakers; possibly a second-stage verifier on the server that can reject false triggers (the audio is already there).
- **Configuration and provisioning** — Wi-Fi and server settings stored in NVS and set without rebuilding; server discovery; no dependence on a USB host.
- **Security** — authenticated, encrypted transport (TLS established at boot, as the study advised, with the RAM budget re-checked); no credentials in the firmware image.
- **Fleet operations** — over-the-air updates, device identity, remote logging, crash reporting.
- **Server** — a scalable ASR service (many concurrent streams), monitoring, and a proper command-understanding layer.
- **Hardware** — a production PCB with the microphone's recommended pull-down and decoupling, and measured power in the intended power mode.

> **CAUTION**
>
> **Prototype, not product.** Everything in this report shows that the *architecture* works on the chosen hardware within the PS's limits, with honest measurements. It does not show that the device is ready for real homes: the false-trigger rate in normal household conditions, the two-speaker evidence, the fixed network configuration and the absence of security are each individually disqualifying for a product.

CHAPTER 15

Future Technical Development

Every item in this chapter is FUTURE: not implemented, not tested. Each follows from a limitation measured in Chapters 12 and 14, and is written so that it can be picked up as a work package: the limitation it addresses, the change, the subsystem, the resources, how to validate it, and what it could break.

15.1 Reduce household false triggers without losing new speakers

- **Addresses:** 15.8 false triggers per hour with TV and conversation; v4's loss of recall on a new speaker.
- **Change:** record keyword sessions from 5–10 more speakers (different ages, accents, distances) *and* several hours of household audio from more than one home; retrain the same architecture with speaker-disjoint splits; mine hard negatives from household hours only; keep positives balanced across speakers.
- **Subsystem:** data and training (`model/data`, `model/train`); no firmware change unless the operating point moves.
- **Resources:** volunteers, the existing capture tool and scripts, ≈ 1 h of training per model on the current PC.
- **Validation:** declare the rule first (as with v3/v4) and include a new-speaker criterion; report recall per speaker and false triggers per condition with exposure; confirm live-equals-replay on at least one new recording.
- **Risks:** more negatives can narrow the keyword (the v4 effect); new speakers must never leak into the test set.

15.2 Use the operating curve that already exists in the recordings

- **Addresses:** the choice of N = 3 and threshold 0.80 was made on one speaker's validation session.
- **Change:** because replay is exact, sweep the averaging length (3–10 scores), the threshold, and a two-threshold rule on the *existing* ≈ 13 h of recordings, and plot false triggers per hour against recall and against latency — the study's EXP-11 "latency-vs-FA/h operating curve".
- **Subsystem:** evaluation only (`eval_stream.py` already supports threshold and N sweeps).
- **Validation:** choose the point on validation data only; confirm on fresh recordings.
- **Risks:** longer averaging adds latency (≈ 60 ms per extra score) and more pre-roll memory.

15.3 Convert the model to stateful streaming

- **Addresses:** inference costs ≈ 3.9 % of core 1 because the whole 1 s window is recomputed every 60 ms (DEC-11); the busiest-core margin is ≈ 0.5 point.
- **Change:** convert the trained "valid-padding" network to a streaming form that keeps each layer's recent outputs and computes only the new time steps (the reason X5 was followed), then run it every 30–60 ms.
- **Subsystem:** `model/train` (conversion), `kws_engine` (state buffers), `app_main.c` (scheduling).
- **Validation:** streaming outputs must equal windowed outputs on recorded audio (a new parity test), then EXP-2 again.
- **Risks:** extra RAM for layer state (must be re-measured against the 5 kB margin); TFLM support for the required state operators.

15.4 Measure what is still estimated

- **Addresses:** the 10 ± 7 ms capture term; the unused GPIO instruments; n = 12 latency sample.

- **Change:** a logic analyser on GPIO15/GPIO16 and a loudspeaker playback rig that marks the keyword's end on a third channel (the study's IN-1/IN-2); ≥ 100 activations at several distances and volumes.
- **Subsystem:** test harness only.
- **Validation:** the hardware path and the software path must agree within their stated uncertainty.
- **Risks:** none to the product; loudspeaker playback is a different acoustic source from a person.

15.5 Measure and reduce power

- **Addresses:** power has never been measured; Wi-Fi modem sleep is disabled.
- **Change:** measure current in idle listening and while streaming (USB power meter or shunt); then test modem sleep and a lower CPU clock, re-measuring latency and CPU.
- **Validation:** current in mA with conditions; the latency and CPU tests repeated.
- **Risks:** modem sleep delays server messages (the reason it was disabled); 160 MHz would raise CPU percentages by 1.5×.

15.6 Make the device deployable

- **Addresses:** compiled-in server address and credentials; 30 s boot wait; a network change needs a rebuild (E-41).
- **Change:** store Wi-Fi and server settings in NVS with a provisioning method; discover the server (for example mDNS); remove the host-wait and the M1 probes from the product build (keep them in the capture build); add TLS established once at boot if the link leaves a trusted LAN.
- **Subsystem:** `netstream`, `app_main.c`, `sdkconfig.product`.
- **Validation:** EXP-1 and EXP-2 re-run (TLS costs heap); a network-change test; a cold-boot time measurement.
- **Risks:** TLS could consume the remaining RAM margin — the study estimated ≈ 16 kB steady state.

15.7 Bound the heap and automate the checks

- **Addresses:** the RAM claim is measured, not enforced; parity and memory checks are run by hand.
- **Change:** the study's "arena discipline" (a bounded heap region and an allocation-failure hook that reports and halts); a CI job that builds the product firmware, runs the dependency gate, checks static RAM against a ceiling, and runs the parity test on a stored recording.
- **Validation:** a deliberate over-allocation must be caught.
- **Risks:** none functionally; time to set up.

15.8 Compress the stream for poor networks

- **Addresses:** only 180 ms of ring slack beyond the pre-roll; untested on congested Wi-Fi; L3 not computed.
- **Change:** the study's IMA-ADPCM mode (4:1, negligible CPU) selectable per stream, a throughput probe, and an L3 measurement.
- **Validation:** word error rate with and without ADPCM on the same recordings; overruns and L3 under a throttled link.
- **Risks:** ASR accuracy loss; protocol complexity.

15.9 Improve command recognition on the server

- **Addresses:** the off-the-shelf recogniser's accuracy (2/5 and 3/4 in small tests).

- **Change:** a measured comparison of the open-vocabulary model, the 12-phrase grammar mode already in `ingest.py`, and a larger or adapted non-transformer model, on a fixed set of recorded streams (possible offline with `replay_session.py`).
- **Validation:** word error rate and command accuracy on held-out recordings, with n reported.
- **Risks:** a grammar rejects anything outside its list.

15.10 Prepare the model for commercial use

- **Addresses:** two TTS voices are licensed for non-commercial use only.
- **Change:** regenerate the synthetic set without `en_US-l2arctic-medium` and `hi_IN-pratham-medium` (a configuration change in `gen_tts.py`), retrain, and re-run the declared tests.
- **Risks:** fewer accented voices in training; the new model must be re-validated on the same test sets.

CHAPTER 16

Technical Glossary and Study Notes

16.1 Glossary

Each entry gives a plain definition and, where useful, what the term means *in this project*.

16.1.1 Hardware and electronics

Table 16.1. Hardware terms

Term	Meaning	In this project
SoC (system-on-chip)	processor, memory and peripherals on one chip	the ESP32-S3
Xtensa LX7	the processor core design used in the ESP32-S3	two cores at 240 MHz
SRAM / internal RAM	fast on-chip working memory, lost at power-off	512 KB on the chip; the 256 KB budget is measured here
PSRAM	external pseudo-static RAM, larger and slower	8 MB present in the package, **disabled** so it cannot hide memory use
Flash	non-volatile storage for code and constants	16 MB on the chip, 4 MB configured; holds the firmware and the model weights
IRAM	part of internal RAM that holds executable code for speed	54 591 B of code; counted in the RAM budget
Cache	small fast memory that keeps recently used flash contents	32 KB instruction + 64 KB data; model weights and flash code compete for it
DMA / GDMA	hardware that moves data between a peripheral and memory without the CPU	carries I2S audio into 4 × 10 ms buffers
GPIO	general-purpose input/output pin	GPIO4/5/6 for the microphone; GPIO15/16 for instrumentation
MEMS microphone	a microphone made with micro-machined silicon	INMP441
I2S	a 3-wire serial bus for digital audio (bit clock, word select, data)	microphone → I2S0 at 1.024 MHz
Philips vs MSB-justified framing	two I2S timings; Philips starts the data one bit after the word-select edge, MSB-justified on it	MSB-justified was measured to be the correct one here
Slot	one channel's time share in an I2S frame	the mic uses the left 32-bit slot
USB-Serial-JTAG	the S3's built-in USB port that appears as a serial port and a debugger	COM7: power, flashing, telemetry, evidence
Group delay	the fixed time a signal takes to pass through a filter	1.078 ms inside the microphone
dBFS	decibels relative to digital full scale (0 dBFS = the largest representable level)	room floor −53 dBFS, speech peaks −38…−42 dBFS

16.1.2 Embedded software

Table 16.2. Embedded software terms

Term	Meaning	In this project
ESP-IDF	Espressif's official open-source development framework	v5.5.5
FreeRTOS	a small real-time operating system with tasks, queues and priorities	tasks pinned to cores; run-time statistics give CPU use
Task / priority / pinning	an independent thread of execution; which runs first; fixing it to one core	`i2s_reader` on core 1 at priority 20
Stack high-water mark	the least free stack a task has ever had	used to shrink the reader stack to 3 584 B

Term	Meaning	In this project
Heap	memory allocated at run time	289 708 B internal heap; lowest free 178 504 B
`.bss` / `.data` / `.text`	zero-initialised globals / initialised globals / code	the parts of static memory the linker reports
sdkconfig / Kconfig	ESP-IDF's configuration system	layered defaults files; project options `PS_*`
Partition table	how the flash is divided	"single app large": a 1.5 MB application slot
Atomic (release/acquire)	memory operations that are safely visible across cores in a defined order	the ring's write index
Watchdog	a timer that resets a stuck system	IDF defaults; not tuned
NVS	ESP-IDF's key-value store in flash	used by Wi-Fi; erased once because of the old Arduino layout

16.1.3 Signal processing and features

Table 16.3. Signal-processing terms

Term	Meaning	In this project
Sample rate	samples per second	16 000 Hz
Hop / window	how far the analysis moves each step / how much audio each step looks at	10 ms / 30 ms
Hann window	a taper applied before the FFT to reduce edge effects	on 480 samples
FFT	fast algorithm for a signal's frequency content	512-point, 16-bit fixed point (KissFFT)
Mel scale / filterbank	a frequency scale matching human pitch perception / filters that group FFT bins into bands on it	40 bands, 125–6 500 Hz
Log-mel features	logarithm of the mel band energies	the model's input, 40 per 10 ms
Noise reduction (spectral)	subtracting a slowly tracked estimate of steady background per band	microfrontend, smoothing 0.025/0.06
PCAN	per-channel energy normalisation — automatic gain control per band	strength 0.95
High-pass filter	removes frequencies below a cut-off	150 Hz 8th-order on the server, against fan rumble
VAD	voice-activity detection: is someone speaking?	server energy VAD in 300–4 000 Hz ends each stream
SNR	signal-to-noise ratio	training mixes at 0–25 dB
Pass band	the frequency range a filter lets through	the microphone's ends at 6 768 Hz at 16 kHz

16.1.4 Machine learning

Table 16.4. Machine-learning terms

Term	Meaning	In this project
KWS	keyword spotting: detecting one phrase in continuous audio	"Hey Vayro"
ASR	automatic speech recognition: audio → words	Vosk on the server
CNN / depthwise-separable convolution	convolutional network / a cheaper convolution split into per-channel and channel-mixing steps	the DS-CNN model
Parameters / MACs	learned numbers / multiply-accumulate operations per inference	10 665 / 212 584
BatchNorm, ReLU, dropout, sigmoid	normalisation layer, activation, training-time regulariser, output squashed to 0–1	used in `build_model()`
Receptive field / context	how much input one output depends on	100 frames = 1.02 s
Quantisation (int8, PTQ)	storing weights and activations as 8-bit integers, calibrated after training	full-int8 model, 28 024 B

Term	Meaning	In this project
Scale / zero point	the two numbers that map int8 values back to real values	input 0.007567 / 4; output 1/256 / −128
Tensor arena	the working memory TFLite Micro uses for intermediate tensors	13 404 B used
TFLite Micro (TFLM)	TensorFlow's interpreter for microcontrollers	via esp-tflite-micro 1.4.1
esp-nn	Espressif's optimised int8 kernels	selected automatically
Streaming vs windowed inference	computing only new steps with saved state vs recomputing a whole window	windowed (DEC-11)
Posterior / score	the model's probability output	*p*; averaged over 3
Threshold, smoothing, refractory	decision level, averaging of scores, dead time after a trigger	0.80, mean of 3, 2 s
TTS	text-to-speech synthesis	Piper, 4 voices
Hard-negative mining	adding the non-keyword examples the model currently gets most wrong	from the device's own live scores
SpecAugment	masking random frequency bands and time spans during training	0–3 bands, 0–7 frames
Class weighting	weighting the rare class more in the loss	positive weight 4 in v2b
PR-AUC	area under the precision-recall curve	early stopping only
Train / validation / test split	data to learn from / to choose models and thresholds / to report results	declared in files before use
Speaker-disjoint	no speaker appears in more than one split	TTS by speaker ID; S02 never trained on

16.1.5 Testing and validation

Table 16.5. Testing terms

Term	Meaning	In this project
Parity (bit-exact)	two implementations give identical numbers	device vs host features and model outputs
Golden vector	a fixed input used to compare implementations	recorded device audio
Replay	re-running recorded audio through the same pipeline offline	`eval_stream.py`, `replay_session.py`
Recall / true-positive rate	fraction of keywords detected	22/25, 20/25
False activation (false trigger)	a detection with no keyword	0.2 /h quiet, 15.8 /h TV
FA/h and exposure	false activations per hour, and the hours observed	always reported together
Rule of three	with 0 events in *T* hours, the 95 % upper bound is ≈ 3/*T* per hour	used for short clean exposures
Confusables	words deliberately similar to the keyword	16 per session
Held-out	data never used for training or selection	night c005–c009, S01-s3, S02
Declared rule	a selection criterion written before seeing the results	`selection_rule_v*.json`
p95	the value below which 95 % of measurements fall	latency, CPU

16.1.6 Architecture, networking and measurement

Table 16.6. Architecture and networking terms

Term	Meaning	In this project
Edge / cloud split	small always-on work on the device, heavy work on a server	KWS on the ESP32-S3, ASR on a PC
Pre-roll	audio from before the trigger that is sent with the stream	571 ms

Term	Meaning	In this project
Ring buffer	a fixed-size buffer written in a circle	0.75 s of int16 audio
Overrun	the writer overtakes the reader, destroying unsent data	detected and flagged; 0 in final runs
Warm connection	a connection opened in advance so no setup delay occurs at trigger time	TCP opened at boot
TCP_NODELAY	disables delaying small packets to combine them	set on both sides
Endpointing	deciding when an utterance has ended	server VAD
Clock offset (NTP-style)	estimating the difference between two clocks from timestamped round trips	±1.2–1.8 ms
L1 / L2 / L3	detection → server / keyword end → server / keyword end → server holds live audio	L1 18 ms, L2 138 ms, L3 not measured
S0–S4	nested memory scopes from "model only" to "whole device"	S4 = 250 899 B
C1–C3	CPU scopes: pipeline on one core / all app tasks / everything on both cores	core 1 9.32 %, both 5.53 %
SBOM	software bill of materials: every component, version and licence	`docs/SBOM.md`
Blob	closed, pre-compiled binary library	Espressif Wi-Fi/PHY libraries

16.2 How to understand this project in 15 minutes

1. **The job (2 min).** Detect "Hey Vayro" on a microcontroller, then send what follows to a speech recogniser — within 256 KB RAM and 10 % idle CPU, with open-source tools and a self-trained model. Read the Executive Summary table.
2. **The data path (4 min).** Look at Figure 6.1 and Figure 6.2: 160 samples every 10 ms → int16 → ring → 40 log-mel features → every 60 ms the last 100 frames → DS-CNN → average of 3 ≥ 0.80 → stream from 571 ms earlier. Everything on core 1 in one task; networking on core 0.
3. **Why you can trust the numbers (3 min).** The device's features and model outputs are bit-identical to the PC's (Figure 13.1); therefore offline replay equals the device, and models can be evaluated on hours of real recordings.
4. **The results (3 min).** Table 12.9: 245.0 KiB, 9.32 %/5.53 % CPU, 138 ms L2, 22/25 and 20/25 recall, 0.2 and 15.8 false triggers per hour.
5. **What went wrong and how it was fixed (3 min).** One-bit I2S misalignment; maths-library rounding; IRAM code over the RAM budget; ASR blocking the network; v4 better on false triggers but worse on a new speaker → v2b shipped. The main open problem is TV/conversation false triggers.

16.3 What to study deeply

These are the concepts an evaluator is most likely to probe, in order of importance for explaining this build:

1. **Keyword spotting as a detection problem** — why recall *and* false triggers per hour matter more than "accuracy"; exposure hours; the rule of three; why the quiet and TV numbers must be quoted together.
2. **Log-mel features and the microfrontend** — window, FFT, mel filterbank, noise reduction, PCAN, log; why the filterbank stops at 6 500 Hz.
3. **Depthwise-separable CNNs and int8 quantisation** — parameters vs MACs; scale and zero point; why BatchNorm disappears at export; why 66 % of the compute is in the first layer.
4. **Train/deploy parity** — why "the same C code" was not enough (maths-library rounding) and how the canonical tables and hashes fix it.
5. **Memory on the ESP32-S3** — static sections, IRAM, heap, stacks, cache; how S4 is computed; why the Wi-Fi stack costs more than the model.

6. **FreeRTOS scheduling and CPU measurement** — tasks, priorities, core pinning, run-time statistics and their blind spot (interrupts).
7. **I2S** — bit clock, word select, slot, framing; how the alignment sweep detects a one-bit shift.
8. **The streaming protocol and pre-roll** — ring indices, overrun, warm TCP, clock-offset estimation, how L2 is decomposed.
9. **Data engineering for a new keyword** — TTS spelling, mixing into device-recorded beds, end-of-keyword labels, speaker-disjoint splits, hard-negative mining from live scores, declared selection rules.
10. **The trade-offs that were chosen consciously** — windowed vs streaming model; smaller ring vs slack; Wi-Fi power save off; v2b over v4.

APPENDIX A

Source and Evidence Index

A.1 Documents

Table A.1. Primary documents

Document	Location	Role
Problem statement	`D:\sih final\sih 2026 (1).pdf`	the only binding source
Architecture study (Track C)	`D:\sih final\PS26172_TRACK_C_HARDWARE_ARCHITECTURE.md`	research basis (Chapter 2)
Project state record	`docs/PROJECT_STATE.md`	requirements R1–R10, decisions DEC-01…26, conflicts C-01…12, milestones
Evidence ledger	`docs/EVIDENCE_LEDGER.md`	E-01…E-63 with raw-file paths
SBOM	`docs/SBOM.md`	components, versions, licences, disclosures
M1 experiment plan	`docs/experiments/M1_audio_path.md`	M1 hypotheses and protocol
Consistency sheet	`docs/submission/EVIDENCE_CONSISTENCY_SHEET.md`, `approved_numbers.json`	audit of claims (30 Sep)
Selection rules and outcomes	`model/artifacts/selection_rule_v2/v3/v4.json`, `selection_v*_outcome.json`, `selection_final.json`	model choice history
Report figure data	`docs/report/figures/figure_data_summary.json`	values computed by `make_figures.py` for this report

A.2 Evidence ledger index

Table A.2. Ledger entries and where their raw data is (paths under `evidence/` unless stated)

ID	What it establishes	Raw data
E-01	pinned build, forbidden-dependency gate PASS	`m0/size_uart_m1build.txt`
E-02	live 24-bit 16 kHz audio through I2S0	`runs/20260926_214931_m1_msb_settle_warm`
E-03	pass-band edge visible at 6 768 Hz	same
E-04	no zero-output start-up; 0.56 ms to first sample	`runs/20260926_221709_m1_sessionA_cold`
E-05	mono mode halves DMA traffic	`@DIAG` lines, M1 runs
E-06	speech vs room levels; int16 shift decision	`runs/20260926_221709_m1_sessionA_cold`
E-07	capture integrity (short runs)	`runs/20260927_125547_m1_link_fix_60s_b`
E-08…E-16	early plan rows, superseded or verified on 30 Sep (see E-14, E-15)	pointers in the ledger
E-14	trained from random initialisation (code review)	`model/train/train.py`
E-15	no voice SDK; closed Wi-Fi/PHY libraries listed	`final_product_build_v2b.log`, link map
E-17	static RAM of the M1 build (IRAM code 60 015 B)	`m0/size_uart_m1build.txt`
E-18	Philips one bit late; MSB aligned	`runs/20260926_214456_m1_align_sweep_warm`
E-19	start-up DC transient; in-band settling < 100 ms	M1 runs

ID	What it establishes	Raw data
E-20	capture-link faults found and fixed	several M1 runs
E-21	feature parity: fail 5 995/5 998 → pass 12 000/12 000	`runs/20260927_134200…`, `…145754…`, `…150156…`
E-22	model output parity 1 984/1 984	`runs/20260927_145754_m2m3_parity_canonical_120s`
E-23	per-stage CPU cycles (capture build)	same, `posterior_parity.json`
E-24	model footprint	`@KWS`, `model/artifacts/v0/metrics.json`
E-25	v0 offline on S01 session 1	`model/artifacts/v0/eval_test_S01.json`
E-26	live triggers = offline replay (27/27)	`runs/20260927_151127_kw_S01_s2/device_trigger_eval.json`
E-27	v0 live on S01 session 2	`model/artifacts/v0/eval_v0_val_s2.json`
E-28, E-31	0 FA in 401 s daytime held-out (v0, v2b)	`…/eval_*heldout_c002.json`
E-29	static RAM of the network capture build	`m5_size_netcheck.txt`
E-30	v2b validation result and selection	`model/artifacts/v2b/eval_val_s2.json`
E-32	Wi-Fi + warm TCP bring-up	`runs/20260927_192653_m5_net_bringup`
E-33	corrected: 150 s session without a keyword	`runs/20260927_192738_m5_live_kw`
E-34	Wi-Fi interference hypothesis rejected (A/B)	`runs/20260927_193239_ab_wifi_OFF`, `…193322_ab_wifi_ON`
E-35	**S01 session 3: 22/25, 2/16**	`runs/20260927_195253_kw_S01_s3`, `model/artifacts/v2b/eval_test_S01_s3.json`
E-36	24/24 streams with pre-roll, 0 overruns	`server/20260927_192112/`
E-37	first latency (95 / 136 ms)	`server/20260927_192112/latency_report.json`
E-38	10 unattended detections, audio lost (unresolved)	`server/20260927_192112/activations.csv`
E-39	overnight false activations by activity	`runs/20260927_203133_ambient_night_20260927`
E-40	10 h capture integrity	same, `capture_summary.json`
E-41	network outage behaviour	same, `device_log.txt`
E-42	EXP-1 product, not streaming (238.1 KiB)	`exp1_product_build_v2.log`, `runs/20260928_075330…`
E-43	EXP-2 first CPU	`runs/20260928_080133_exp2_cpu_selftest`
E-44	23/23 streams bit-exact	`server/20260927_192112/stream_integrity.json`
E-45	first ASR session (corrected) + level control	`server/20260928_175052/`, `m6_asr_control/`
E-46	190 ms L1 artefact diagnosed	`server/20260928_175052/activations.csv`
E-47	false activation from devotional singing	`server/20260928_175052/c175129_act0005.wav`
E-48	M6 command test, small ASR: 1/4	`runs/20260928_192727_m6_cmd_S01`
E-49	M6 v2 with en-in-0.5 + VAD: 2/10 end to end	`server/20260928_201836/`
E-50	capture stalls under host load	`household_eve.stdout.log`
E-51	**household FA: v2b 15.8 /h, v3 19.9 /h**	`model/artifacts/household_compare_20260928.json`
E-52	streaming peak 259 255 B	`exp1_streaming_size.log`, `runs/20260928_222620…`
E-53	after DEC-24: 249 615 B	`exp1_streaming_size_dec24.log`, `runs/20260928_225547…`
E-54	v4 built	`model/artifacts/v4/`
E-55	v4 selection rules a–c	`model/artifacts/selection_v4_outcome.json`
E-56	EXP-2, 12 minutes	`runs/20260928_230150_exp2_cpu_idle_12min_dec24`
E-57	v4 rule (d): 10/10 vs 10/10	`model/artifacts/prompt_hits_20260929_192230_m6_v3_S01.json`

ID	What it establishes	Raw data
E-58	final v4 firmware; 3/31 overruns → AsrJob	`runs/20260929_194642_final_v4_streaming`, `server_m6f.stdout.log`
E-59	**new speaker: v4 14/25, v2b 20/25**	`runs/20260929_201213_kw_S02_s1`, `model/artifacts/*/eval_newspk_S02_s1.json`
E-60	**final firmware: 245.0 KiB, 30/30, idle CPU**	`runs/20260929_202327_final_v2b_confirm`, `final_product_size_v2b.log`
E-61	demonstration run	`demo/PS26172_demo_90s/cut_meta.json`
E-62	latency with ASR in the loop	`server/20260929_{214632,221342,195557}/latency_report.json`
E-63	**T1: 12/12, L2 138 / 196 ms**	`t1/20260930_085535/`, `server/20260930_085548/latency_report.json`

A.3 Decisions index

Table A.3. Implementation decisions (from `docs/PROJECT_STATE.md` §4)

ID	Decision
DEC-01	ESP-IDF v5.5.5 native, not PlatformIO
DEC-02	repository at `D:\ps26172` (no spaces in the path)
DEC-03	CRC-framed USB serial as the M1 debug/data link
DEC-04	capture keeps the full 24-bit sample; host derives int16 identically
DEC-05	unified ring built at M2
DEC-06	**I2S framing = MSB-justified** (measured)
DEC-07	native USB-Serial-JTAG console; firmware waits for the host start byte
DEC-08	keyword "Hey Vayro"
DEC-09	int16 = bits 23:8 kept
DEC-10	MIC_SETTLE 300 ms kept
DEC-11	**windowed DS-CNN every 60 ms instead of a streaming model** (user-approved)
DEC-12	training environment: WSL Ubuntu, uv, CPython 3.12, TensorFlow CPU, Piper, LibriSpeech
DEC-13	ambient recording afternoon + overnight through this microphone
DEC-14	TTS spellings "Hey Vyro" / "हे वाइरो"
DEC-15	microfrontend vendored unmodified; all parameters from `frontend.json`
DEC-16	**canonical front-end tables copied from the host twin**
DEC-17	static batch-1 export (5 operator types)
DEC-18	selection needs a real validation set separate from the test set
DEC-19	v2 split: S01 s1 train, s2 validation, later sessions test
DEC-20	recording protocol v2 and prompt scoring
DEC-21	transport as specified: warm TCP, 8-byte header, 571/624 ms pre-roll, clock pings; modem sleep off
DEC-22	**product configuration** (memory trims, 0.75 s ring, 571 ms pre-roll, IRAM → flash)
DEC-23	v3 not shipped (household FA worse)
DEC-24	**further product trims** (heap poisoning, A-MPDU, TX buffers, IPv6)
DEC-25	v4 trained with a rule declared first
DEC-26	**ship v2b @ 0.80, overriding v4's rule** (new-speaker evidence)

APPENDIX B

Discrepancies Found During This Reconstruction

Cross-checking the records against the code and raw data found the following. None changes a headline number; they are listed so that no reader is misled by an out-of-date file. (The larger set of presentation-wording contradictions audited on 30 Sep, B1–B14, is in the consistency sheet and was resolved there.)

Table B.1. Discrepancies and stale records

#	Where	What it says	What the evidence shows	Verdict
1	`PROJECT_STATE.md` header	"Last updated 2026-09-26 ~21:00"	the file contains 30 Sep content	stale header
2	`PROJECT_STATE.md` §8, M3 row	"v3 @ 0.60 is the candidate; ship decision pending (DEC-23)"	DEC-23 resolved 28 Sep; DEC-26 ships v2b	stale row
3	`PROJECT_STATE.md` §8, M4 row	"logic analyser arrives 28 Sep"	no analyser was ever used (C-06)	stale
4	`PROJECT_STATE.md` §5, C-11	proposes a 2–3 s MIC_SETTLE	withdrawn by DEC-10	stale row
5	`docs/experiments/M1_audio_path.md`	"Results: NOT RUN"	M1 was run and is in E-02…E-07, E-18…E-20	never updated
6	`Kconfig.projbuild` help, `sdkconfig.product` comment	the capture TX pool "shrinks to 4 slots"	`app_main.c`: `POOL_N 1` when capture streams are off	comment inaccurate
7	`sdkconfig.defaults` comment	"Wi-Fi / lwIP trims are added at M5"	they live in `sdkconfig.product`	comment stale
8	`SBOM.md` header	"Status 28 Sep"	content updated 29 Sep (v2b, Apache-2.0)	stale date
9	Ledger E-60	idle CPU only	the same run's streaming phase: core 1 10.51 % mean, 10.83 % max (this report, Figure 10.4)	**new finding**, not a contradiction
10	Ledger E-56	core 1 p99 8.97 %	recomputed p99 8.94 % (nearest-rank); mean and max identical	percentile-method difference only
11	`DEMO_RUNBOOK.md`	"ASR model load ~30 s"	11.7 s in the T1 run	condition-dependent
12	Architecture study §B5	pins BCK 4 / WS 5 / SD 6	wired BCK 6 / WS 5 / SD 4 (C-05)	documented deviation
13	Architecture study §3.5	S4 ≈ 227 KiB	245.0 KiB measured; IRAM code missing from the estimate (C-10)	estimate vs measurement

APPENDIX C

Completeness Audit

Table C.1. The report's quality-control checklist, with the result of each check

Check	Result
The original research document has been examined	**Yes** — the Track C study was read in full; the PS text read in full. The documents it cites (Master Spec, Tracks A/B, datasheets) were not available and are marked as such.
The actual project requirements are documented	**Yes** — R1–R10 from the PS, with interpretations (Chapter 3).
The actual hardware inventory is complete	**Yes, with gaps** — chip identity measured; board maker/model, breadboard, pull-down and decoupling not recorded (Chapter 4).
Important component specifications are accurate	**Yes** — measured values where they exist; datasheet values quoted via the study and labelled DOCUMENTED.
Relevant wiring and interfaces are documented	**Yes** — from `sdkconfig.board`, confirmed by measurement.
Software tools and confirmed versions are included	**Yes** — from lock files, logs and build caches; four versions could not be confirmed and are listed.
The actual architecture is explained	**Yes** (Chapter 6).
Repository structure and important source files are documented	**Yes** (Chapter 7).
Important code paths are explained	**Yes** — boot, reader task, decision, ring, front end, KWS engine, streamer, server.
Relevant algorithms and processing stages are documented	**Yes** (Chapter 8).
Hardware-software integration is explained	**Yes** (Chapter 9).
Deployment and flashing details are documented where evidence exists	**Yes** — the final flash is evidenced by the firmware's own ELF hash, not by a flash log (stated).
The chronological development process is reconstructed	**Yes** (Chapter 10, Figure 10.1).
Important experiments and tests are documented	**Yes** — 33 tests, including failures and corrections.
Failed tests and debugging iterations are included	**Yes** (Chapters 10–11).
Final results are separated from projections	**Yes** — study estimates appear only in comparison tables, labelled as estimates.
The actual innovations are analysed in depth	**Yes** (Chapter 13), with ordinary choices listed separately and credit to the study stated.
Remaining limitations and unverified claims are identified	**Yes** (Chapters 12, 14; UNVERIFIED labels).
Every important factual statement has an identifiable source	**Yes, where possible** — file paths and ledger IDs throughout.
No unrelated material is included	**Yes** — presentation and video content excluded.
No PPT or demo-video content is reproduced as a deliverable	**Yes.**
The report accurately describes the final implemented state	**Yes** — final configuration identified by model SHA-256, firmware ELF hashes and configuration files.

Limits of this reconstruction. Results labelled DOCUMENTED were taken from the ledger without re-running their analysis; all headline numbers were re-computed. The reconstruction could not observe the physical set-up, so everything about the room, distances and the board's construction rests on the records.

APPENDIX D

Command Reference

```
# Build and flash the final product firmware (from any directory; COM7 = the board's native USB)
powershell -ExecutionPolicy Bypass -File D:\ps26172\tools\idf.ps1 -Console usbjtag -Port COM7 `
    -ExtraDefaults D:\ps26172\firmware\sdkconfig.product -BuildTag product build flash

# Regenerate the model source + operating point from a trained model folder
D:\ps26172\.venv-host\Scripts\python.exe D:\ps26172\tools\gen_model_source.py model\artifacts\v2b

# Start the ASR server (final configuration)
D:\ps26172\.venv-host\Scripts\python.exe -u D:\ps26172\server\ingest.py --port 5005 `
    --asr D:\ps26172_ext\vosk\vosk-model-en-in-0.5 --min-after-s 0.8 --stream-s-max 6

# Device console / evidence recording (resets the board, writes evidence\runs\<time>_<label>\)
D:\ps26172\.venv-host\Scripts\python.exe D:\ps26172\host\capture.py --port COM7 --wall-s 600 --label my_run

# Scripted keyword session (prompts on screen, audio + features + scores recorded)
D:\ps26172\.venv-host\Scripts\python.exe D:\ps26172\host\capture.py --port COM7 `
    --script-file host\scripts\kw_session_v2.txt --label kw_S03_s1

# Latency report and stream integrity for a server session
D:\ps26172\.venv-host\Scripts\python.exe D:\ps26172\server\latency_report.py evidence\server\<session>
D:\ps26172\.venv-host\Scripts\python.exe D:\ps26172\server\stream_integrity.py evidence\runs\<capture run>
evidence\server\<session>

# T1 latency test (starts the server and console, shows 12 prompts)
D:\ps26172\.venv-host\Scripts\python.exe D:\ps26172\host\t1_session.py
```

```
# In WSL (training environment ~/.venvs/ps26172)
python model/frontend/parity_test.py /mnt/d/ps26172/evidence/runs/<run>        # device vs host features
python model/eval/eval_stream.py --model model/artifacts/v2b/model_int8.tflite \
       --runs /mnt/d/ps26172/evidence/runs/<run> --tag my_test                   # device-identical replay
bash model/train/run_v4.sh                                                       # full dataset + train +
export (v4 recipe)
```

```
# Rebuild this report (figures from evidence, then HTML + PDF)
D:\ps26172\.venv-host\Scripts\python.exe D:\ps26172\docs\report\make_figures.py
D:\ps26172\.venv-host\Scripts\python.exe D:\ps26172\docs\report\build_report.py
```